

# Network Management and Monitoring

**AI5049, Winter Semester 2026/27**

12 October 2026

Lucas Immanuel Nickel

Fulda University of Applied Sciences  
Network Operations Engineer @ DE-CIX

# About me

Lucas Immanuel Nickel

- Network Operations Engineer at DE-CIX
- Research Associate at Fulda University of Applied Sciences
- M.Sc. Applied Computer Science, research stay at the University of Toronto

# What is this course about?

We work on real network problems and the evidence needed to solve them.

- Find what is actually broken
- Make changes without making things worse
- Measure whether the service works
- Turn operational questions into science

# Our lens: large IP networks

We look at networks that carry traffic between organizations and regions.

- ISP and carrier backbones
- transit, peering and IXPs
- interdomain routing
- automation and monitoring at scale

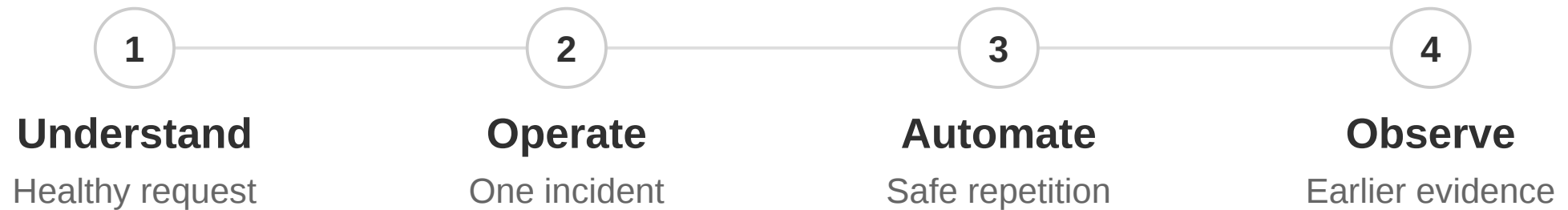
The same principles apply elsewhere. Our operating context is the Internet.

# The technical block

| Date            | Time        | Topic                       |
|-----------------|-------------|-----------------------------|
| 12 October 2026 | 13:45–17:00 | Networking 101 + Operations |
| 13 October 2026 | 08:45–15:15 | Automation + Incidents      |
| 14 October 2026 | 08:45–15:15 | Monitoring / Observability  |

**Incidents close day 2, after the Automation workflow.**

# One lab carries the technical block



The topology stays familiar. The question changes.  
The slides remain the workshop guide.

# The sessions after the lab

Friday sessions: **13:45–18:45**

| Date             | Focus                               |
|------------------|-------------------------------------|
| 30 October 2026  | Research questions and sources      |
| 13 November 2026 | Experimental design                 |
| 27 November 2026 | Analysis and scientific argument    |
| 11 December 2026 | Forecasts, control loops and agents |
| 22 January 2027  | In-class peer review and revision   |
| 5 February 2027  | Presentations and reproducibility   |

# Fridays: work on your project

The shared part gives you a method. Most of the session is project work.

- Short input together
- 45 minutes of shepherding with me per team
- Code, data, experiments or writing
- From November: brief team updates

Bring blockers early. We can fix scope before it becomes a deadline problem.



# Your project

One network-management question, teams of 3-4.

- Build a system or analyze data
- Compare against a baseline
- Collect evidence for one limited claim

**Paper:** six pages including references. English, IEEE two-column.

## Some project ideas

| Area              | Possible comparison                     |
|-------------------|-----------------------------------------|
| Monitoring        | Polling vs streaming for short failures |
| Digital twins     | Emulation vs modeling before deployment |
| Automation        | Automated vs manual emergency repair    |
| AI for networking | LLM troubleshooting vs a runbook        |

Bring your own question if you can evaluate it.

## Research venues close to this course

| Venue                 | Typical contribution                            |
|-----------------------|-------------------------------------------------|
| <u>IEEE/IFIP NOMS</u> | operating, managing and automating networks     |
| <u>CNSM</u>           | management systems, telemetry and orchestration |
| <u>ACM IMC</u>        | measuring and understanding Internet behavior   |

Start here for literature on **management, monitoring and measurement**.

# What counts towards your grade?

Individual work in the **paper and artifact** is assessed.

| Paper area                          | Weight |
|-------------------------------------|--------|
| Introduction & related work         | 20%    |
| Approach & methodology              | 25%    |
| Evaluation & results                | 30%    |
| Discussion & conclusion             | 15%    |
| Abstract, writing & reproducibility | 10%    |

**20 + 25 + 30 + 15 + 10 = 100%.**

# Who did what?

Record individual and joint work in DECLARATION.md.

| Record     | Make it concrete                           |
|------------|--------------------------------------------|
| Paper      | Sections, figures and revisions            |
| Artifact   | Code, configuration and experiments        |
| Joint work | Shared task and each person's contribution |

Point to the work so the contribution can be checked.

# What goes with the paper?

The paper makes the argument. The **artifact** lets me check it.

- Code and configurations
- Data and software versions
- Reproduction steps
- Seeds, failed runs and exclusions

Public repository: optional. Access for the examiner: required.

# Scientific integrity

Follow the IEEE publishing ethics requirements.

- Cite ideas, methods, data and figures
- Check references against the original
- Report what you actually measured
- Keep counter-evidence
- State limitations and uncertainty

A result against your hypothesis is still a result.

# AI use and disclosure

Follow the IEEE policy on AI-generated content.

- Spelling and grammar fixes only: disclosure optional
- Generated or substantially rewritten text: disclose
- Generated figures or code: disclose
- Name the system, affected parts and extent in the **Acknowledgments**

You are responsible for the references, code and technical claims.



# Dates and deadlines

| Date                      | Deliverable / session                     |
|---------------------------|-------------------------------------------|
| 11 January 2027, 23:59    | Full draft + supporting material → Moodle |
| 22 January 2027, in class | Read, write and hand in the paper review  |
| 5 February 2027           | Presentations in class (ungraded)         |
| 12 February 2027, 23:59   | Paper, artifact, declaration → Moodle     |

## Optional peer-review bonus

One qualifying review improves the individual grade by **one step**.

Example: **2.3** → **2.0**.

The best possible grade is **1.0**. Failing grades remain unchanged.

The review should fit on **one page**.

Read in class. Write **legibly by hand**. German or English accepted.

# What makes a qualifying review?

A qualifying review covers four things:

- **Summary:** question and main result
- **Method:** assess a methodological choice
- **Judgment:** strength and limitation with reasons
- **Revision:** propose a feasible improvement

I record the four criteria and my reasons.

# Where are we starting?

Five minutes. Work alone first, then compare in pairs.

| Topic                  | Used before | Can explain | New to me |
|------------------------|-------------|-------------|-----------|
| IP, routing and DNS    |             |             |           |
| Linux and logs         |             |             |           |
| Python, APIs and Git   |             |             |           |
| Papers and experiments |             |             |           |

Add one thing you want to get better at.

# Administration ends here.

From now on, **one network carries the technical block.**

The concepts travel further than the lab.

# One network. Four questions.

| Module         | The question                                 |
|----------------|----------------------------------------------|
| Networking 101 | How does this network actually work?         |
| Operations     | What broke, and how do we restore it safely? |
| Automation     | How do we repeat that change safely?         |
| Observability  | How could we detect and explain it first?    |

**understand → operate → automate → observe**

## Then challenge the evidence.

**observe → question → measure → research**

Research tests what our observations can establish.

The project turns that evidence into a paper and artifact.

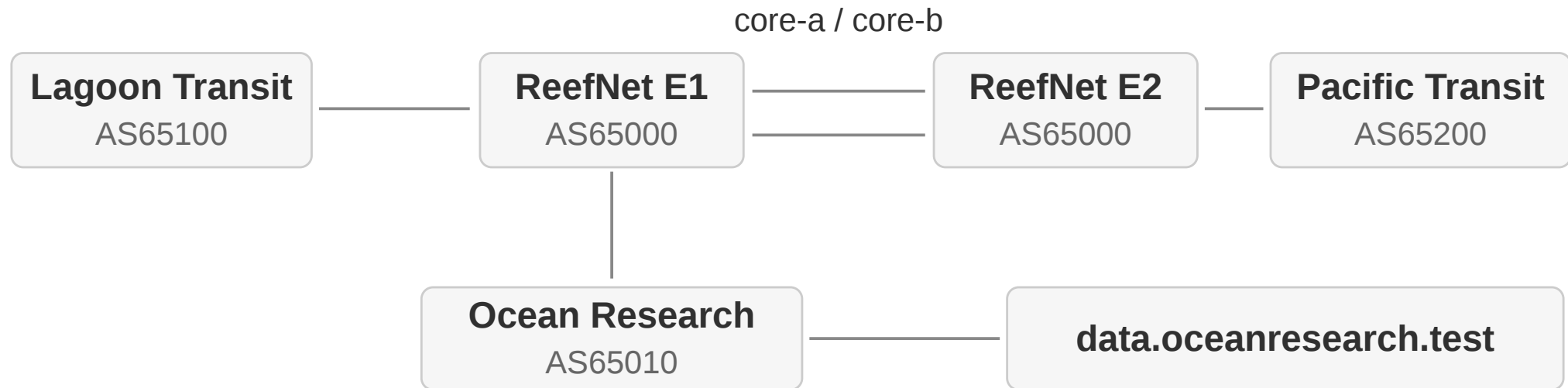
Peer review tests the argument.

# Meet ReefNet at BOB1

**ReefNet** operates BOB1 on Bora Bora.

**Bora Bora Ocean Research** needs its data service reachable.

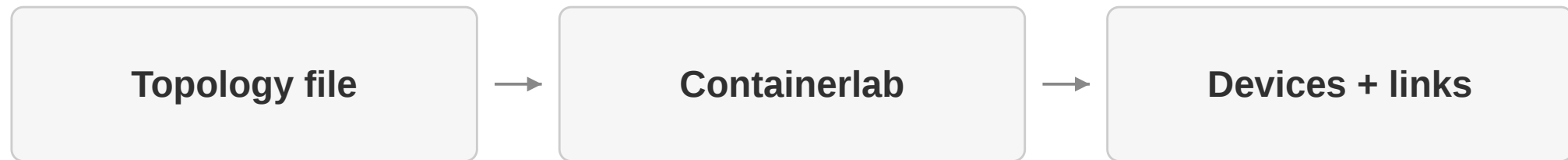
**Lagoon Transit** and **Pacific Transit** are our upstreams.





# Containerlab creates the network.

It creates the topology, virtual devices and links we operate.



Creation is not verification.

We will recreate the same baseline at each technical session.

# NetBox is not the router.

**NetBox describes what we intend to exist.**

Inventory connects devices, interfaces, circuits and addresses.  
Follow those relationships before choosing a change target.

In BOB1, trace the Lagoon circuit to its ReefNet interface.

An inventory record cannot tell you whether packets pass now.

# The router shows the present.

The CLI shows what is **configured and operational now**.

| Configured             | Operational                 |
|------------------------|-----------------------------|
| Interface enabled      | Carrier up or down          |
| Export policy attached | Prefix advertised or absent |

Keep the observation time. “Now” does not stay current.

# Grafana shows observations over time.

Check the age and scope of the data behind a panel.



A panel is the end of an observation pipeline.

A green panel can contain old data.

## Look from outside.

An external probe shows the service from **another vantage point**.

Lagoon and Pacific probes can see different outcomes.  
Neither sees every possible customer path.

**The customer is not your monitoring system.**

## Four views. Four questions.

| View           | Question                                      |
|----------------|-----------------------------------------------|
| NetBox         | What should exist?                            |
| Router / CLI   | What exists now?                              |
| Grafana        | What has been happening over time?            |
| External probe | What does the service look like from outside? |

Disagreement is evidence. It is not a reason to trust the prettiest screen.

## Prepare the lab once.

Before the first session, open PowerShell: `ws1 -d Containerlab`

Run the following commands **inside WSL**:

```
cd ~  
git clone https://github.com/Stinktopf/network-management-and-monitoring.git  
cd network-management-and-monitoring  
make setup
```

Wait for **READY FOR CLASS**. All four scenarios are checked.

The runtime stops. NetBox stays prepared. Diagnostics: `.state/`.

# Know the three reset levels

| Command                 | Use it when                               | What it keeps                     |
|-------------------------|-------------------------------------------|-----------------------------------|
| <code>make reset</code> | you changed or broke the current exercise | prepared NetBox data              |
| <code>make down</code>  | you are finished for now                  | NetBox data and downloaded images |
| <code>make clean</code> | you want a complete AI5049 factory reset  | downloaded base images only       |

After `make clean`, run:

```
make setup
```

`make clean` is **project-local**. It does not run a global Docker prune.



# 12 October 2026

## Networking 101

How does this network actually work?

Follow **one service request**, then its response.

# Open the lab

In the **lab-directory terminal**, start the healthy network:

```
make networking
```

Optional: run `code .` here to open the lab folder in VS Code.  
The Containerlab extension opens `lab.clab.yml` in TopoViewer.

Then enter the Lagoon probe:

```
make enter NODE=probe01.bob1.lagoontransit.test
```

# Follow one service request.

In ReefNet: a Lagoon probe requests `data.oceanresearch.test`.

One request crosses several independently operated links.



What must work before the application can return a response?

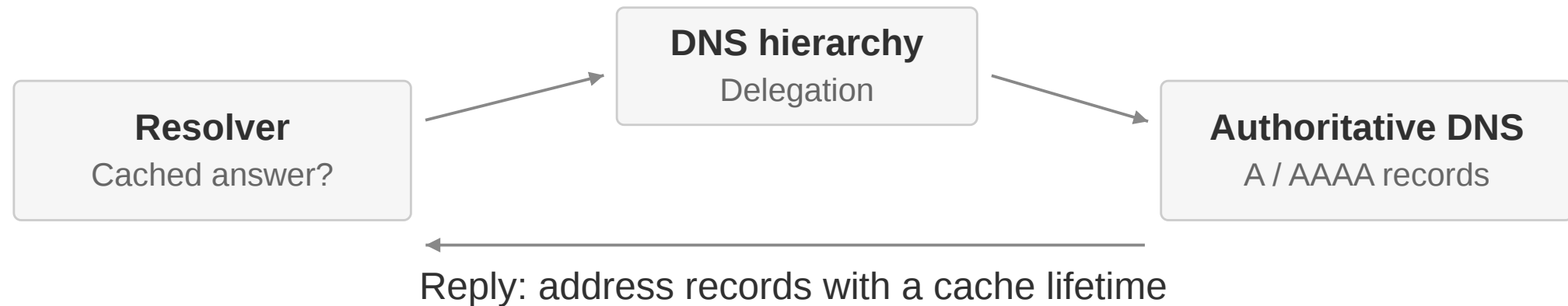
APPLICATION → TRANSPORT → IP → ROUTING → LINK → PATH

# The TCP/IP stack

|                    |                      |
|--------------------|----------------------|
| <b>Application</b> | HTTP, DNS            |
| <b>Transport</b>   | TCP, UDP             |
| <b>Internet</b>    | IP, ICMP             |
| <b>Link</b>        | Ethernet             |
| <b>Physical</b>    | Copper, fiber, radio |

Each layer carries data for the layer above it.

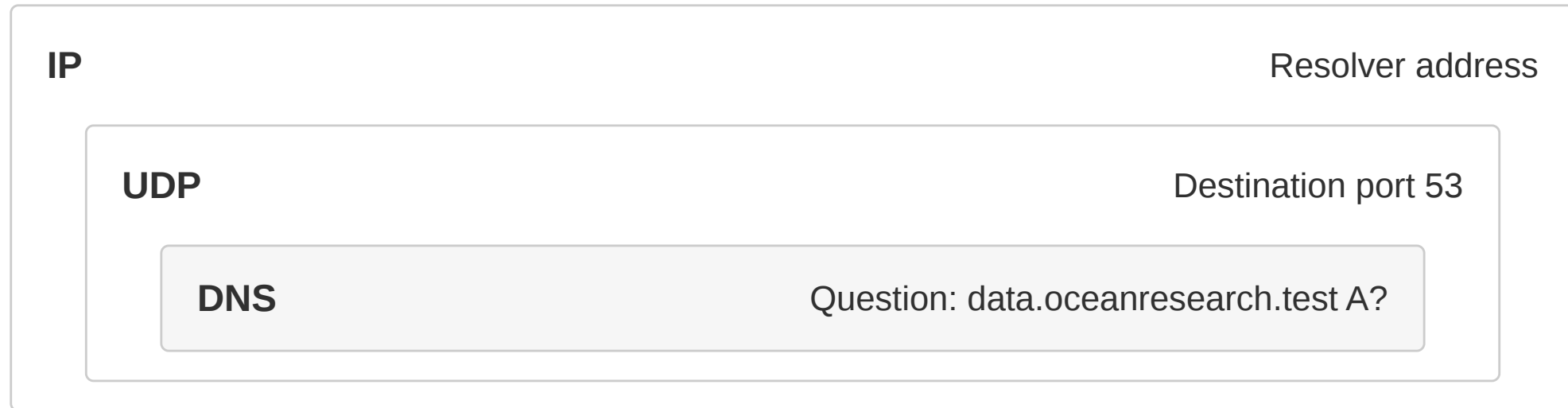
# A name needs an address.



An address is an answer about naming, not service health.

A records carry IPv4 addresses. AAAA records carry IPv6 addresses.  
ReefNet probes query the lab DNS server directly.

# A DNS query becomes transport data



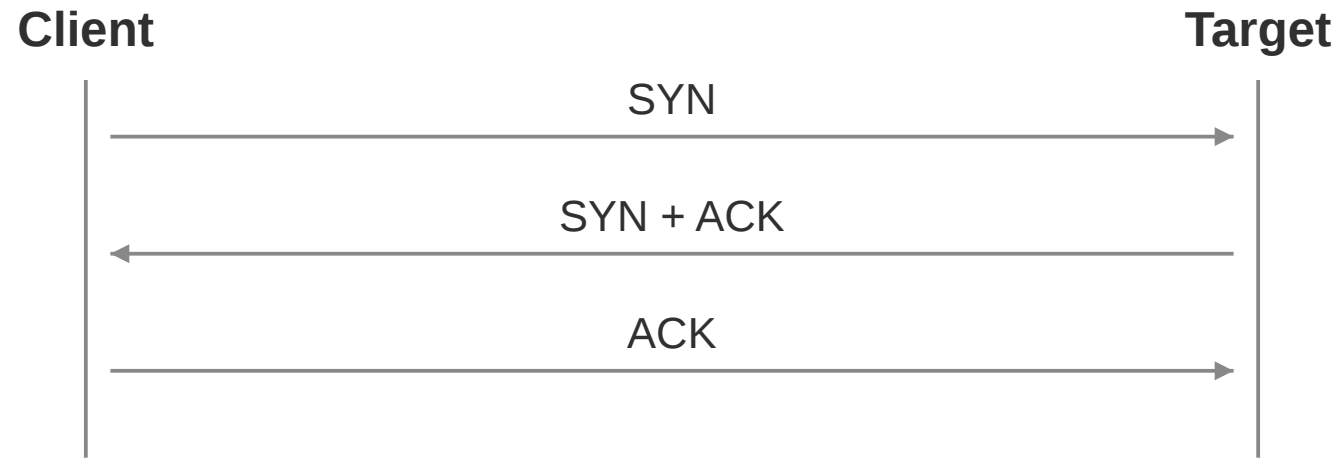
This query uses UDP. DNS also supports TCP.

# TCP and UDP

|                | TCP                 | UDP                  |
|----------------|---------------------|----------------------|
| Data           | Ordered byte stream | Individual datagrams |
| Connection     | Required            | None                 |
| Retransmission | Built in            | Application decides  |

Both use IP addresses and ports. The application chooses the transport.

# TCP gets the conversation started.



TCP setup precedes the application response.

After DNS, our HTTP request opens a TCP connection.  
TCP orders bytes, retransmits loss and controls congestion.  
A completed handshake is not an application response.

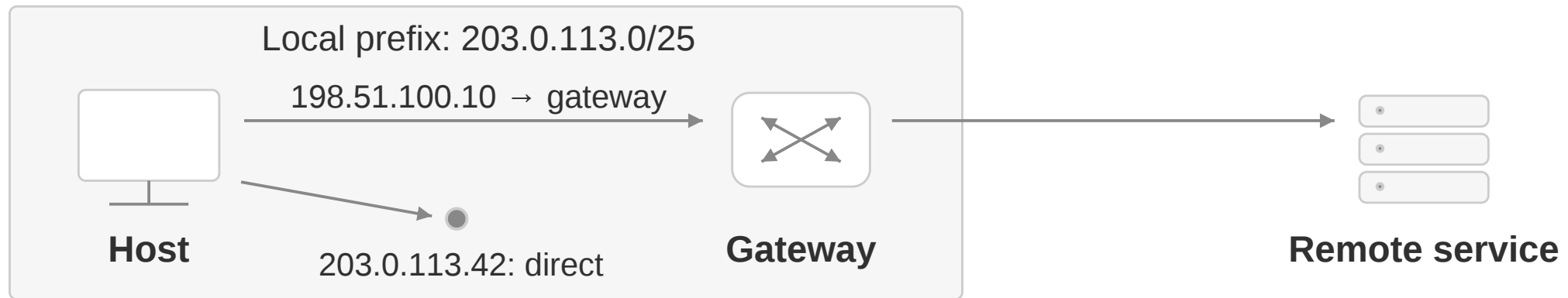


# IP gives the packet a destination.



IPv4 uses 32-bit addresses. IPv6 uses 128-bit addresses.  
Routers choose a path from the destination IP address.

# Is the destination local or remote?



A local destination is reached directly. A remote one needs a router.  
Use `ip route` and `ip -6 route` to inspect the host's decision.

# The route selects a next hop.



The packet keeps its final IP destination.

Resolve the next hop on the selected outgoing link.

# The most specific route wins

A `/25` matches the first 25 address bits. Illustrative lookup table:

| Prefix                         | Next hop               |
|--------------------------------|------------------------|
| <code>0.0.0.0/0</code>         | <code>192.0.2.1</code> |
| <code>198.51.100.0/24</code>   | <code>192.0.2.2</code> |
| <code>198.51.100.128/25</code> | <code>192.0.2.3</code> |

Destination: `198.51.100.200`

The `/25` is the **longest matching prefix**.

Forwarding chooses from routes that are already installed.

## Now we need Layer 2

IP chooses the **next hop**.

On Ethernet, send a frame to that next hop's **MAC address**.

Neighbor discovery connects these two decisions.

# MAC addresses belong to a link.

An Ethernet frame carries source and destination MAC addresses.

On the Lagoon probe:

```
ip -br link
```

MAC addresses are normally **48 bits** and identify local-link interfaces.

`ff:ff:ff:ff:ff:ff` is the Ethernet broadcast address.

For a remote IP destination, use the gateway's MAC address.

# Ethernet carries frames across one link



A frame belongs to one link. Its FCS detects corruption.

## Bits need a physical link.

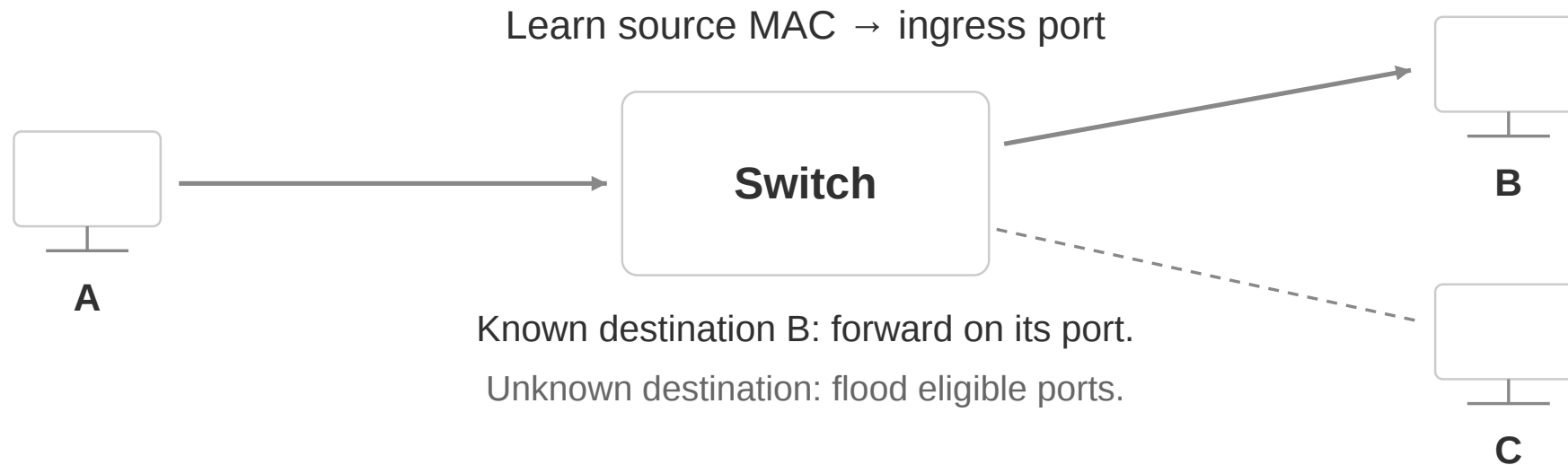
| Medium | Signal     | Fault evidence                   |
|--------|------------|----------------------------------|
| Copper | Electrical | Link loss, errors, cable damage  |
| Fiber  | Light      | Loss of light, low receive power |

A route cannot repair a dirty connector.

BOB1 has virtual links. Optical diagnostics need physical equipment.

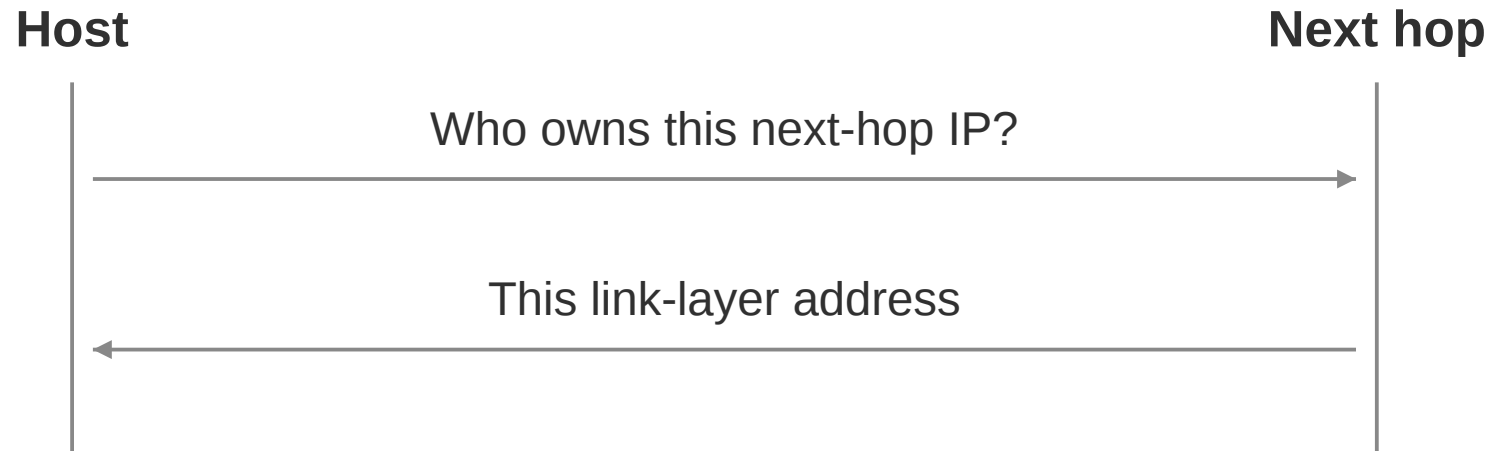


# A switch forwards frames by MAC address



Broadcast and multicast handling depend on the link and switch configuration.

# Resolve the next hop.

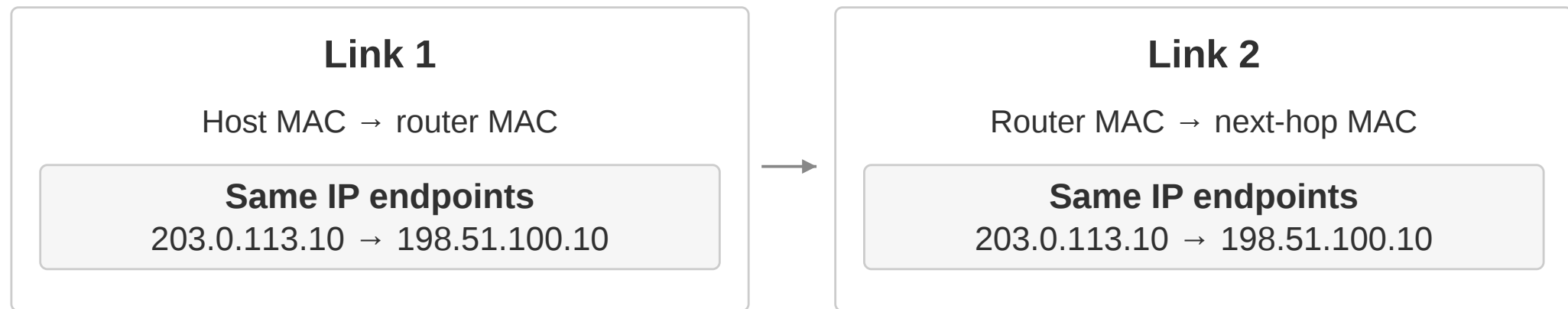


ARP uses broadcast. ND uses multicast solicitation.

IPv4 uses **ARP**. IPv6 uses **Neighbor Discovery**.

Resolve the next hop on this link.

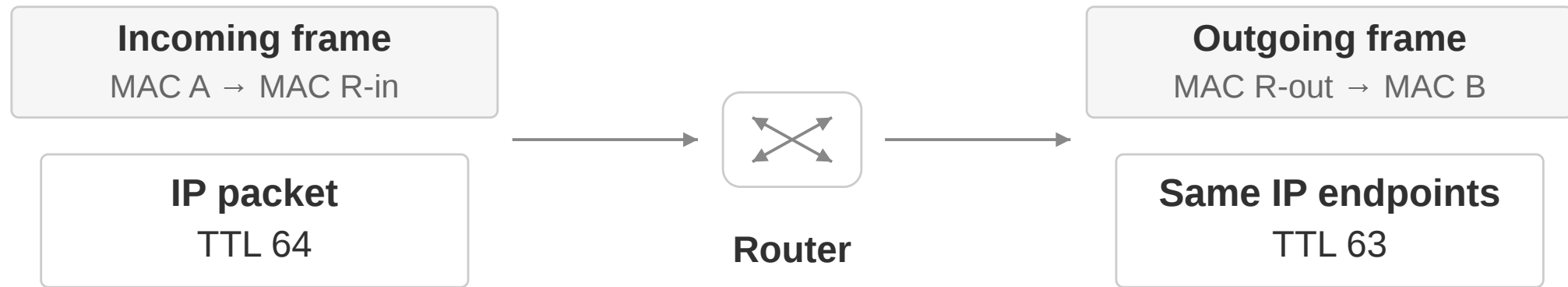
# Remote IP, local MAC



Each router builds a new link-layer frame.

IP identifies the end-to-end destination. MAC addresses serve the current link.

# Routers repeat the process hop by hop



Rebuild the frame. Keep the IP destination. TTL values are illustrative.

Each hop performs a new route lookup and builds a new frame.  
TTL / Hop Limit is reduced so packets cannot loop forever.

# Look inside the provider.

Keep the Lagoon probe open. In another lab-directory terminal:

```
make enter NODE=operations01.bob1.reefnet.test
```

Inside the operator workstation:

```
ssh edge01.bob1.reefnet.test
```

Use router FQDNs. The lab SSH configuration is already present.  
Credentials: `admin / NokiaSr11!`. Leave the router with `quit`.

# Where do routes come from?

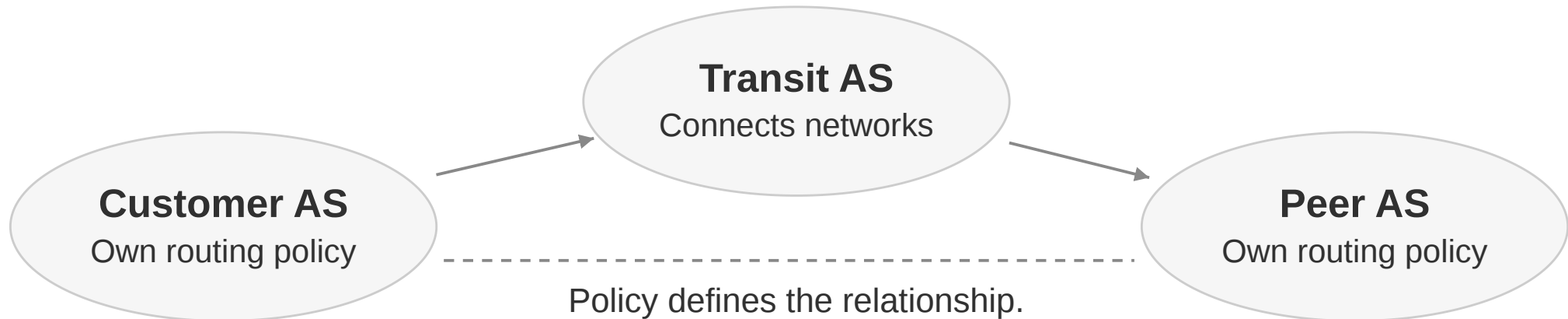
| Route source | Origin                 |
|--------------|------------------------|
| Connected    | Addressed interface    |
| Static       | Operator configuration |
| Dynamic      | Routing protocol       |

On ReefNet Edge 01:

```
show network-instance default route-table ipv4-unicast
```

Next: how IGP and BGP supply dynamic reachability.

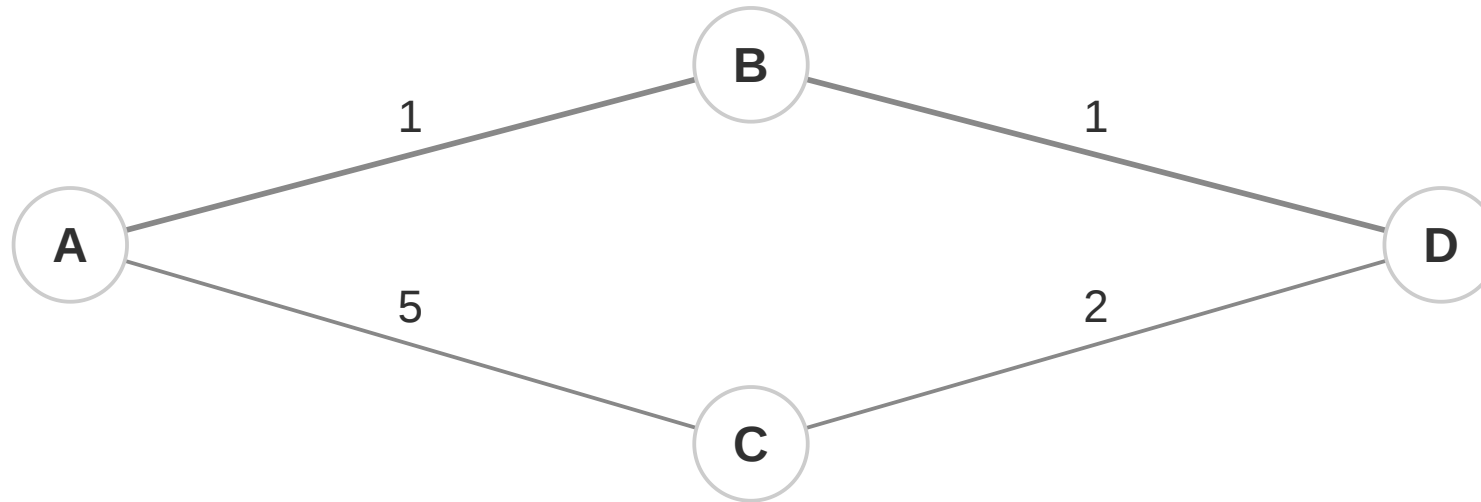
# An AS is one routing domain



An AS is an administrative boundary, not one router.

**AS: autonomous system**, under one routing administration.  
Inside: internal reachability. Between ASes: prefixes and policy.

# Inside an AS: IGPs



Example link costs: the shortest internal path costs 2.

OSPF and IS-IS compute internal paths from link state.

In ReefNet: `show network-instance default protocols ospf neighbor`



# BGP exchanges reachability.

BGP advertises IP prefixes together with path attributes.

Our lab has four autonomous systems.

On `edge01.bob1.reefnet.test`:

```
show network-instance default protocols bgp neighbor  
show network-instance default protocols bgp routes ipv4
```

A router can receive several routes to the same prefix.

BGP selects according to local policy and route attributes, not an IGP-style shortest path.

# BGP is policy first.

For comparable routes to the same prefix:

1. apply local policy
2. prefer higher local preference
3. compare AS\_PATH and further tie-breakers

Inspect the live prefix on ReefNet Edge 01:

```
show network-instance default protocols bgp routes ipv4 prefix 198.51.100.0/24
```

A longer path can be selected on purpose.

# IGP and BGP

|            | OSPF / IS-IS          | BGP                     |
|------------|-----------------------|-------------------------|
| Carries    | Link-state topology   | Prefixes and attributes |
| Chooses by | Internal metric       | Policy and attributes   |
| Provides   | Internal reachability | Route distribution      |

**eBGP** connects ASes. **iBGP** distributes routes inside an AS.  
BGP next hops still need an internal path.

# Transit and peering

|              | Transit                | Peering                |
|--------------|------------------------|------------------------|
| Reachability | Wider Internet         | Peer and its customers |
| Payment      | Customer pays provider | Often settlement-free  |

An **IXP** provides shared infrastructure for peering.  
Business relationships shape route preference and export.

# Who assigns the prefixes we route?



Administrative delegation, not a forwarding path

Delegation assigns address space. Routing advertises reachability.  
The two are related, but they are not the same operation.

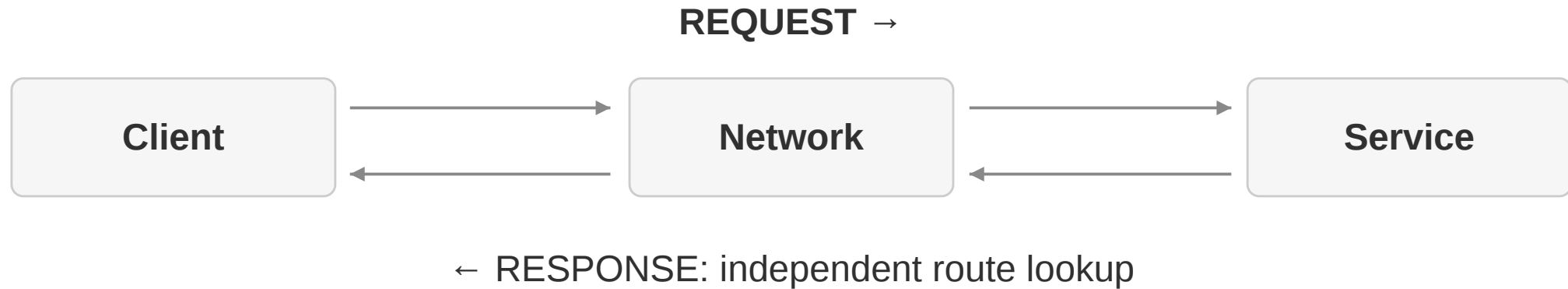
BOB1 uses documentation prefixes. Its service is not on the public Internet.

## The destination still has work to do.



The application must accept the request and return a response.

# The response needs its own route.



The destination now becomes the source.

Its routing decision is independent of the request path.

Reachability needs both directions. Paths may be asymmetric.

# ICMP: errors and diagnostics

| Message                 | Meaning                 |
|-------------------------|-------------------------|
| Echo Request / Reply    | Reachability test       |
| Destination Unreachable | Cannot deliver          |
| Time Exceeded           | TTL / Hop Limit expired |
| Packet Too Big (IPv6)   | Packet exceeds link MTU |

ICMP belongs to the Internet layer.



# Ping uses ICMP Echo

`ping` sends an ICMP Echo Request.

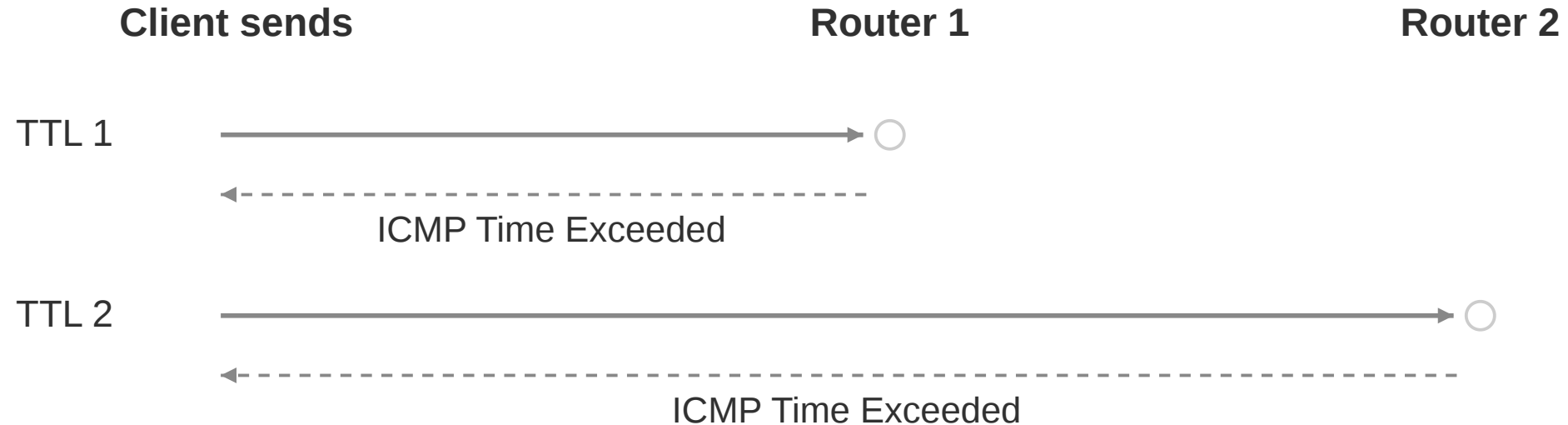
On `probe01.bob1.lagoontransit.test`:

```
ping -4 -c 3 198.51.100.10  
ping -6 -c 3 2001:db8:100::10
```

A reply shows that this ICMP exchange worked in both directions.

No reply does not identify where it failed.

# Traceroute expires the TTL.



Successively larger TTL values reveal responding hops.  
A missing reply does not identify the failed forwarding hop.

# Read the live host from the bottom up

Stay on `probe01.bob1.lagoontransit.test`.

```
ip -br link
ip -br addr
ip route
ip -6 route
ip route get 198.51.100.10
ip neigh
```

For each command, ask one question:

**What state did I just inspect?**

## Then test the path and protocol

Still on `probe01.bob1.lagoontransit.test`:

```
dig data.oceanresearch.test
ping -4 -c 2 198.51.100.10
traceroute -n -4 198.51.100.10
ss -lunpt
tcpdump -ni eth1
```

`ss` shows local sockets. Generate capture traffic from another terminal.  
Stop `tcpdump` with **Ctrl+C**.

# What does this mean in ReefNet?

Build one evidence chain for `data.oceanresearch.test`:

| Evidence               | Question answered         |
|------------------------|---------------------------|
| DNS response           | Which destination?        |
| Route + neighbor table | Which next hop and frame? |
| External HTTP request  | Does the service answer?  |

On the Lagoon probe, finish with HTTP over both families:

```
curl --fail --max-time 5 -4 http://data.oceanresearch.test/  
curl --fail --max-time 5 -6 http://data.oceanresearch.test/
```

# The service does not load.

The request crossed many boundaries.  
Any one of them can fail.

Now stop reciting the stack and start collecting evidence.

**A customer reports partial reachability.**

# 12 October 2026

## Network Operations

The service is broken.

What happened, and how do we restore it safely?

# One incident, from report to recovery.



Evidence before action. Recovery before closure.

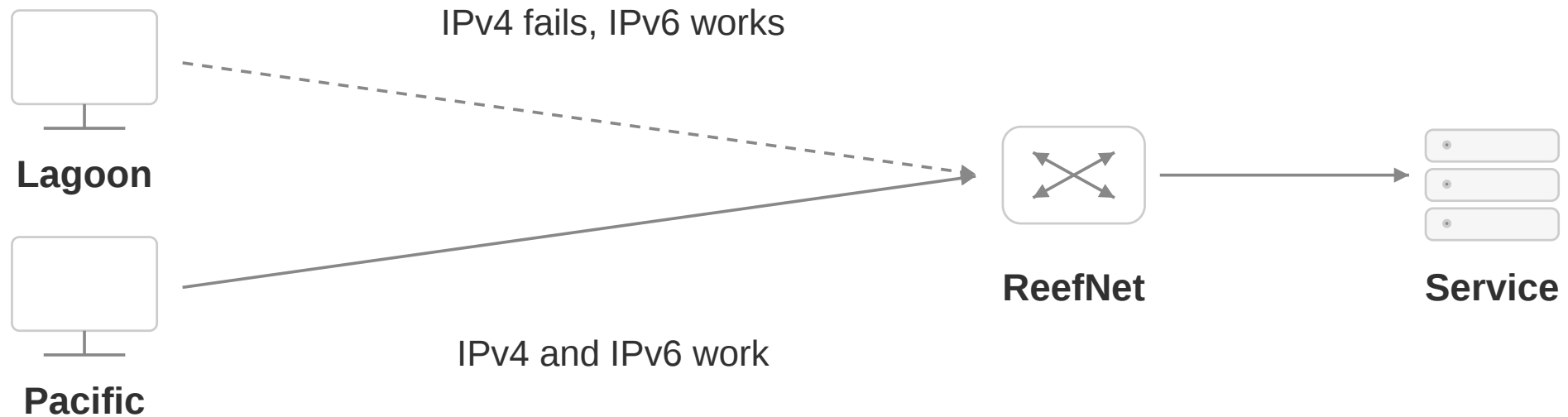
Establish known-good state before reproducing the failure.  
Close the incident only when service and baseline agree.

---

REPRODUCE → BOUND → OBSERVE → HYPOTHESIZE → REPAIR → VERIFY



# A customer reports partial reachability



Record the source, address family, protocol and time.

REPRODUCE → BOUND → OBSERVE → HYPOTHESIZE → REPAIR → VERIFY

# Establish the reference first.

Record a timestamped baseline. In the **lab-directory** terminal:

```
make networking  
make test
```

These are ICMP checks. Enter each external probe and also test HTTP:

```
curl --fail --max-time 5 -4 http://198.51.100.10/  
curl --fail --max-time 5 -g -6 'http://[2001:db8:100::10]/'
```

On ReefNet Edge 01, also save the healthy policy attachment:

```
info from running network-instance default protocols bgp group lagoon-v4
```

# Start the Operations scenario.

From the lab directory:

```
make operations  
make enter NODE=operations01.bob1.reefnet.test
```

| Evidence                        | Expected start state |
|---------------------------------|----------------------|
| BGP sessions                    | Established          |
| Lagoon IPv4                     | Broken               |
| Lagoon IPv6 + Pacific IPv4/IPv6 | Healthy              |

`make reset` rebuilds this incident state.

# Use the operator workstation.

From the operator workstation, connect to **one router at a time**:

```
ssh edge01.bob1.reefnet.test  
ssh edge02.bob1.reefnet.test  
ssh edge01.bob1.oceanresearch.test  
ssh edge01.bob1.lagoontransit.test  
ssh edge01.bob1.pacifictransit.test
```

All five routers use the same SR Linux CLI.

Lab login: `admin` / `NokiaSr11!`

Use `quit` to return to the calling terminal.

# Reproduce the complaint from two places

In two **lab-directory** terminals, enter the external probes. Terminal A:

```
make enter NODE=probe01.bob1.lagoontransit.test  
ping -4 -c 2 198.51.100.10  
ping -6 -c 2 2001:db8:100::10
```

Another terminal:

```
make enter NODE=probe01.bob1.pacifictransit.test  
ping -4 -c 2 198.51.100.10  
ping -6 -c 2 2001:db8:100::10
```

Which combination fails?

# Bound the incident.

|         | IPv4          | IPv6          |
|---------|---------------|---------------|
| Lagoon  | FAIL<br>----- | PASS<br>_____ |
| Pacific | PASS<br>_____ | PASS<br>_____ |

Keep the service and test method fixed. Vary the viewpoint.

# Separate state from service.

|                         |                              |
|-------------------------|------------------------------|
| <b>Control plane</b>    | Routes learned and selected  |
| <b>Forwarding plane</b> | Entries used to move packets |
| <b>Service behavior</b> | The customer gets a response |

A healthy control-plane session does not prove that the service works.

# Is BGP actually established?

On `edge01.bob1.reefnet.test`:

```
show network-instance default protocols bgp neighbor
```

For IPv4, focus on these peers:

| Neighbor                | Role                |
|-------------------------|---------------------|
| <code>192.0.2.0</code>  | Ocean Research      |
| <code>192.0.2.2</code>  | Lagoon Transit      |
| <code>10.255.0.2</code> | ReefNet edge02 iBGP |

What does **established** prove?

---

REPRODUCE → BOUND → OBSERVE → HYPOTHESIZE → REPAIR → VERIFY



# Established is not reachable.

The session is up. The customer route may still be missing.

# Did the customer route arrive?

On `edge01.bob1.reefnet.test`:

```
show network-instance default protocols bgp routes ipv4 prefix 198.51.100.0/24
```

Then inspect IPv6:

```
show network-instance default protocols bgp routes ipv6 prefix 2001:db8:100::/48
```

A route in BGP is not yet proof of forwarding.

## Is IPv4 installed for forwarding?

```
show network-instance default route-table ipv4-unicast prefix 198.51.100.0/24
```

Look for:

**active route → resolved next hop → outgoing interface**

The forwarding entry toward the customer is present.

## ReefNet also needs internal reachability

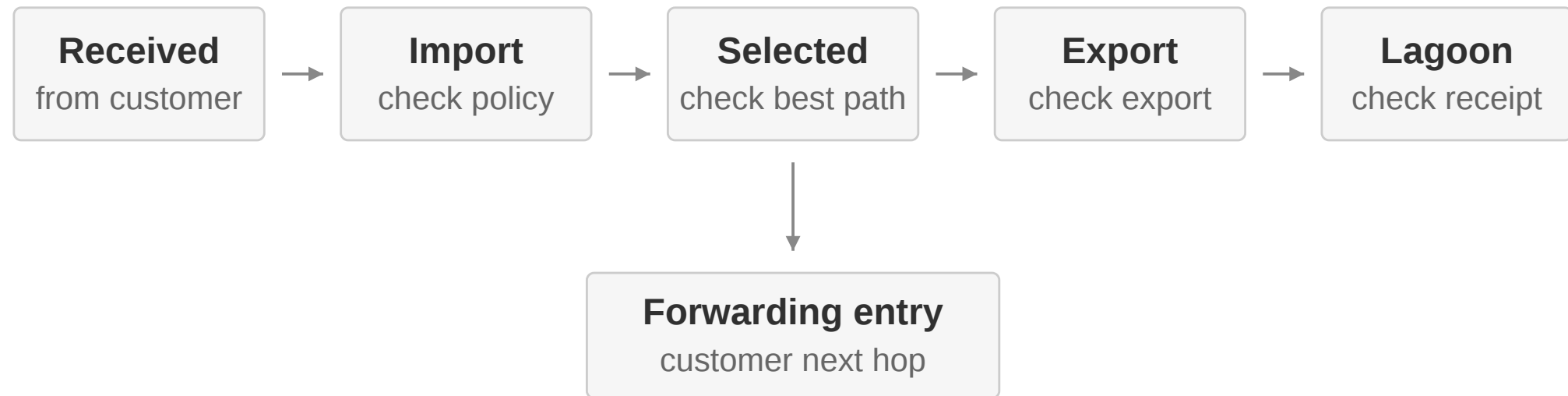
`edge01.bob1.reefnet.test` and `edge02.bob1.reefnet.test` use OSPF inside AS65000.

```
show network-instance default protocols ospf neighbor
```

The IGP gets us through **our** network.

BGP decides which external reachability we advertise and prefer.

# The evidence now points outward



The route arrived and is installed. Next, compare the two upstream exports.

# Compare the advertisements

On `edge01.bob1.reefnet.test`, toward `edge01.bob1.lagoontransit.test`:

```
show network-instance default protocols bgp neighbor 192.0.2.2 advertised-routes ipv4
```

On `edge02.bob1.reefnet.test`, toward `edge01.bob1.pacifictransit.test`:

```
show network-instance default protocols bgp neighbor 192.0.2.5 advertised-routes ipv4
```

Find `198.51.100.0/24`.

## A testable hypothesis.



Predict the missing evidence before changing the router.

What evidence would disprove this hypothesis?

# Inspect the policy attachment

On `edge01.bob1.reefnet.test`:

```
info from running network-instance default protocols bgp group lagoon-v4
```

Compare it with:

```
info from running network-instance default protocols bgp group lagoon-v6
```

What is attached only to IPv4?



# Make the smallest justified change

On `edge01.bob1.reefnet.test`:

```
enter candidate

delete / network-instance default protocols bgp group lagoon-v4 export-policy

diff
commit now
```

The group now inherits the healthy instance export policy.

Do not restart BGP. Do not touch `edge01.bob1.pacifictransit.test`.

# The change is not the recovery proof.

Recheck the advertisement. From each external probe, test HTTP:

```
curl --fail --max-time 5 -4 http://198.51.100.10/  
curl --fail --max-time 5 -g -6 'http://[2001:db8:100::10]/'
```

In the **lab-directory** terminal:

```
make test
```

Fail on HTTP errors. Allow at most **5 seconds** per request.  
Keep results from both upstreams and address families.

## Prove success and containment.

**Positive:** the formerly failing Lagoon IPv4 service works.

**Invariant:** Pacific and IPv6 still work.

**Negative:** override absent. Unrelated configuration unchanged.

This repair tests policy attachment, not a new prefix allowlist.

## Confirm the known-good state.

After the service checks pass, compare configuration with the healthy baseline.

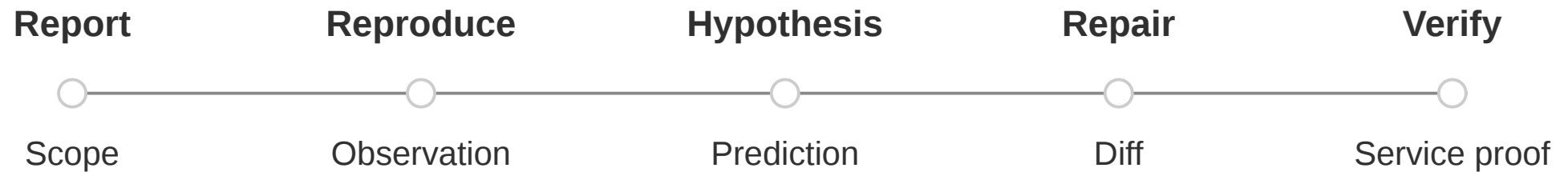
Explain any remaining difference. Keep the repair diff and service results.

A reachable service can still hide unintended changes.

Mark the state **known good** only when configuration and service agree.

Keep this evidence before resetting the lab.

# Keep an incident timeline.



Record timestamps, actions and evidence at each boundary.

Separate observed facts, interpretations and actions.

# Write the update from the evidence

A useful update states:

**impact** → **evidence** → **action** → **verification**

For example:

IPv4 through Lagoon Transit was missing because the customer route was filtered on export. The policy attachment was removed and reachability now succeeds from both external probes. IPv6 was unaffected.

# Write the runbook

1. Reproduce and bound the symptom.
2. Compare intended and observed state.
3. Test a hypothesis before changing anything.
4. Repair, repeat the failing test, record the result.

Unexpected evidence means **revise the hypothesis**.

# Write an RFO after the incident

**RFO: Reason for Outage.** Record impact, cause and recovery.

- **Impact:** IPv4 unreachable through Lagoon Transit
- **Cause:** wrong export policy attached to `lagoon-v4`
- **Recovery:** remove the attachment. Verify HTTP through both upstreams and families
- **Follow-up:** test IPv4 and IPv6 exports before deployment

Unknown time? Write **unknown**, not a guess.



# RCA goes beyond the bad line

**RCA: Root Cause Analysis.** Look beyond the wrong attachment:

- Why did validation accept it?
- Why did the postcheck miss one address family?
- What test prevents the same failure next time?

Carry these prevention questions into the change contract.

# That was network management.

Place this repair in the wider management picture:

- **Fault:** isolate lost reachability
- **Configuration:** repair export policy
- **Performance:** verify service behavior
- **Security:** control who may change the router
- **Accounting:** attribute resource use when needed

Configuration is only one part of operating a network.

## Four views, one recovery claim.

| View            | Recovery evidence                        |
|-----------------|------------------------------------------|
| NetBox          | Intended service and circuit still match |
| CLI             | Correct policy and route advertisements  |
| Grafana         | Fault and recovery history, if collected |
| External probes | Service works through both upstreams     |

No single view replaces the others.

# Operations: before you leave

Save your timeline, repair diff and recovery proof.

```
make healthy  
make test  
make down
```

Tomorrow: make **this class of repair** safe to repeat.

# 13 October 2026

## Automation + Incidents

Yesterday we repaired the network manually.

Today we need to make the same class of change **safely and repeatedly.**

At the end of today: **Who Broke the Internet?**

## Same network, new job

From the lab directory:

```
make automation  
make enter NODE=operations01.bob1.reefnet.test
```

The same IPv4 export incident is back.

This time the repair needs a reviewed input and repeatable checks.

# The safe change loop



Failed verification: record actual state, back out, verify again.

**The workflow is the automation story.**

YANG, NETCONF and gNMI are mechanisms within it.

---

INTENT → OBSERVE → PLAN → CHANGE → VERIFY → RECORD

## Agree on a change contract.

| Boundary | Agree before execution                  |
|----------|-----------------------------------------|
| Scope    | Target, permitted diff, preserved state |
| Entry    | Preconditions and reviewed input        |
| Exit     | Service checks and deadline             |
| Recovery | Backout, owner and access path          |

If the contract is vague, the script automates the ambiguity.



# The change contract

Target: **ReefNet Edge 01**, BGP group `lagoon-v4`.

|          | Required state                                           |
|----------|----------------------------------------------------------|
| Before   | Override: <code>BLOCK-CUSTOMER-V4</code>                 |
| After    | Inherit <code>EXPORT-BGP</code> without a group override |
| Preserve | Lagoon IPv6 and Pacific Transit                          |
| Prove    | Customer IPv4 export and external reachability           |

Which facts need a fresh read from the router?

---

INTENT → OBSERVE → PLAN → CHANGE → VERIFY → RECORD

# Intent, policy, configuration

| Level         | ReefNet example                           |
|---------------|-------------------------------------------|
| Intent        | Customer reachable through both upstreams |
| Policy        | Lagoon IPv4 inherits EXPORT-BGP           |
| Configuration | lagoon-v4 currently has a group override  |

A correct policy on the wrong group still fails the intent.

# Inventory gives the target context.

An inventory is a graph of relationships.



Join the intended circuit to the actual change target.

Use inventory to identify the target. Read its live policy before changing it.

# Read the device inventory.

Inside the operator workstation:

```
TOKEN=$(cat /state/netbox-token)
NETBOX_API=http://netbox.bob1.reefnet.test:8080/api
curl -sG -H "Authorization: Bearer $TOKEN" \
  "$NETBOX_API/dcim/devices/" \
  --data-urlencode name=edge01.bob1.reefnet.test | jq .
```

The lab provisions the token. Keep it out of reports and screenshots.  
Check that exactly the intended device is selected.

## Read the circuit intent too

The same API exposes the Lagoon Transit circuit:

```
curl -sG -H "Authorization: Bearer $TOKEN" \
  "$NETBOX_API/circuits/circuits/" \
  --data-urlencode cid=BOB1-LAGOON-001 | jq .
```

Compare inventory intent with live state before changing the router.

**NetBox describes intent. The router shows live state.**

# Join inventory into one change input

The API gives us separate objects. We need the relationship.

```
GET /api/dcim/devices/?name=edge01.bob1.reefnet.test  
GET /api/dcim/interfaces/?device=edge01.bob1.reefnet.test  
GET /api/ipam/prefixes/?prefix=198.51.100.0%2F24  
GET /api/ipam/asns/?asn=65100
```

The useful part is not the endpoint itself.

It is joining the right objects into one approved change input.

# Intent and observation can drift.

Compare **meaning**, not raw text.

Before calling it drift, check:

- same scope?
- same point in time?
- fresh observation?
- equivalent representation normalized?

Then decide whether to reconcile or investigate.

## Unavailable input is not an empty list.

| Input              | State         |
|--------------------|---------------|
| Inventory snapshot | Current       |
| Monitoring API     | Unavailable   |
| Previous output    | Known working |

Stop publication or use an approved, age-limited fallback.

**Unknown targets must not silently disappear.**



# Freeze the input you reviewed

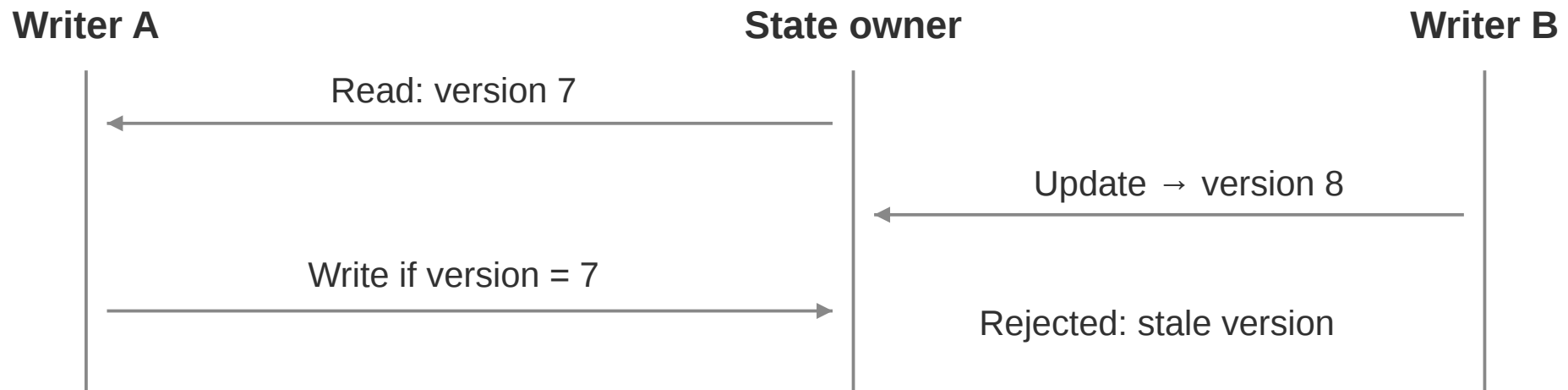
```
{  
  "change_id": "CHG-017",  
  "device": "edge01.bob1.reefnet.test",  
  "bgp_group": "lagoon-v4",  
  "expected_override": "BLOCK-CUSTOMER-V4",  
  "action": "delete group export-policy",  
  "expected_inherited_policy": "EXPORT-BGP"  
}
```

Reviewer and executor need the **same approved input**.

Also keep the current state needed for backout.

# Prevent a lost update

**Optimistic concurrency:** write only if the version you reviewed is still current.



Check and write must be atomic at the state owner.

A CLI precheck is useful, but it cannot make the later write atomic.

# Validate before rendering

The request is small. The failure modes are not.

| Check                     | Catches                                               |
|---------------------------|-------------------------------------------------------|
| Device + BGP group        | right action, wrong target                            |
| Current override          | stale input or already repaired state                 |
| Direction                 | import edited instead of export                       |
| Inherited instance policy | deleting the override would expose the wrong fallback |

Validate the request **and** the state it depends on.

## Generate and inspect the candidate.

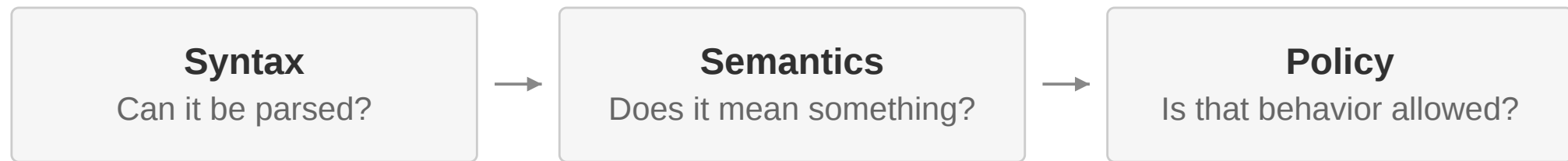
Generate the candidate from the frozen **CHG-017** input. Expected diff:

```
- export-policy [ BLOCK-CUSTOMER-V4 ]
```

Retain instance policy **EXPORT-BGP**. Reject unrelated changes.

Review now. Execute the lab repair later with gNMI.

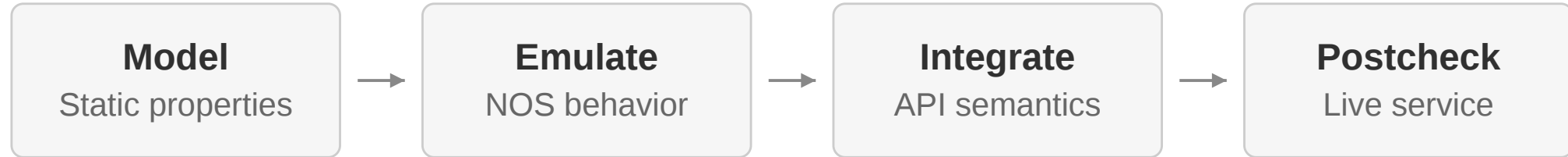
# Validation has three layers.



A valid configuration can still violate the change contract.

Passing one gate does not prove the next.

# Model it, emulate it, then check the service



Different tests expose different failure classes.

Passing a static model check cannot prove target API behavior.

# Plan the maintenance window



Stop time + recovery owner apply at every gate.

**Method of Procedure (MOP):** steps, checks and recovery ownership

Agree on acceptable impact before the window opens.

# Define success before the change

| Check         | This repair                             |
|---------------|-----------------------------------------|
| Precondition  | lagoon-v4 still uses BLOCK-CUSTOMER-V4  |
| Postcondition | Customer IPv4 export and probes recover |
| Invariant     | Lagoon IPv6 and Pacific remain healthy  |

Read routes locally. Test the service from outside.



## Bound the rollout.

Choose a representative canary before increasing the target set.

1. Read live preconditions again.
2. Change one failure domain. Wait for service evidence.
3. Continue only while invariants hold.

Define **execution order**, **blast radius** and **stop time**.

Make the backout path reachable before the first write.

## Give automation a bounded identity.

| Identity            | Read                         | Write          |
|---------------------|------------------------------|----------------|
| Telemetry collector | Required state               | No             |
| Change service      | Preconditions + verification | Approved scope |
| Reviewer            | Evidence                     | No             |

Load credentials at runtime. Record actor and change ID, never secrets.  
Test emergency access and credential rotation before the window.

# Execute and recover

Now assume the candidate is approved.

The next question is different:

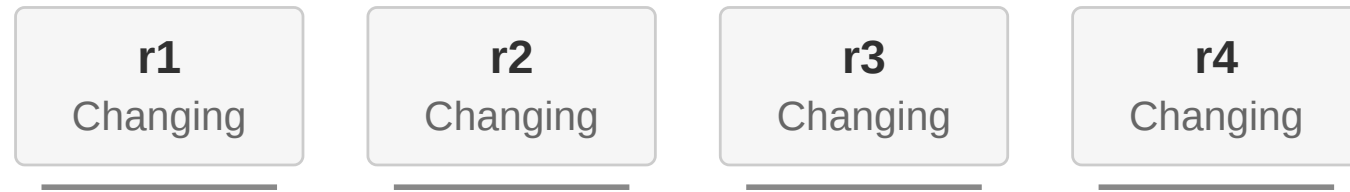
**What happens when the write only partly succeeds, or we cannot tell?**

# A fleet changes the failure model.

**One at a time**



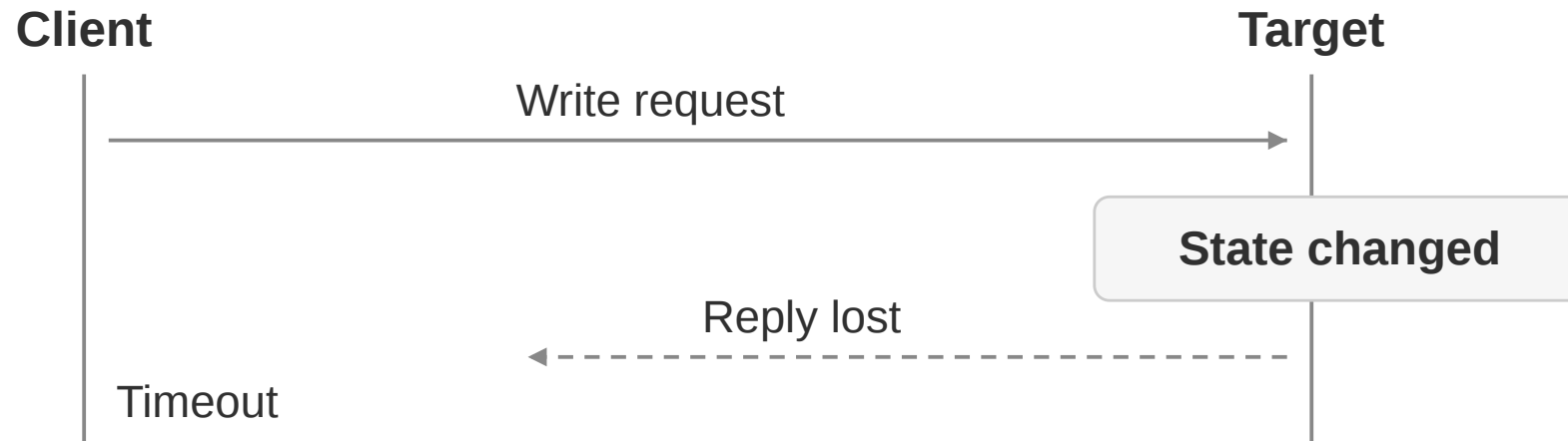
**All at once**



How many devices can this change affect before the next check?

More concurrency increases throughput and blast radius.

# The write may already have happened



Unknown outcome  $\neq$  no change. Read before retrying.

Record an unknown outcome explicitly in the audit trail.

# Run it twice. What changes?

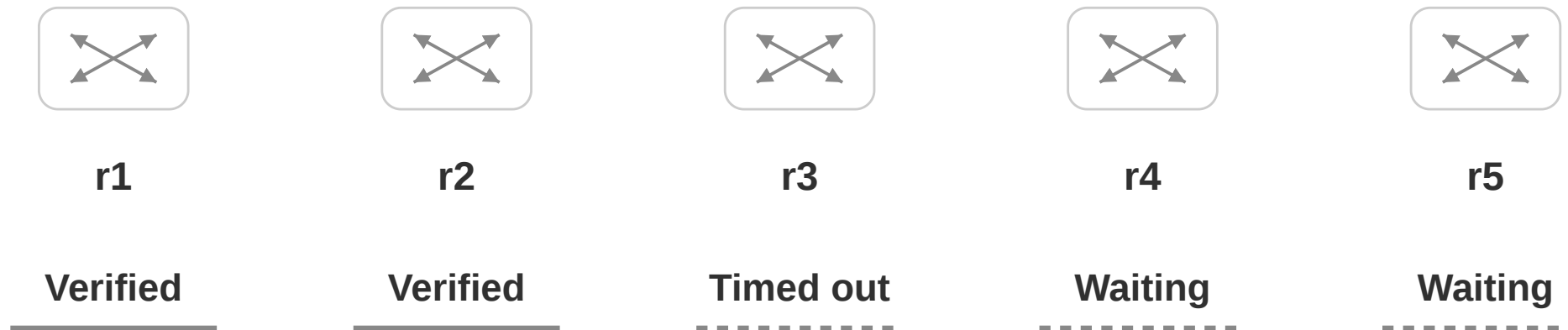
**Idempotence**, for a fixed operation  $F$  and state  $s$ :

$$F(F(s)) = F(s)$$

Useful after an uncertain outcome:

- repeated execution does not keep changing state
- retry is still not automatic
- service verification is still separate

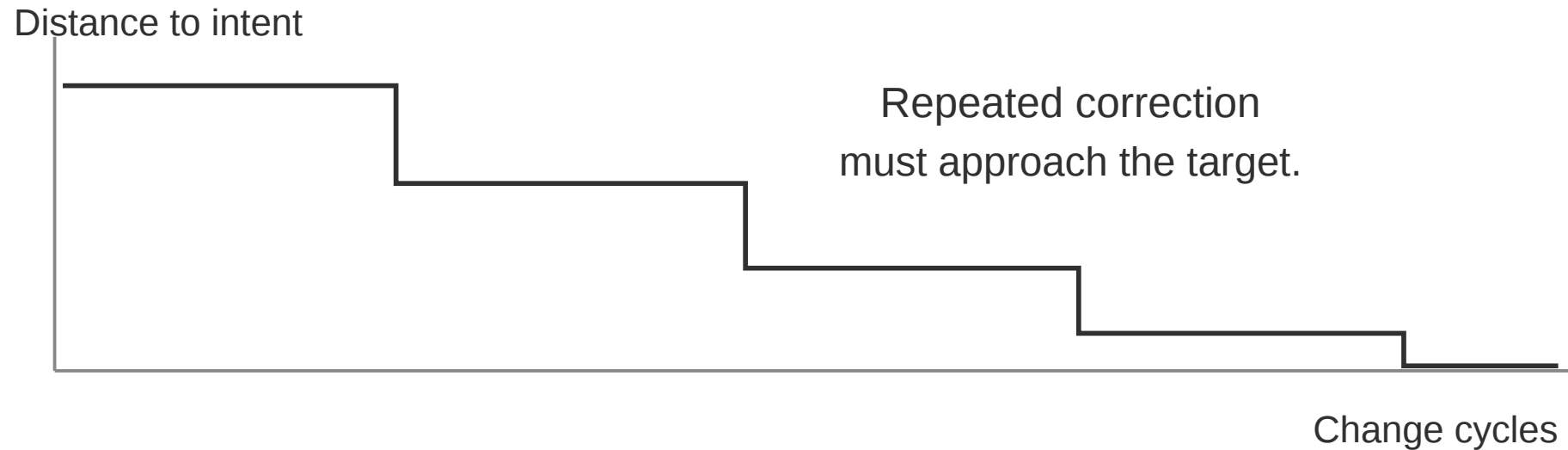
# One router timed out after the write



Progress is per device. A timeout is not a rollback.

Which device needs a fresh read before any retry?

# Does the loop converge?



Idempotence alone does not prove convergence.



# Job success is not service success.

**Exit code:** 0

**Probe:** timeout

The script has successfully finished being wrong.

# Test the intended behavior

The device accepted the change.

That only answers:

"Was the write accepted?"

Now test what should pass, what should fail, and what must remain unchanged.

## Plan the repair verification

| Check                      | Expected         |
|----------------------------|------------------|
| Lagoon: 198.51.100.0/24    | Export restored  |
| Lagoon: 2001:db8:100::/48  | Still exported   |
| Pacific IPv4 path          | Still healthy    |
| Both probes, both families | Service responds |

Use this matrix after the gNMI repair. The lab is still in the fault state.

## Test rejected routes too.

For a different change, test route admission.

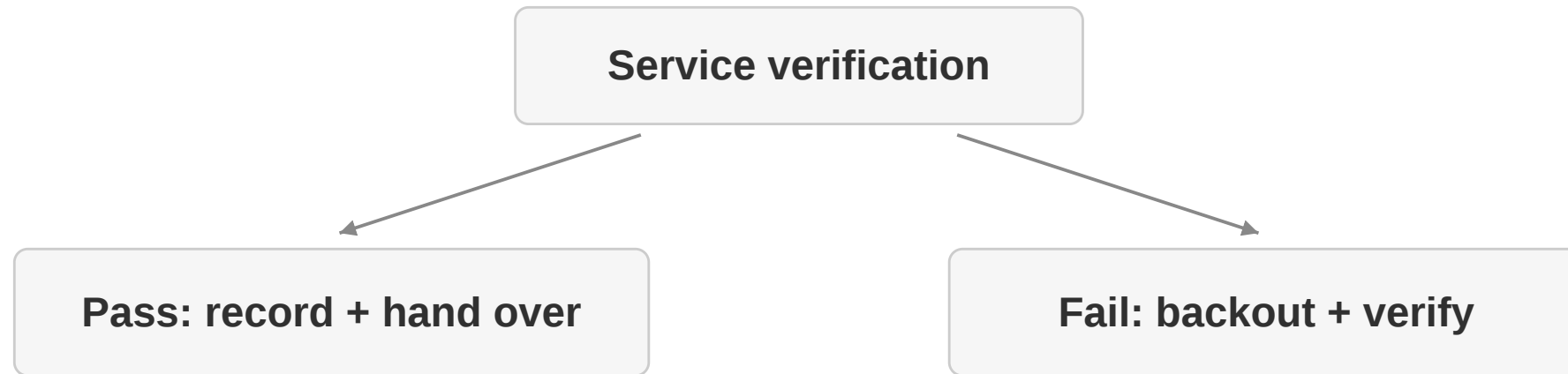
**Route Origin Validation (ROV)** checks RPKI origin authorizations.

| Result   | Meaning                                          |
|----------|--------------------------------------------------|
| Valid    | Origin and prefix length permitted               |
| Invalid  | Covered, but no authorization permits this route |
| NotFound | No covering authorization                        |

Policy decides the action. Test accepted and rejected routes.

This example is outside the BOB1 lab.

# Rollback and backout



Rollback: configuration. Backout: recovery procedure.

Record the backup's identity, time and version.  
Assign an owner to the full recovery procedure.

# Which state are we restoring?

03:00. Someone says: "restore last Tuesday."

That is not precise enough.

| Golden configuration | Backup                |
|----------------------|-----------------------|
| approved baseline    | captured device state |

Record identity, time, version and checksum.

Keep history outside the device failure domain.

# Keep an audit trail.

```
{  
  "actor": "course-automation",  
  "target": "edge01.bob1.reefnet.test",  
  "change_id": "CHG-017",  
  "deleted_override": "BLOCK-CUSTOMER-V4",  
  "inherited_policy": "EXPORT-BGP",  
  "lagoon_ipv4_probe_passed": true  
}
```

Keep inputs, timestamps and before/after evidence.

Never keep credentials in the audit record.

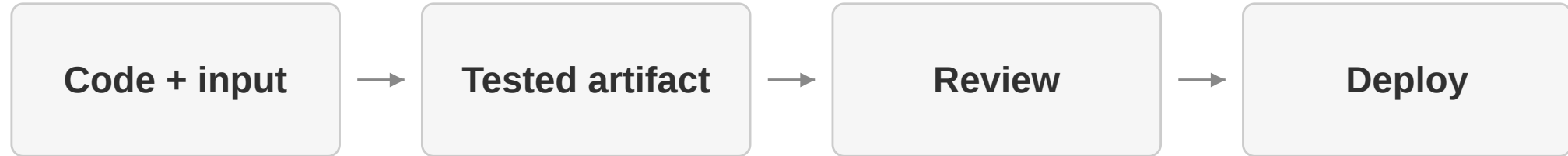
# Who operates the automation?

- A named owner and bounded runtime
- One writer per target with explicit retry rules
- Fresh inputs and verified outcomes
- Tested recovery and a retained audit trail

The next shift must be able to run it.



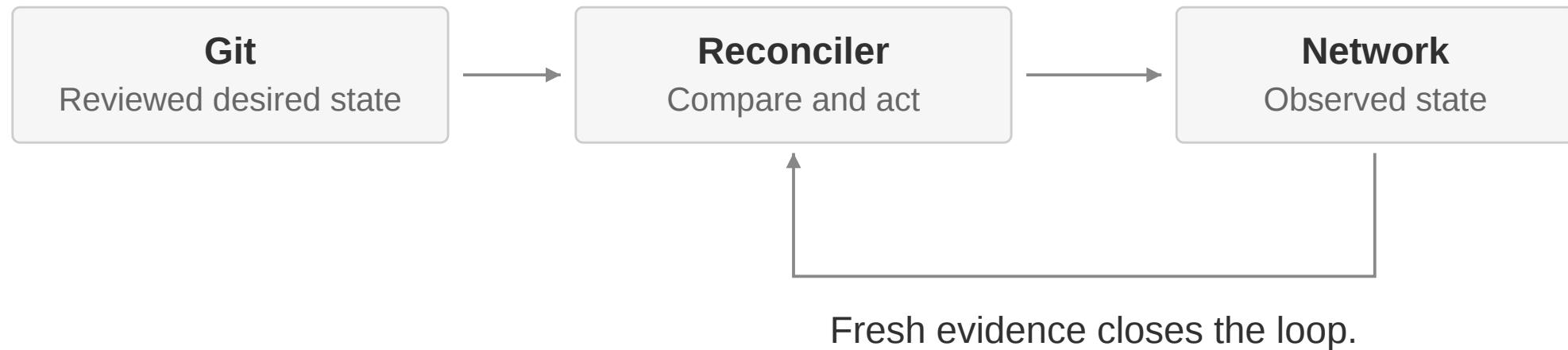
## Promote the tested artifact



Deploy the reviewed artifact without regenerating it.

Approval applies to a specific code revision and input snapshot.

# GitOps gives us a reviewed desired state



Reconcile only from fresh evidence and reviewed intent.

## Now choose the mechanism.

The safe workflow is already defined.

**CLI → structured models → YANG / OpenConfig → NETCONF / gNMI**

Each mechanism must preserve the same preconditions,  
service checks and recovery contract.

# Netmiko: automate the CLI you already know

Netmiko manages a CLI session over SSH. An illustrative SR Linux read:

```
from netmiko import ConnectHandler

with ConnectHandler(**device) as conn:
    output = conn.send_command("show interface")
```

It handles prompts and modes.

**You still have to interpret the result.**

# A prompt is not a postcheck.

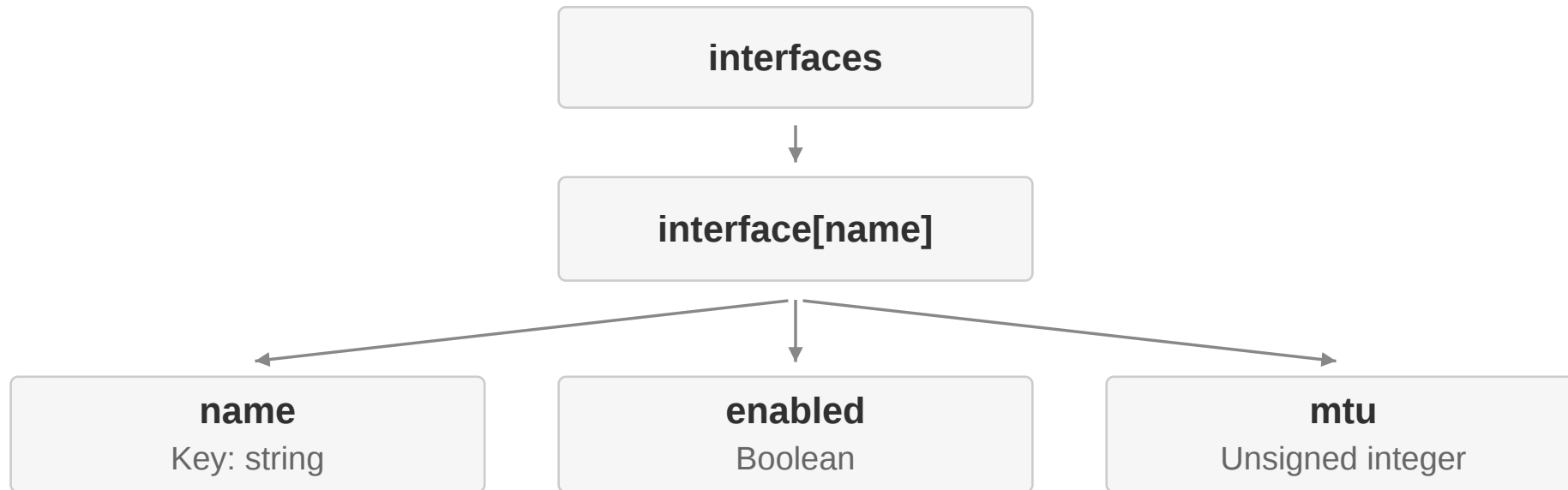
Illustrative CLI error, not SR Linux syntax:

```
router(config)# interface does-not-exist
% Invalid interface
router#
```

The session survived. The command did not.

Keep raw output. Distinguish an empty result from a parser failure.  
Read state back and run the service check after a write.

# YANG describes management data.



The model constrains data. The change contract constrains behavior.

# YANG keys and constraints

```
list interface {  
    key "name";  
    leaf name { type string; }  
    leaf enabled { type boolean; default true; }  
    leaf mtu { type uint16 { range "1280..9216"; } }  
}
```

A module contains this fragment. The MTU range is illustrative.

A list key must name an existing child leaf.

## Enabled, but no carrier

| View           | Interface fact       |
|----------------|----------------------|
| Configuration  | Interface is enabled |
| Observed state | Carrier is down      |

Configuration and observed state answer different questions.

Check the physical link and the peer before changing the configuration.



# OpenConfig

Vendor-neutral YANG models for interfaces, routing and telemetry.

A common model helps tools address the same concepts.  
It does not guarantee identical device support.

**Check the supported paths and operations.**

# NETCONF edits a datastore.

With a supported candidate datastore:

1. Lock, edit and validate.
2. Commit.
3. Verify the service independently.
4. Unlock and record.

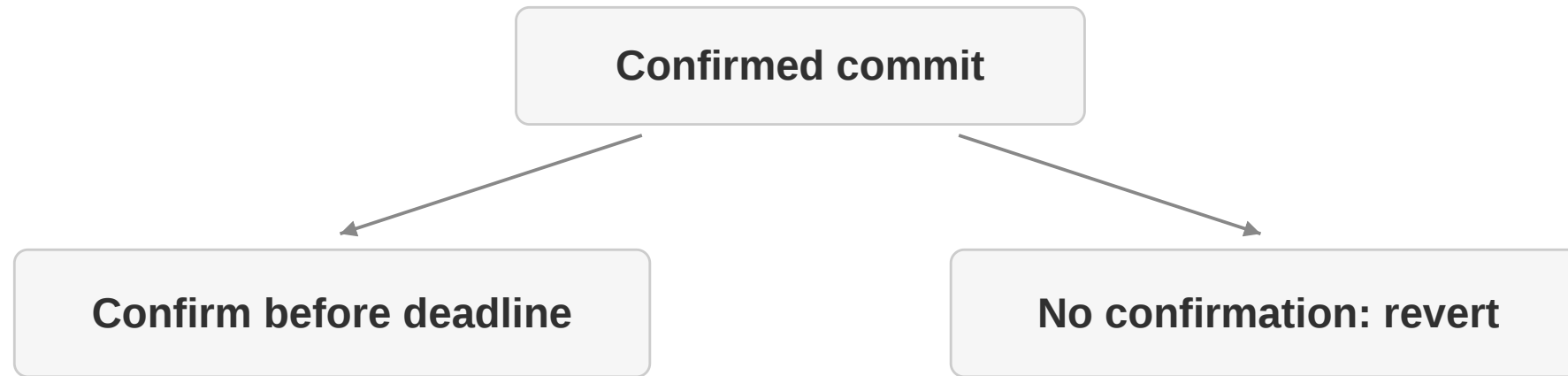
The session advertises which capabilities are available.

## Read the NETCONF error.

```
<rpc-error>  
  <error-type>application</error-type>  
  <error-tag>invalid-value</error-tag>  
  <error-severity>error</error-severity>  
</rpc-error>
```

Inspect the tag, path, message and datastore state after failure.  
Then decide whether a retry is safe.

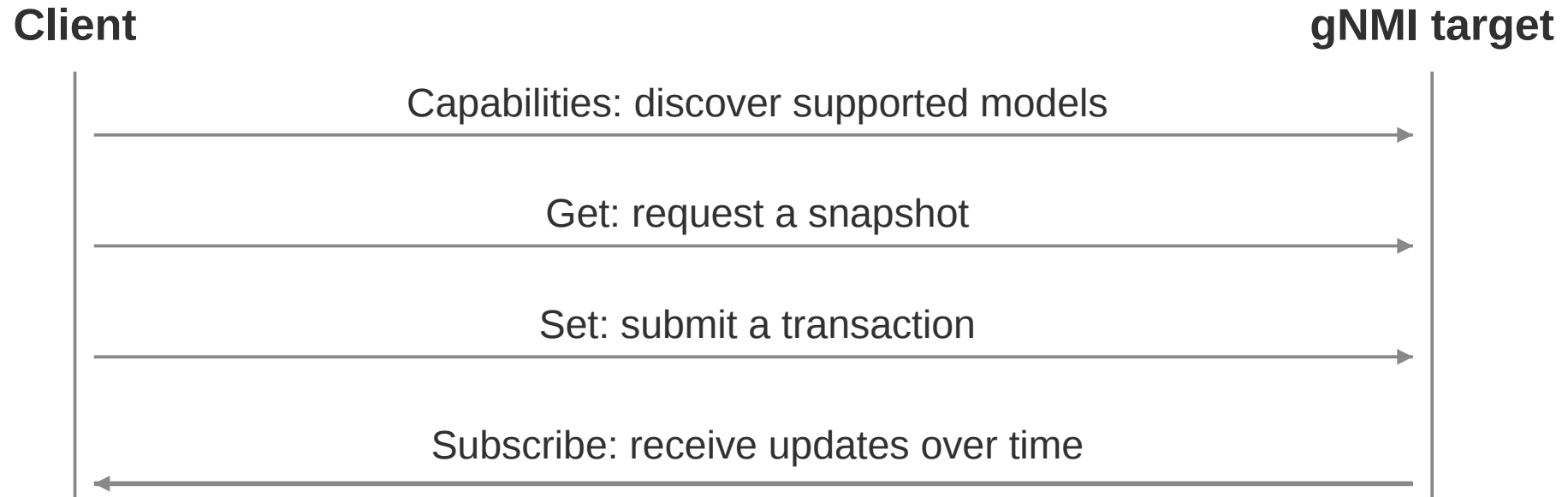
# Confirmed commit



Verify service before confirming. Requires target support.

Who confirms if automation loses contact? Test the revert path.

# gNMI exposes four operations.



Check which models and operations the target supports.

## Read a gNMI path

```
/interface[name=ethernet-1/1]/statistics
```

- `name=ethernet-1/1` selects one interface.
- `statistics` selects its subtree.
- The **target** identifies the router.

Next: read the BOB1 policy attachment.

# Read the live policy over gNMI.

From the operator workstation, read the group and instance policies:

```
BGP_PATH='/network-instance[name=default]/protocols/bgp'
POLICY_PATH="$BGP_PATH/group[group-name=lagoon-v4]/export-policy"
gnmic -a edge01.bob1.reefnet.test:57400 \
  -u admin -p 'NokiaSrl1!' --skip-verify \
  --encoding json_ietf get --path "$POLICY_PATH" \
  --path "$BGP_PATH/export-policy"
```

Confirm the override and inherited instance policy before writing.  
Keep this shell open for the Set exercise.

# gNMI Set

One request, one target, one transaction.

| Result                 | Required behavior                               |
|------------------------|-------------------------------------------------|
| Target reports success | Apply all requested changes                     |
| Target reports failure | Restore affected state to its pre-request state |

**A timeout leaves the outcome unknown.** Read back before retrying.  
Coordinate routers separately. Verify the service afterward.



## Repair the same incident with gNMI.

In the same operator shell, using the reviewed `POLICY_PATH`:

```
gnmic -a edge01.bob1.reefnet.test:57400 \  
-u admin -p 'NokiaSr11!' --skip-verify \  
--encoding json_ietf set --delete "$POLICY_PATH"
```

Re-read state. Repeat HTTP from both external probes over both families.  
Run `make test` in the **lab-directory terminal** for ICMP checks.

**The API response is not the service postcheck.**

# Review a change plan

Eight minutes in groups of 3–4.

| Proposed plan |                       |
|---------------|-----------------------|
| Input         | Yesterday's inventory |
| Validation    | Syntax only           |
| Rollout       | All routers at once   |
| Success       | Exit code 0           |
| Recovery      | Rerun the change      |

Agree on **two improvements** and their acceptance checks.

# Automation: close the loop.

Two minutes. Individual notes.

A write times out after the request was sent.

- What evidence establishes the current state?
- Which service check proves the repair worked?
- What must the next operator find in the change record?

In the lab terminal, stop the runtime with `make down`.

Next: test these safeguards against real production incidents.

# Who Broke the Internet?

13 October 2026. **Day 2, closing block.**

Now test the change workflow against real outages.

**Everything is redundant until the shared dependency breaks.**

Which safeguard would have changed the outcome?

# The rules of the show

For each case:

1. who lost what, and for how long
2. the failure boundary
3. evidence that distinguishes the cause
4. the recovery dependency
5. one safeguard worth testing

We are studying mechanisms, not awarding blame.

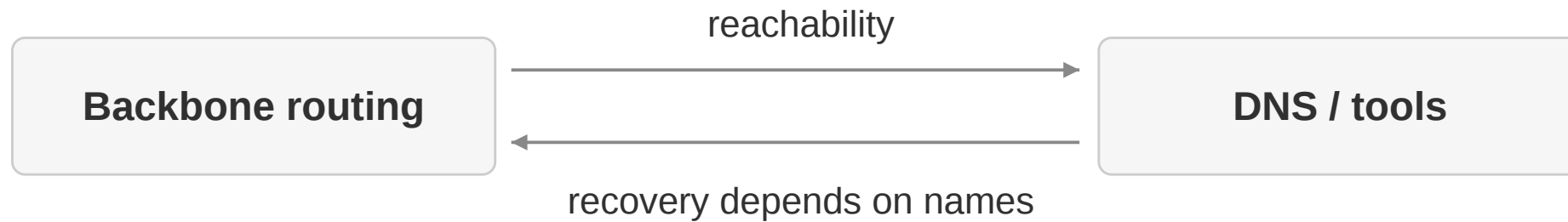
# Facebook: the backbone took DNS with it

4 October 2021.

- A maintenance command was meant to assess backbone capacity.
- An audit-tool bug let it disconnect the backbone instead.
- DNS sites lost data-center reachability and withdrew their BGP routes.
- Authoritative DNS kept running, but clients could no longer reach it.

**Impact:** Facebook, Instagram and WhatsApp became unreachable.

# Facebook: recovery lost its own tools



**Break the loop before the outage.**

Normal and out-of-band access failed. Engineers had to go onsite.

They restored the backbone, then brought services back under controlled load.

# Can you still reach the router?

**In-band:** management traffic uses the production network.

**Out-of-band:** a separate path to the device or console.

A separate path can still share dependencies:

- power
- fiber route
- jump host
- DNS and authentication (AAA)



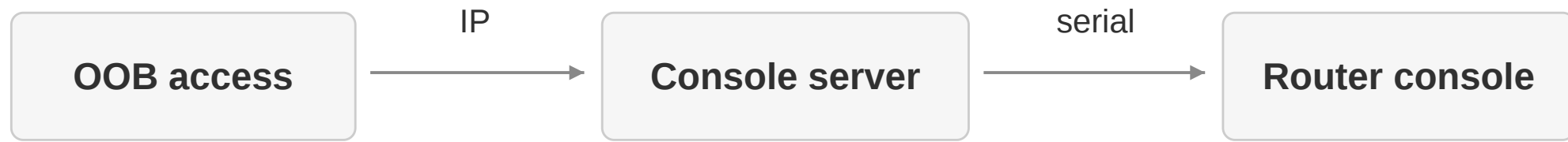
# A console port is not a management port

A **console server** gives remote access to attached serial ports.

| Connection on a production router | What it needs                                      |
|-----------------------------------|----------------------------------------------------|
| Ethernet management port          | IP connectivity and a working management service   |
| Serial console port               | correct cable, serial settings, responsive console |

Serial bypasses IP forwarding. It still needs power and a responsive console.

# Production recovery example: a console path



Power, emergency authentication and reachable credentials still matter.

This production recovery path is not emulated by BOB1.

Test login and repair while the production path is unavailable.

## Task: the console is reachable, login fails



Exercise: AAA uses the failed backbone. Which fallback must exist?

# Recovery only works if it was prepared

For the production recovery path, record each dependency.

| Need           | Prepared detail                      |
|----------------|--------------------------------------|
| Transit        | circuit and provider contact         |
| Console        | server, port, cable, serial settings |
| Authentication | tested emergency login               |
| DNS / secrets  | reachable resolver and credentials   |
| Power          | backup for every device in the path  |

After login, verify that the account can actually change configuration.

## Remote hands need an exact job

If remote recovery is blocked, onsite staff become your eyes and hands.

```
Location:   Frankfurt, rack R12
Equipment:  edge-a, port 2
Task:       verify label and photograph LEDs
Scope:      inspect only, no disconnect
Return:     timestamp, photos, label mismatch
```

For vendor escalation, add version, symptoms and logs.

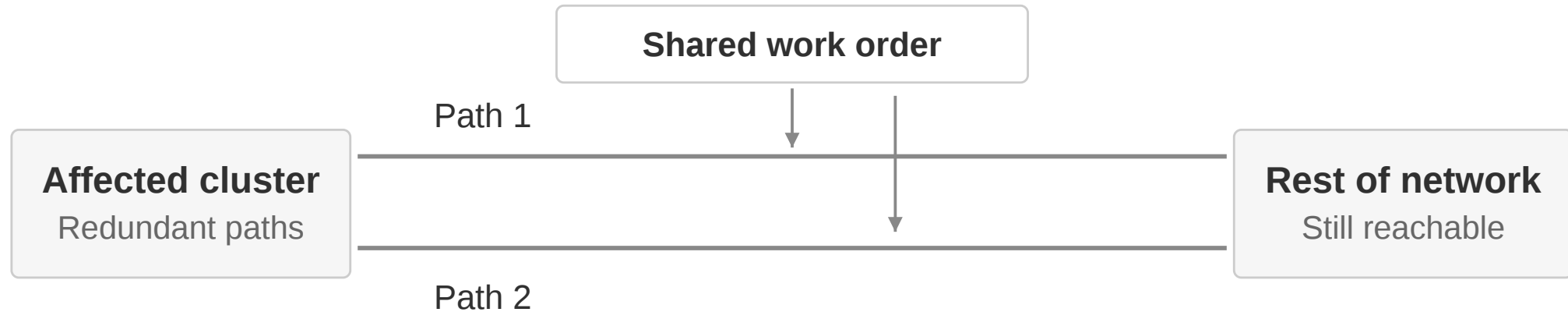
# Google Cloud: incompatible optics

1 September 2026. A router capacity upgrade in **us-central1**.

- Technicians replaced optical transceivers on redundant routers.
- The new optics were incompatible with the fabric-side optics.
- The work crossed redundant routers before connectivity was checked.

**Impact:** affected VMs lost incoming and outgoing connectivity.  
Only parts of **us-central1-b** and **us-central1-f** were isolated.

# Google Cloud: one worklist defeated redundancy



1 September 2026: all affected paths were unplugged within **13 minutes**.  
The worklist omitted router-by-router sequencing and verification.

Recovery: divert traffic, reinstall the original optics, verify links.

# OVHcloud: undamaged did not mean available

Strasbourg, 10 March 2021. Fire began in SBG2 at 00:47 CET.

- SBG2 was destroyed. SBG1 lost four of its twelve rooms.
- SBG3 and SBG4 were undamaged, but powered down.
- Recovery involved site repairs, replacement servers and available backups.

Replacement capacity does not recreate lost data.



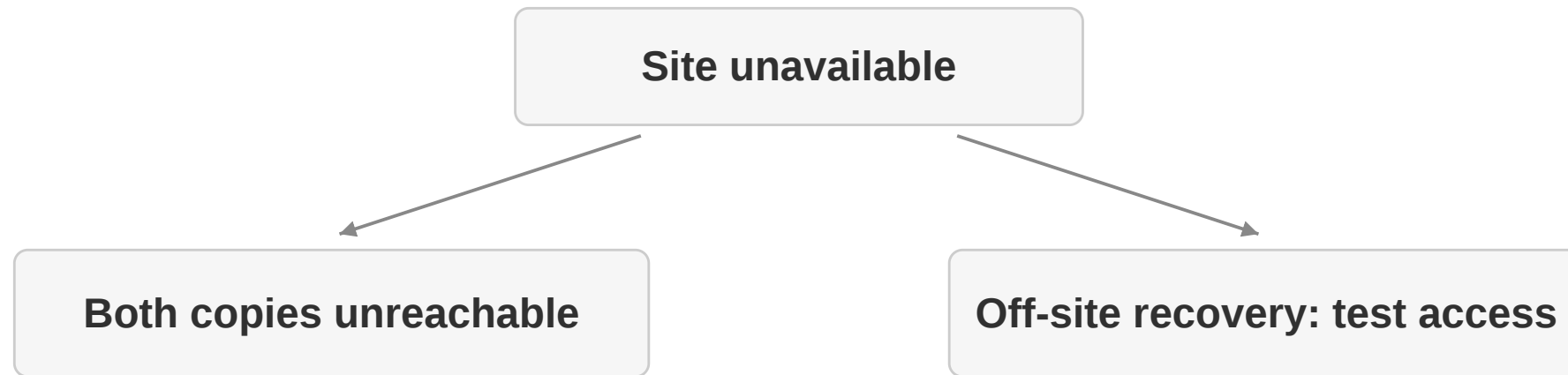
## Same fire. Different recovery.

OVHcloud Strasbourg, 10 March 2021.

| Service                 | What users lost                     | Recovery                        |
|-------------------------|-------------------------------------|---------------------------------|
| <b>Rust / Facepunch</b> | Game data on 25 affected EU servers | Data unrecoverable              |
| <b>Lichess</b>          | 24 hours of puzzle history          | Backup from another data center |

A replacement server does not bring back player progress.

## Two copies, one failure domain

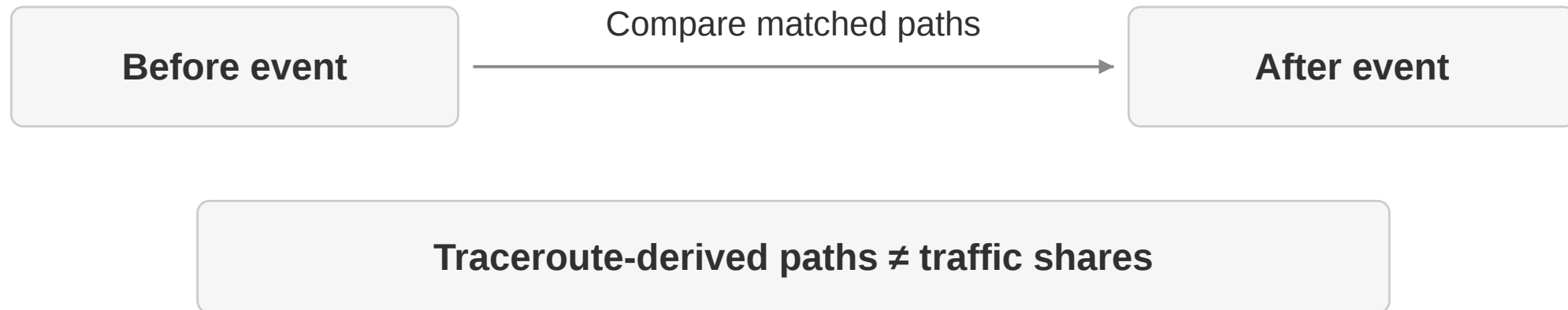


Copies do not remove a shared failure domain.

Exercise: assume both copies are inside the unavailable site.  
Which recovery plan still works?

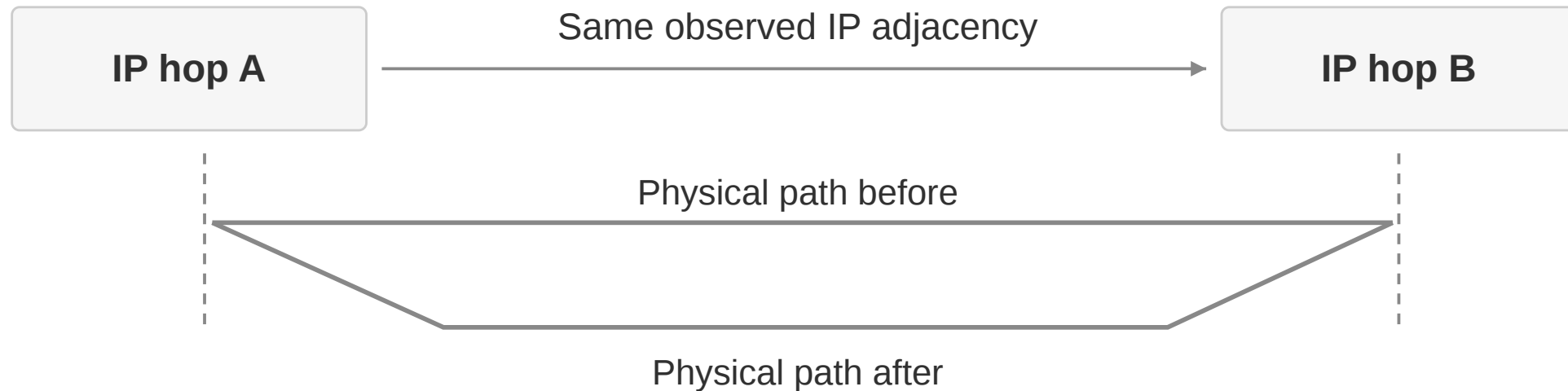
# Baltic Sea: paths after the cuts

RIPE Atlas, November 2024: Germany–Finland paths.



A different physical boundary: the cable route. Compare matched observers.  
This diagram shows the method, not traffic shares.

# The IP path can look unchanged



Traceroute alone cannot identify a damaged cable.

# Red Sea: damage is not attribution

24 February 2024: three cable systems were damaged.

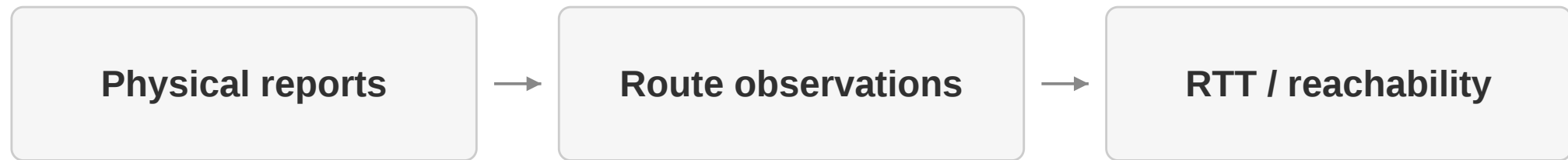
Kentik's analysis supports an anchor-dragging explanation involving the *Rubymar*.

Keep the claims separate:

- network reachability and delay: observable
- physical cause: external evidence
- deliberate attack on cables: not established by the network data

## Red Sea: the cloud takes a detour.

September 2025: connectivity can continue while paths get slower.

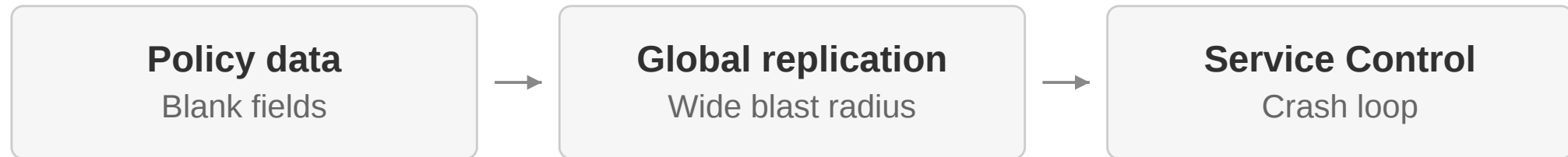


Triangulate evidence. An IP path cannot identify a damaged cable.

Which measurement separates **slower** from **unreachable**?

# Google Cloud: a dormant bug goes global

12 June 2025. One policy update crosses regional boundaries.



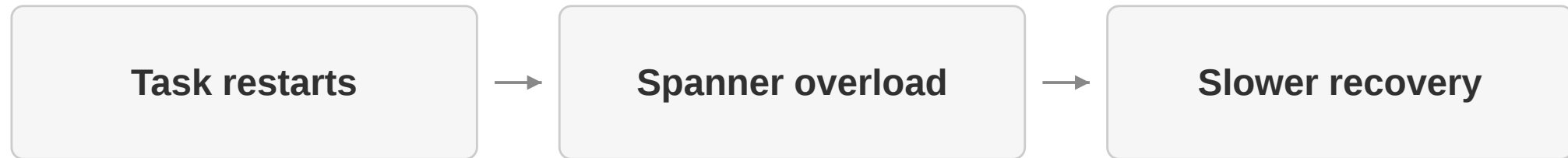
A null pointer crashed the API policy-checking process.

Service Control checks API policy. Blank fields triggered HTTP 503 errors.

**Impact:** API errors affected Cloud Storage, IAM and other products.

# Google Cloud: recovery overloaded Spanner

12 June 2025. Disabling the faulty check stopped the crash loop.



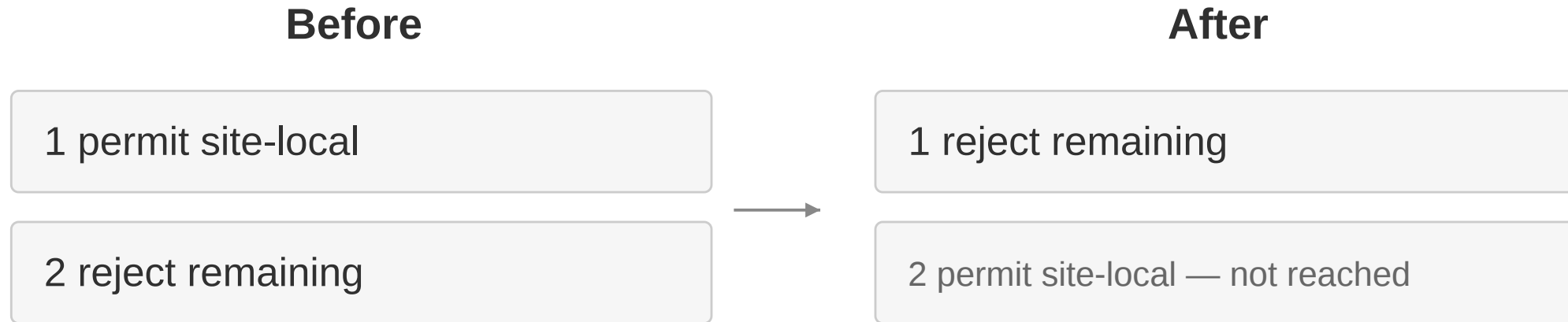
Missing randomized exponential backoff amplified the restart load.

In us-central1, Google throttled restarts and redirected database traffic.  
Some customer monitoring failed too. The first status update took about an hour.



# Cloudflare: policy order withdrew routes

21 June 2022. Simplified BGP export-policy order:



First matching rule wins. Later permission cannot undo rejection.

Site-local routes disappeared. Servers lost origin access.  
Internal load balancing also failed.

# The canary never exercised the new architecture

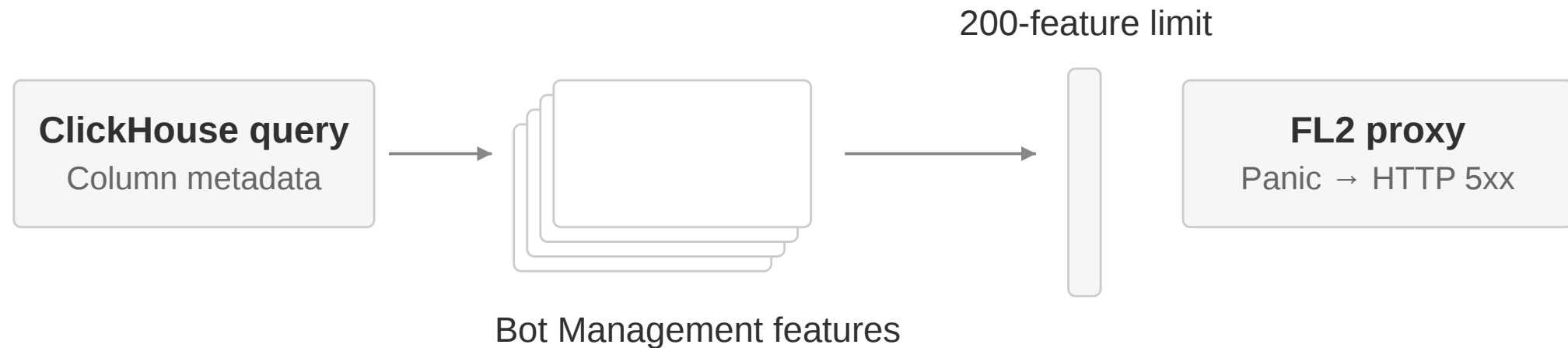
Cloudflare, 21 June 2022. Times in UTC.

- Early rollout stages used the older architecture.
- **06:27**: rollout reached 19 Multi-Colo PoP (MCP) sites.
- These sites handled about half of all requests.
- Backup access enabled rollback. The last revert finished at **07:42**.

Canary the architecture that runs the changed policy.

# Cloudflare: metadata became a bad feature file

18 November 2025. A permissions change exposed more column metadata.



Without a database filter, duplicate metadata pushed the file past **200 features**. Older proxies returned zero bot scores. Blocking depended on customer rules.

# Cloudflare: stop distributing the bad file

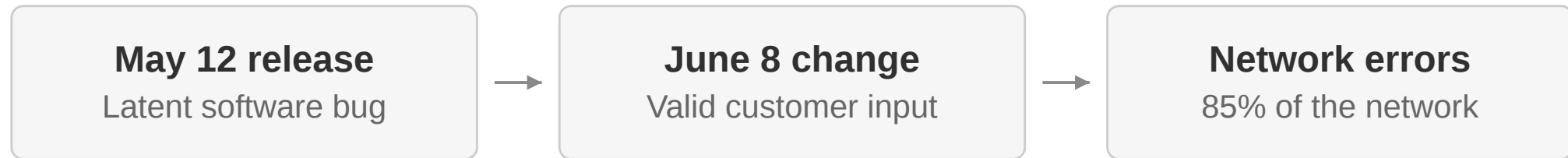
18 November 2025. Recovery milestones in UTC.

| Time  | What changed                                          |
|-------|-------------------------------------------------------|
| 14:24 | Stopped generating and distributing new feature files |
| 14:30 | Known-good file restored most traffic                 |
| 17:06 | Remaining services recovered                          |

**Impact:** websites returned 5xx errors. Turnstile failures blocked logins. Restoring the file also required restarting the core proxy.

# Fastly: valid input, latent bug

8 June 2021.



The input was valid. The software was not prepared for it.

**Impact:** CNN, The New York Times and Reddit became unavailable.  
Fastly disabled the triggering configuration. The software fix followed later.

# Fastly: detection is not recovery

8 June 2021. Times in UTC.



**12:35:** incident mitigated. **17:25:** permanent bug-fix rollout began.  
Detection, mitigation and permanent repair are different clocks.

## Name the dependency.

| Symptom                              | Pattern to investigate          |
|--------------------------------------|---------------------------------|
| Remote repair cannot start           | Recovery shares the failed path |
| Redundant routers disappear together | Shared procedure or site        |
| Every region fails after one update  | Global data distribution        |
| Dashboard stays green during impact  | Missing or stale evidence       |

These are hypotheses, not diagnoses.

## Your turn: redesign one boundary.

Pick one case. Draw its service path and recovery path.

Mark the shared dependency. Remove one dependency from recovery.  
Propose a test that proves the revised path survives.

**A second box is not a second failure domain.**



# From postmortem to early warning

Choose one failure pattern from today.

Which signal would have warned you?

How would you verify recovery?

**14 October: monitoring and observability.**

# 14 October 2026

## Monitoring / Observability

In Operations, the customer had to tell us something was wrong.

**How could the network have told us first?**

# What do we want to know?

| Service          | Network       | Device            |
|------------------|---------------|-------------------|
| Availability     | Reachability  | Health            |
| Latency          | Loss          | Capacity          |
| Correct response | Routing state | Resource pressure |

Choose the question before the tool.

# An observation needs an expectation.

Monitoring compares observations with expected behavior.  
Observability helps explain the system's state from those observations.

**Question → evidence → interpretation → action**

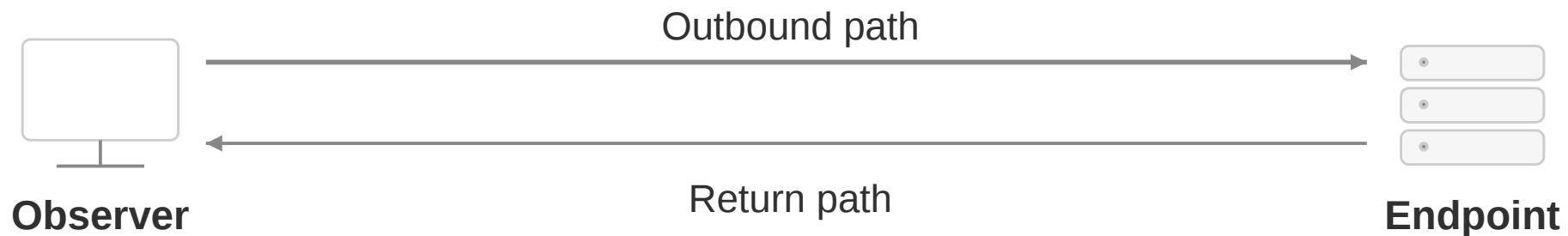
A collector transports observations. A dashboard displays them.  
Neither chooses the right question for you.

# Different signals answer different questions

| Signal         | Good for                      |
|----------------|-------------------------------|
| Metric sample  | how much / how fast?          |
| Event or log   | what changed?                 |
| Flow record    | which traffic?                |
| Packet capture | what was on the wire?         |
| Active probe   | can a client use the service? |

SNMP and gNMI are **ways to obtain data**, not extra signal types.

# Delay and loss need endpoints



RTT contains both paths and endpoint processing.

One-way delay additionally needs synchronized clocks or bounded clock error.

## Choose the probe from the question

| Probe        | Tells us                                 |
|--------------|------------------------------------------|
| ICMP echo    | IP reachability to a responding endpoint |
| TCP connect  | transport handshake                      |
| HTTP request | application transaction                  |
| traceroute   | responding hops along a path             |

A successful ping does not prove the web service works.

# Define the probe before you run it

One BOB1 probe is:

```
source:          probe01.bob1.lagoontransit.test
target:          198.51.100.10
address family:  IPv4
protocol:        ICMP echo
reply wait:      1 s
success:         ping exits successfully
```

The lab exporter also runs DNS probes and repeats the checks continuously.

Two checks called "reachability" can still measure different things.



# The monitoring brief

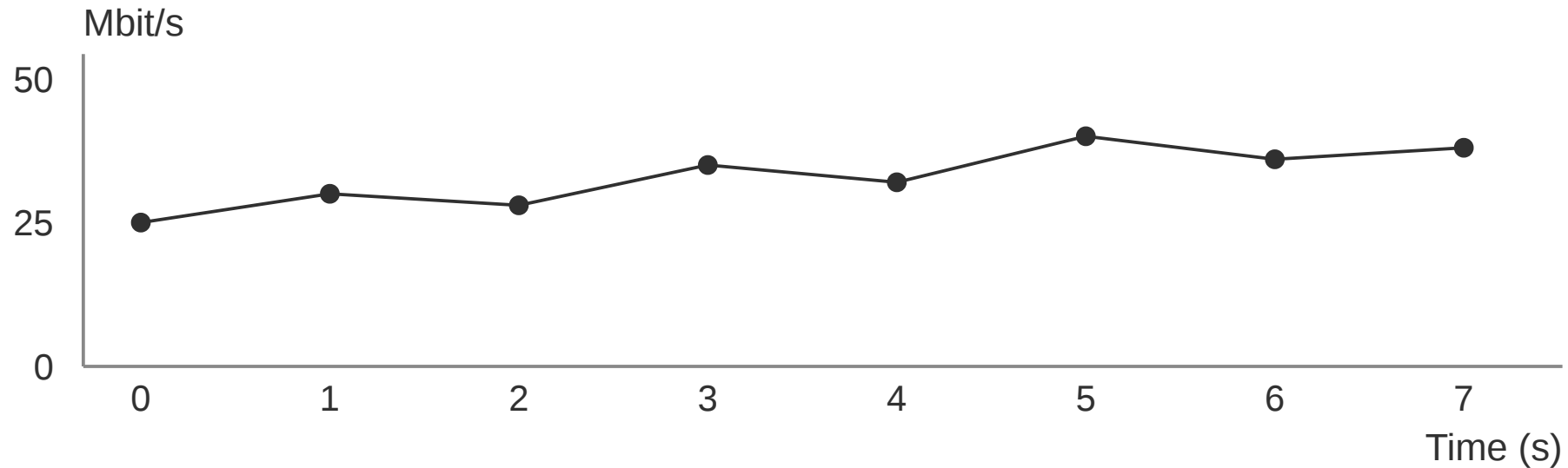
**40 targets:** 20 services, IPv4 and IPv6.

Two observation sources per target.

- Detect interruptions of **5 seconds or longer**.
- Stay within **20 HTTP requests/s**.
- Mark stale data **unknown within 15 seconds**.

Sizing exercise: this exceeds the small BOB1 probe set.

# One ReefNet interface, one series

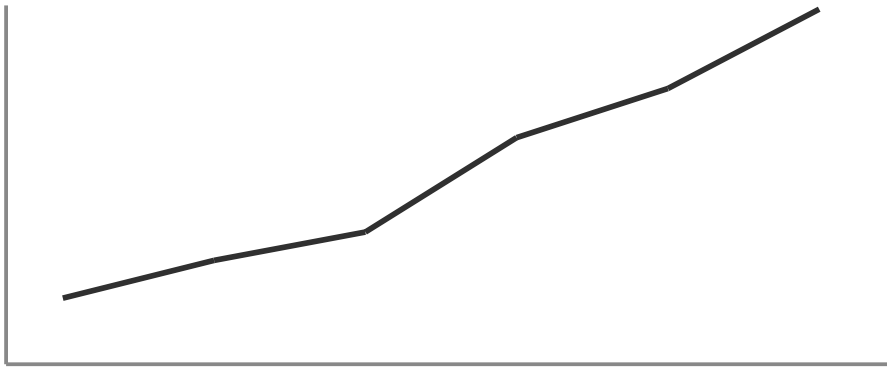


edge01, ethernet-1/3: traffic arriving from Lagoon Transit.

Each sample keeps that identity. The plotted values are illustrative.

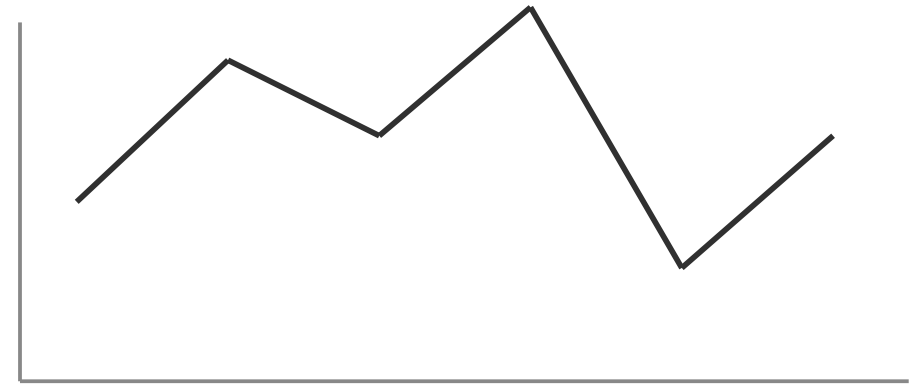
# Counter or gauge?

**Counter**



Bytes on edge01 / ethernet-1/3

**Gauge**



DNS-check duration from Lagoon

The byte counter accumulates. The probe duration can rise or fall.

# Timestamps in the pipeline

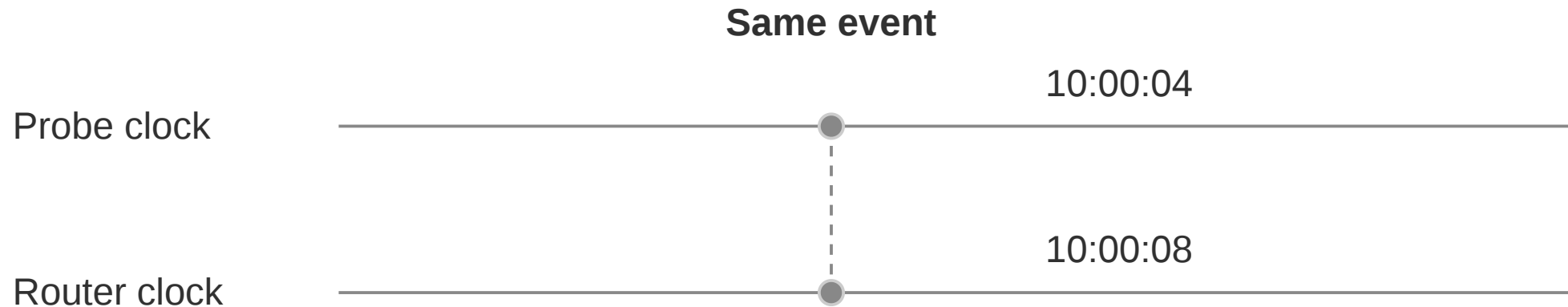
One event, several clocks.



Clock offset and transport delay are different quantities.

Subtract timestamps only after checking clock source and offset.

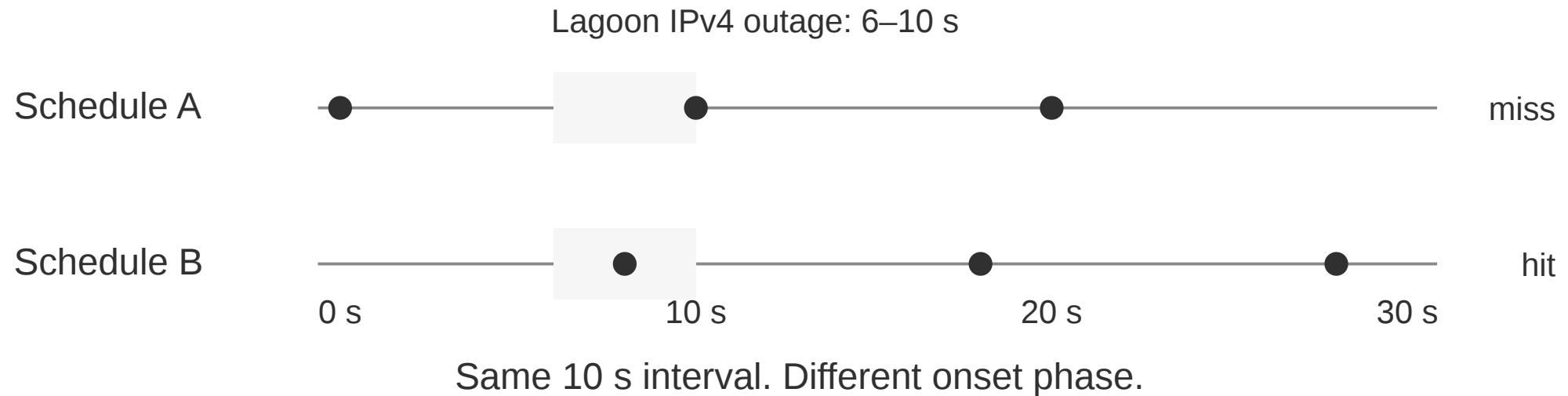
# Clock error changes the story



Unsynchronized clocks distort the apparent event order.

Bound clock error before inferring event order.

# Four seconds can disappear



Imagine a short Lagoon IPv4 outage. A healthy sample can miss it.

# Sampling leaves gaps

An event can begin and end between observations.

- Interval and onset phase affect detection.
- Equal average rates can hide different gaps.
- A stream can carry sampled state.

Streaming periodically sampled values still leaves gaps.

# Budget the ReefNet probe round

Ocean Research is checked from **Lagoon and Pacific**.  
Each source tests **IPv4 and IPv6**, using **ICMP and DNS**.

**$2 \times 2 \times 2 = 8$  checks per round.**

Planning example: a round every 5 seconds needs **1.6 checks/s**.

In the lab, each round finishes before a 1-second pause.  
What happens to the interval when a check times out?



# Before subtracting counters

Check identity, timestamps and discontinuities.

A negative delta may mean **reset, wrap or bad data**.

Two values cannot reveal multiple wraps.

Invalid evidence becomes **unknown**, never a fabricated rate.

## Turn ReefNet byte counters into bit/s

Use `in-octets` on `edge01`, interface `ethernet-1/3`.

$$rate = \frac{8(C_2 - C_1)}{t_2 - t_1}$$

Counter in **bytes**, time in **seconds**. Use a reset-free interval.

Compare the result with the **50 Mbit/s Lagoon handoff**.

# Calculate the Lagoon ingress rate

Illustrative `in-octets` on `edge01`, interface `ethernet-1/3`.

| Time     | Byte counter | Reset?         |
|----------|--------------|----------------|
| 10:00:00 | 1,000,000    | Initial sample |
| 10:00:10 | 32,250,000   | No             |
| 10:00:20 | 20,000       | Yes            |
| 10:00:32 | 37,520,000   | No             |

Four minutes: calculate valid rates. Compare them with the 50 Mbit/s handoff.

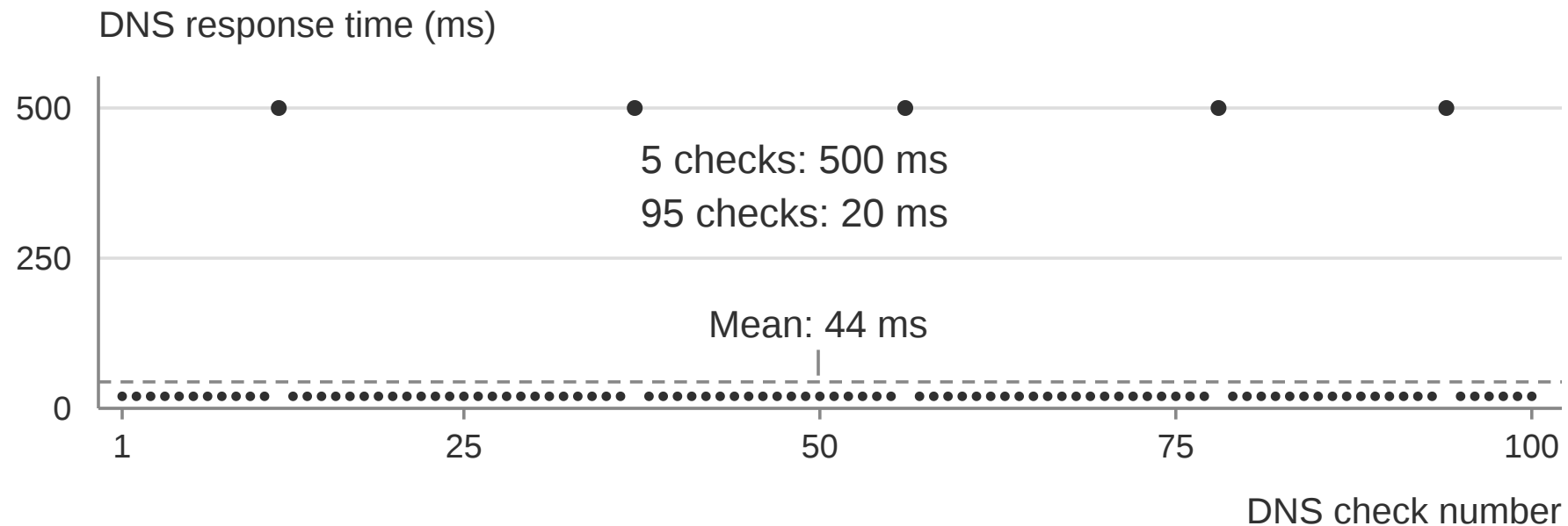
## 25 Mbit/s on a 50 Mbit/s handoff

| Interval | Calculation                | Result    |
|----------|----------------------------|-----------|
| 00–10 s  | $8 \times 31,250,000 / 10$ | 25 Mbit/s |
| 10–20 s  | Counter reset              | Exclude   |
| 20–32 s  | $8 \times 37,500,000 / 12$ | 25 Mbit/s |

Both valid intervals use **50% of the Lagoon handoff capacity**.

The 20,000-byte reading starts a new comparison after the reset.

# What Lagoon sees when it probes DNS



100 illustrative DNS checks to Ocean Research. Start with the individual waits.  
Would the **44 ms mean** explain the five long waits?

# Read the distribution

| View      | What it tells us                |
|-----------|---------------------------------|
| Median    | Middle observation              |
| p95       | Threshold covering at least 95% |
| Histogram | Counts within ranges            |
| ECDF      | Fraction at or below each value |

A percentile is not an “average slow request”.

# Percentiles do not average

A separate DNS example for `data.oceanresearch.test`. Nearest-rank p95:

| Source  | Checks | Duration of each check |
|---------|--------|------------------------|
| Lagoon  | 100    | 20 ms                  |
| Pacific | 1      | 1,000 ms               |

Average of the two p95s: **510 ms**.

Combined observations: **p95 = 20 ms**.

Combine observations first, then calculate the percentile.

# Start from a healthy network.

In the lab directory:

```
make monitoring  
make enter NODE=operations01.bob1.reefnet.test
```

This starts healthy routing, telemetry and **25 Mbit/s** background traffic.

| View    | Open                               |
|---------|------------------------------------|
| Grafana | <code>http://localhost:3000</code> |
| NetBox  | <code>http://localhost:8000</code> |

Grafana: **ReefNet / BOB1** folder. Refresh every 5 seconds.



# SNMP: what does the device report?

|             |                                            |
|-------------|--------------------------------------------|
| Observation | OID, typed value, device, row, poll time   |
| Limit       | Polls can miss short events                |
| ReefNet     | Compare with gNMI. SNMP is not provisioned |

A manager queries an agent. **OID**: object identifier.

The **MIB**, Management Information Base, defines object meaning.

Interface “up” does not prove service health.

## Keep the object identity.

```
GET      request-id 41    ifOperStatus.17
RESPONSE request-id 41    noError    INTEGER up(1)
```

The OID selects an object. 17 selects an interface row.

Check response status, device identity and the row's name/alias.

After restart or replacement, verify the mapping before joining counters.

# Syslog: what changed?

|             |                                       |
|-------------|---------------------------------------|
| Observation | Time, host, severity, app and message |
| Limit       | Missing logs do not prove health      |
| ReefNet     | SR Linux → Alloy → Loki → Grafana     |

Keep source and receive times. Severity is the device's judgment.

## IPFIX: which traffic crossed here?

|             |                                                       |
|-------------|-------------------------------------------------------|
| Observation | Flow keys, counts and time interval                   |
| Limit       | Typical records omit payload. Sampling affects counts |
| ReefNet     | Record exercise without a live exporter               |

A **template** defines the fields. Keep its exporter and observation domain.  
Port 443 alone does not identify an application.

# A template ID is not globally unique.

Illustrative ReefNet exporters, not provisioned in BOB1:

| Exporter | Template 256 means           |
|----------|------------------------------|
| edge01   | source, destination, bytes   |
| edge02   | source, destination, packets |

Record from edge02: 203.0.113.10, 198.51.100.10, 8000

That is **8,000 packets**, not bytes.

What context must the collector retain to decode it?

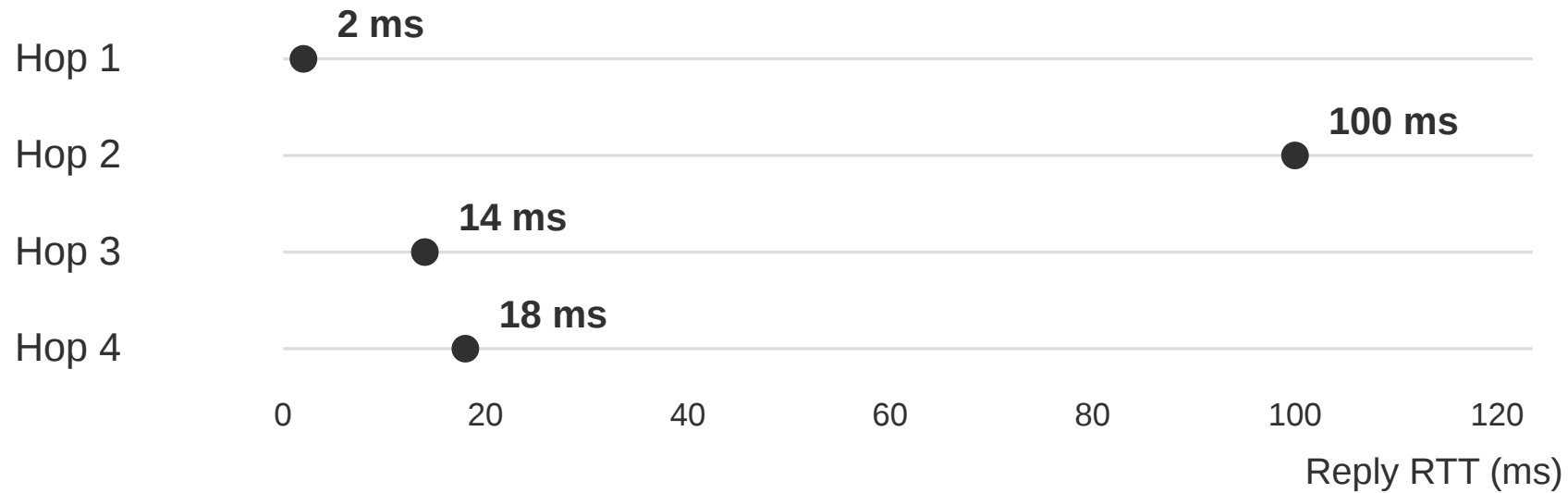
## Active probes: does this exchange work?

|             |                                                |
|-------------|------------------------------------------------|
| Observation | Source, target, family, protocol, time, result |
| Limit       | Failure does not locate the fault              |
| ReefNet     | ICMP/DNS exporters and manual HTTP checks      |

The DNS exporter checks command success, not the returned answer.

# Traceroute: read replies carefully

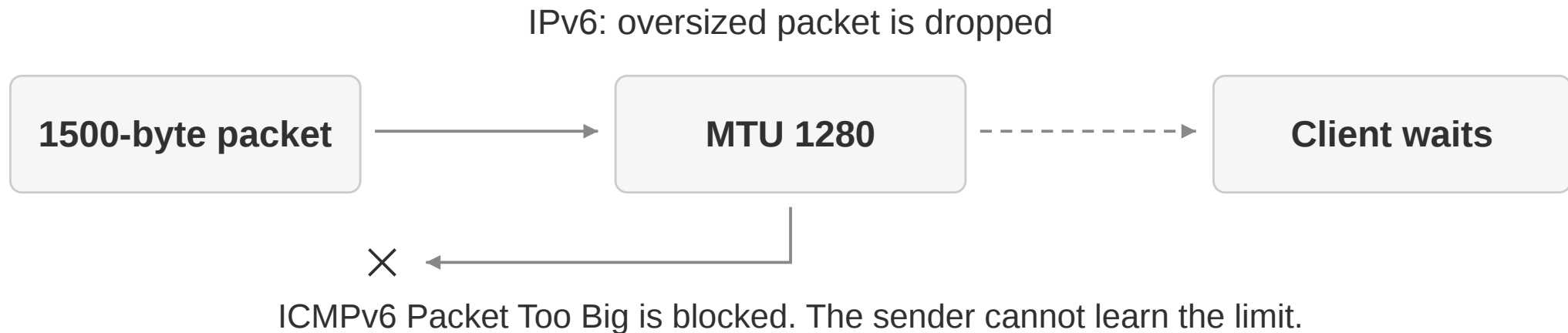
One illustrative reply per hop



Hop 2 replies slowly. It does not delay all forwarded packets by 100 ms.

# Small ping, stalled download.

Thought experiment: small IPv6 responses work. Large ones stall.



Force a large IPv6 transfer after repair. Recheck IPv4.



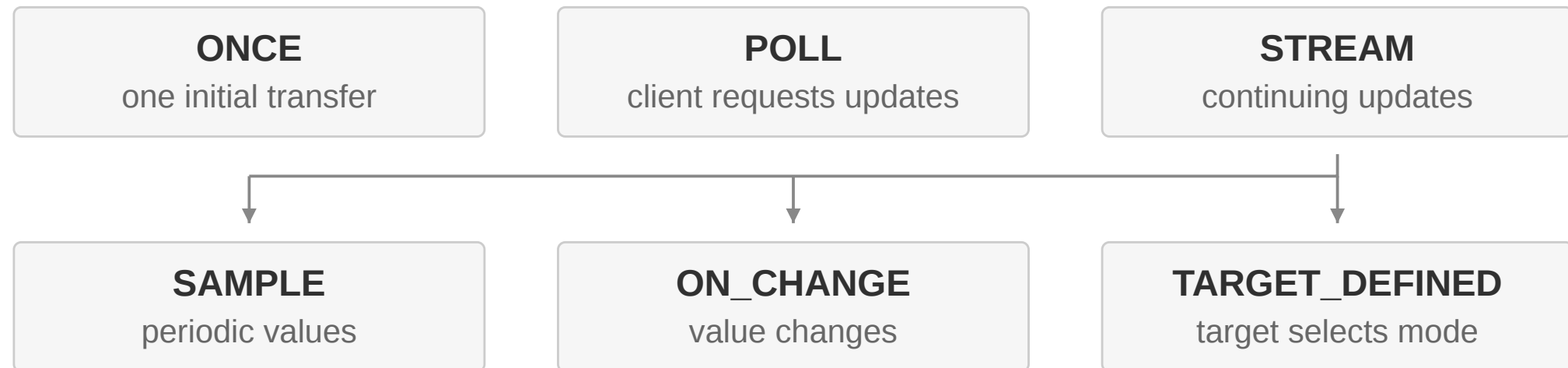
## Streaming state: what changed?

|             |                                                |
|-------------|------------------------------------------------|
| Observation | Target, keyed path, timestamp, update/deletion |
| Limit       | Reconnect cannot restore every missed event    |
| ReefNet     | gNMlc samples SR Linux state and counters      |

The delivery mechanism does not define freshness.

# Choose the subscription semantics.

SubscriptionList mode



SAMPLE is periodic. ON\_CHANGE depends on target support.

# Subscribe to the live interface counters

Inside `operations01.bob1.reefnet.test`:

```
gnmic -a edge01.bob1.reefnet.test:57400 \  
-u admin -p 'NokiaSr11!' --skip-verify \  
--encoding json_ietf \  
subscribe \  
--path '/interface[name=ethernet-1/3]/statistics' \  
--stream-mode sample --sample-interval 5s
```

Watch Lagoon ingress on `ethernet-1/3`. Grafana shows its rate in bit/s.  
**Ctrl+C** stops this manual subscription.

# Change load and watch the slope

The Monitoring scenario starts with **25 Mbit/s** offered load.

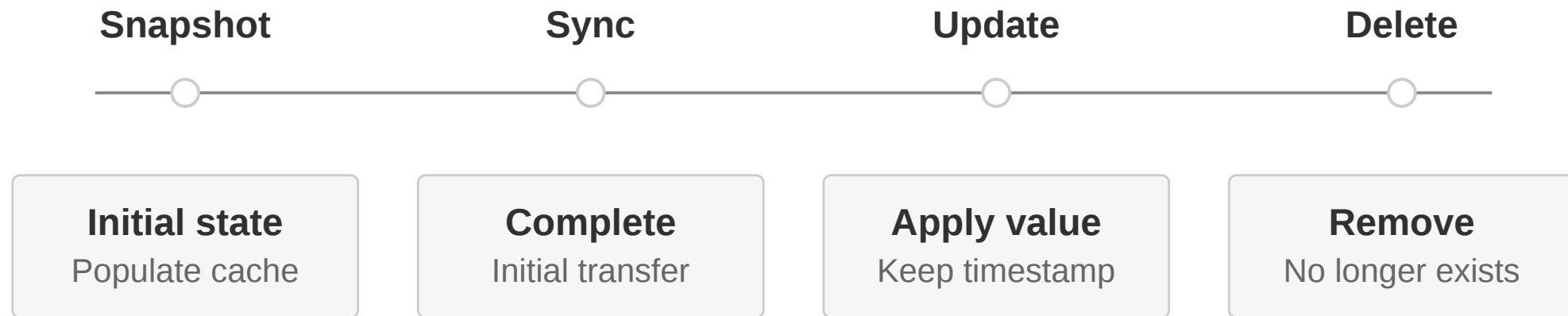
From the **lab-directory** terminal:

```
make traffic-40mbit  
make traffic-10mbit  
make traffic-25mbit
```

Run one command at a time. Observe each level for **30 seconds**.

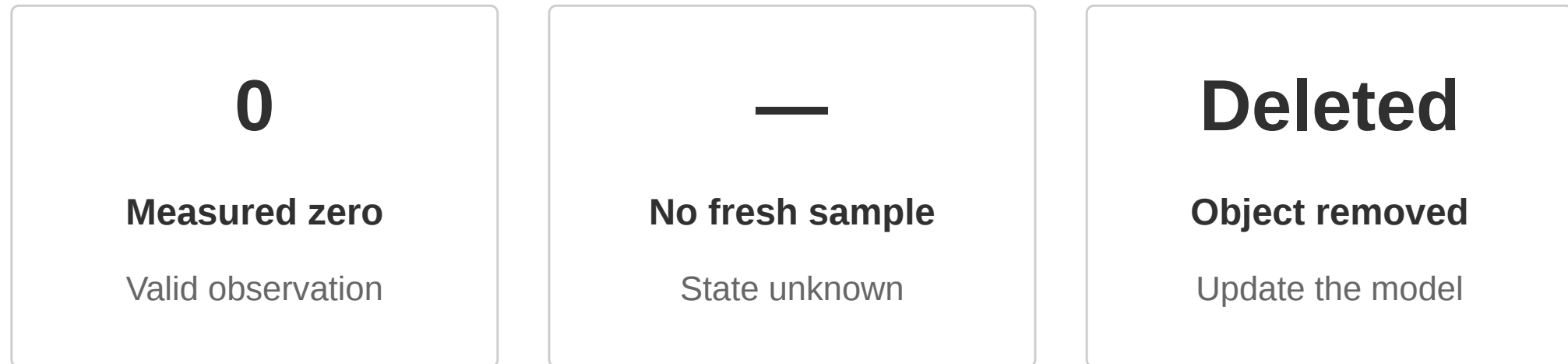
Which counter changes now? When does Grafana show the change?

# Build local state from a gNMI stream



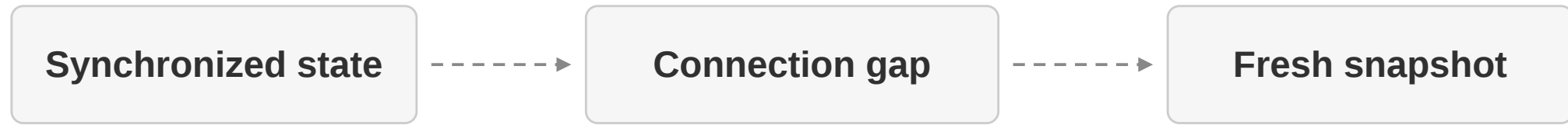
Sync completes the initial transfer. It does not guarantee future freshness.

# Deletion is not zero



These states need different storage, query and alert behavior.

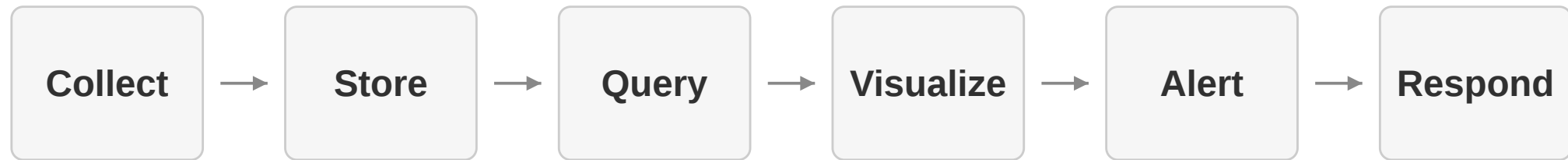
## Reconnect cannot restore lost history.



Events during the gap may be irretrievable.

Rebuild current state after reconnect. Keep the observation gap visible.

## Turn observations into a response.



Every boundary can lose meaning, freshness or context.  
Keep enough metadata to detect that loss.

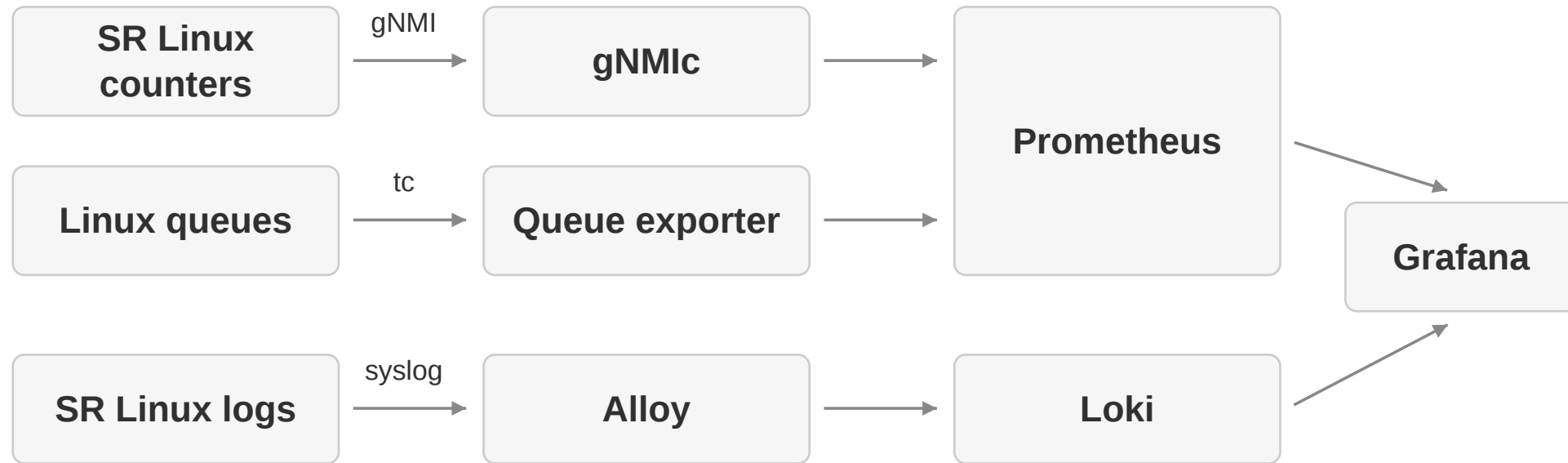


# Where should the collector live?

| Placement          | Main dependency                     |
|--------------------|-------------------------------------|
| Central            | WAN reachability                    |
| At the site        | Site power and local collector      |
| Several collectors | Coordination and duplicate handling |

Test isolation. A second collector may share the first one's failure domain.

# Metrics and logs in ReefNet



Grafana queries metrics and logs. Each collection path can fail separately.

# Prometheus stores time series.



A successful scrape can still return stale source data.

Trace a suspicious panel back to its source timestamp.

# Cardinality: count ReefNet's series



## 8 series for one metric

One target: data.oceanresearch.test

`count(reefnet_probe_success)` should return **8** after both exporters report.

Success, duration and timestamp give **24 probe series** in total.

# Would you add a request ID?

A proposed change to ReefNet's probe metric:

```
source="probe01.bob1.lagoontransit.test"  
probe="icmp"  
af="ipv4"  
target="data.oceanresearch.test"  
request_id="a-new-value-for-every-run"
```

The first four labels already identify the check.

What would the new label do to cardinality? Where should the ID go?

## When would we need a queue?

Do independent consumers need **replay** from the same observations?

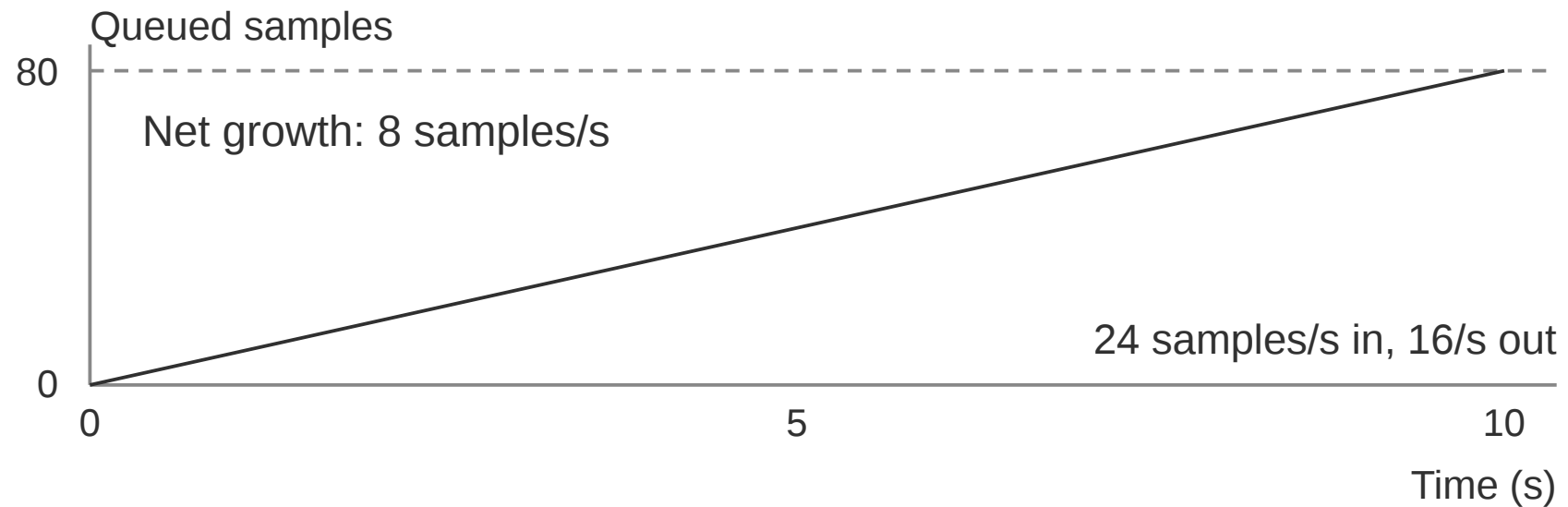
If yes, a retained event log may help.

For one dashboard and one alert path, direct storage may suffice.

Kafka also needs retention, lag monitoring and recovery.

# What if ReefNet adds a slow archive?

Design exercise: forward the probe samples to a remote archive.



The buffer buys ten seconds. It cannot fix the slow writer.

# Retention changes what you can ask.

Raw samples



4-s means



12-s mean

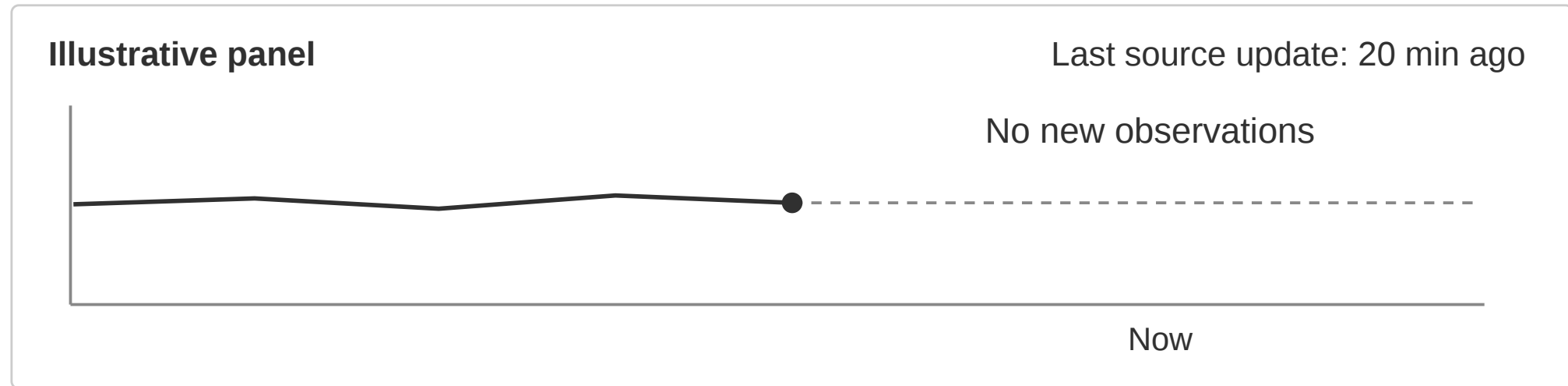


Same scales. Twelve seconds per row.

Aggregation saves space but can erase short failures.



## Up can still mean stale.

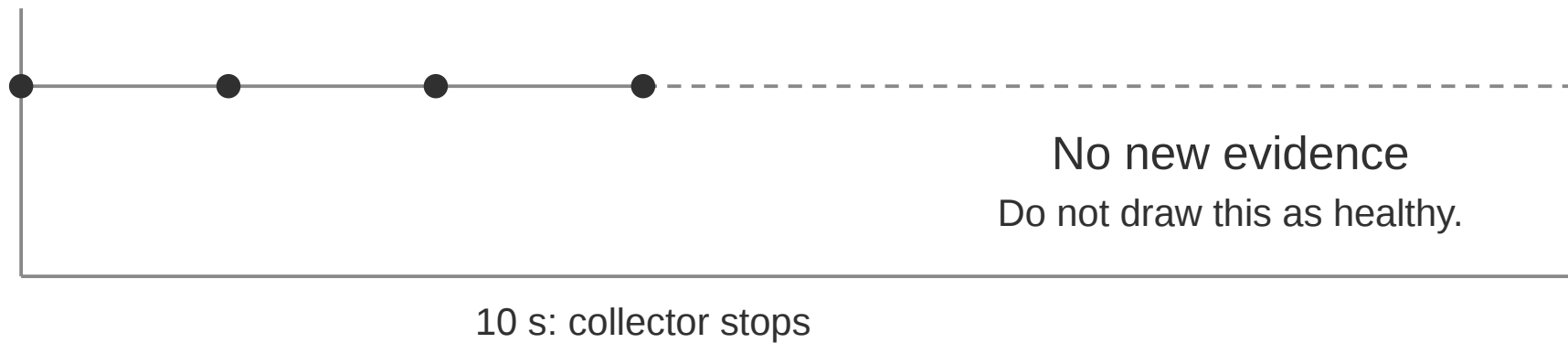


Watch source age, even when the scrape succeeds.

# Healthy yesterday.

Today's status is still unknown.

# Missing data on a dashboard



When should the panel stop looking healthy?

# Monitor the monitoring

| Component         | Watch                              |
|-------------------|------------------------------------|
| Collector         | Last valid observation             |
| Broker / consumer | Buffer use, progress and event age |
| Rule engine       | Errors and evaluation time         |
| Notification path | Delivery result                    |

Check the central monitoring service from outside.

# Choose a dashboard

| Question            | BOB1 dashboard   |
|---------------------|------------------|
| Scope?              | Network Overview |
| Link and path?      | Topology & Paths |
| Investigate cause?  | Fault Analysis   |
| Service responding? | Service Health   |
| Fresh data?         | Telemetry Health |
| Router detail?      | Device Detail    |

One question. One entity. One time window.

## Same symptom, different causes

Lagoon IPv4 fails in both cases.

| Evidence                 | Export rejected | Session down    |
|--------------------------|-----------------|-----------------|
| BGP session              | Established     | Not established |
| Customer prefix exported | No              | No              |

Session state distinguishes these hypotheses.  
Next, test a different fault: an internal link goes down.

## Watch a known link failure.

In Grafana **Topology & Paths**, select ReefNet core-b.  
Record **admin enable + oper up**. In the lab directory:

```
make fault-link
```

On ReefNet Edge 01:

```
show interface ethernet-1/4 detail  
show network-instance default protocols ospf neighbor
```

Expect **admin disable + oper down**, then routing changes.

# Admin state and oper state

The fault just injected disables `ethernet-1/4` on ReefNet Edge 01:

```
/interface[name=ethernet-1/4]/admin-state    "disable"  
/interface[name=ethernet-1/4]/oper-state     "down"
```

**Admin:** configured intent. **Oper:** observed behavior.

Prometheus exposes `reefnet_interface_admin_state` and `reefnet_interface_oper_state`. Read them with their timestamps.



# Restore the link and watch convergence

In the lab directory:

```
make clear-link
```

Verify admin enable, oper up and the OSPF adjacency.  
Correlate event time with external reachability.

The Lagoon-to-customer flow does not need `core-b`.  
Its service check can remain green while the topology changes.

# Capacity is not throughput

**Lagoon probe**

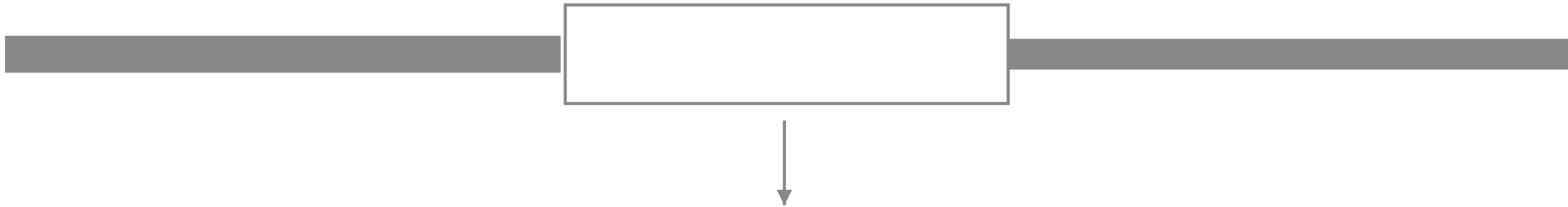
60 Mbit/s

**Lagoon handoff**

50 Mbit/s limit

**Ocean Research**

$\leq 50$  Mbit/s

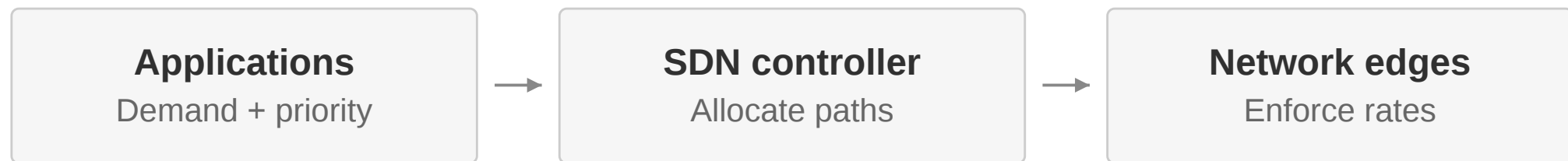


Excess: queue, drop or sender backpressure

A faster customer handoff does not remove the Lagoon bottleneck.

# Google B4: busy by design

Google's private data-center WAN ran many links **near 100% utilization**.  
Reported in the B4 paper, SIGCOMM 2013.

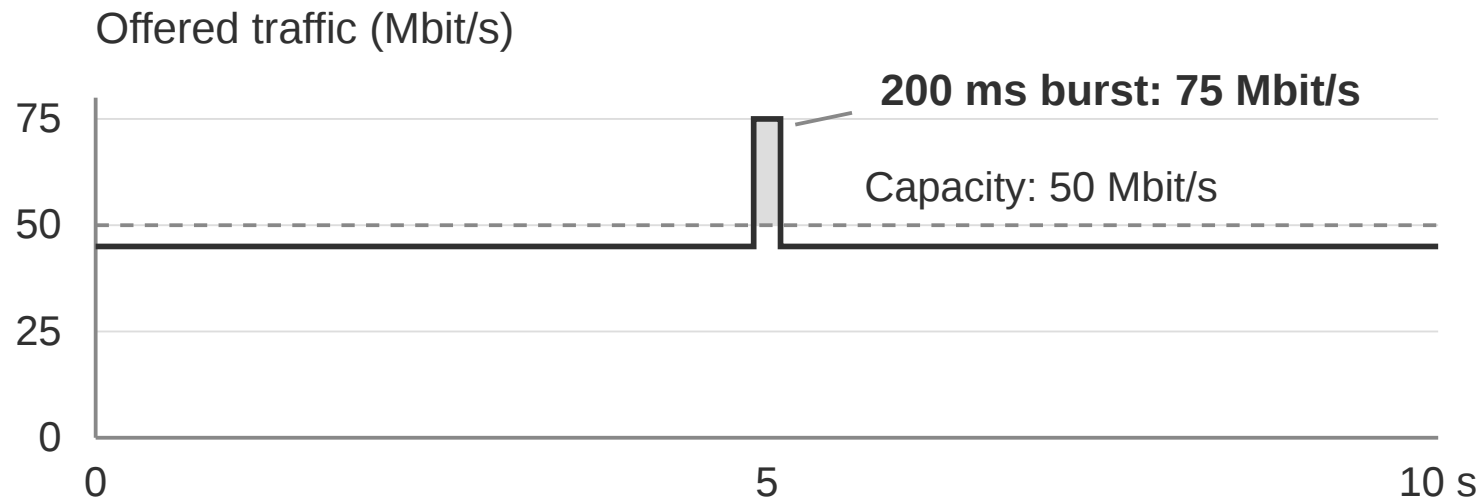


Capacity is allocated across paths as demand and failures change.

Bulk transfers yield to higher priorities. Google controls the senders, too.

High utilization needs **traffic control and a failure plan**.

## 90% average. Still dropping packets.



Dashboard

≈90%

mean output  
over 10 s

Synthetic Lagoon example. The brief peak is lost in the average.

The burst fills the queue. **Once it is full, packets are dropped.**

Check **drop counters and queue peaks**, not just average utilization.

# Cross the capacity boundary.

Transit: **50 Mbit/s**. Customer handoff: **100 Mbit/s**.

```
make traffic-40mbit  
make traffic-60mbit  
make traffic-75mbit
```

Observe each level for **30 seconds**. Compare load, rate and throughput.  
Leave **75 Mbit/s** running for the next checks.

Grafana: compare **Ingress discards** and **Egress discards**.  
For Lagoon → ReefNet, inspect **Linux queue** on Lagoon Transit.

# Compare offered and delivered traffic.

On ReefNet Edge 01, inspect transit and customer interfaces:

```
show interface ethernet-1/3 detail
show interface ethernet-1/1 detail
```

From the lab directory, inspect the receiving service:

```
make traffic-diagnostics
make enter NODE=service01.bob1.oceanresearch.test
tail -f /tmp/iperf-server.log
```

After recording results: **Ctrl+C**, then `exit`.

In the lab terminal, run `make traffic-10mbit`.

# Keep the simulator out of the result

At 75 Mbit/s with 1,200-byte UDP datagrams:

**$75,000,000 / (1,200 \times 8) \approx 7,800$  packets/s**

SR Linux container limit: **10,000 packets/s**.

The **50 Mbit/s transit link** should limit first. Check host load and drops.

The cap uses a **Linux egress queue** on Lagoon Transit.

Grafana shows its drops separately from SR Linux ingress/egress discards.

A **5-minute increase** can plateau during steady loss.

## Short bursts, long rate windows

Now isolate averaging: **40 Mbit/s stays below the 50 Mbit/s limit.**

```
make traffic-stop  
make traffic-burst-40mbit
```

This five-second burst adds bytes. A long rate window smooths its peak.

Inspect the graph. Can it recover the burst's height and duration?

Then restore `make traffic-25mbit` in the lab terminal.



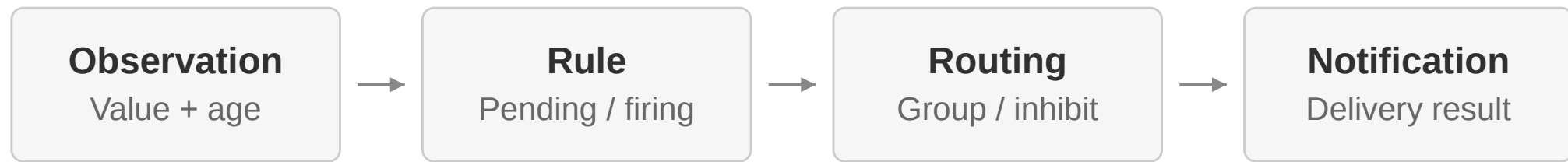
## Several signals, one incident.



One controlled link fault, several observations.

Illustrative timestamps. Check clock offsets before inferring order.

## The trigger is not the notification.



The rule fires, but nobody is paged. Which boundary failed?

# An alert rule from the lab metric

Illustrative Prometheus rule, using the actual BOB1 metric:

```
- alert: LagoonIPv4ProbeFailed
  expr: |
    reefnet_probe_success{
      source="probe01.bob1.lagoontransit.test",
      probe="icmp", af="ipv4"
    } == 0
  for: 30s
```

The condition must stay active across evaluations for 30 seconds.  
This sustained-failure rule does **not** meet our 5-second detection brief.

# Alert on stale observations

```
time() - reefnet_probe_last_run_timestamp_seconds > 15
```

This detects old exported results.

Also check missing series and scrape health.

Use the expected target set to detect disappearing sources.

# Flapping and notification noise

**Flapping:** repeated state changes.

| Mechanism     | Purpose                                        |
|---------------|------------------------------------------------|
| Deduplication | Skip duplicate notifications                   |
| Inhibition    | Mute notifications while a related alert fires |
| Grouping      | Combine related notifications                  |

A firing delay reduces noise but can hide short outages.

# Precision and recall

|          | Fault              | No fault           |
|----------|--------------------|--------------------|
| Alert    | 90 true positives  | 99 false positives |
| No alert | 10 false negatives | 801 true negatives |

1,000 illustrative observation windows.

**Recall:  $90/100 = 90\%$ .** We detect most faults.

**Precision:  $90/189 = 47.6\%$ .** More than half the alerts are false.

# SLI and SLO

**SLI:** measured service behavior

**SLO:** target for that behavior over a defined window

$$SLI = \frac{\textit{successful requests}}{\textit{eligible requests}}$$

Define how missing records are handled before calculating the SLI.

# An error budget for the Lagoon probe

Example target: IPv4 ICMP checks from Lagoon to Ocean Research.

**SLO: 99.9%** across the next **10,000 completed checks**.

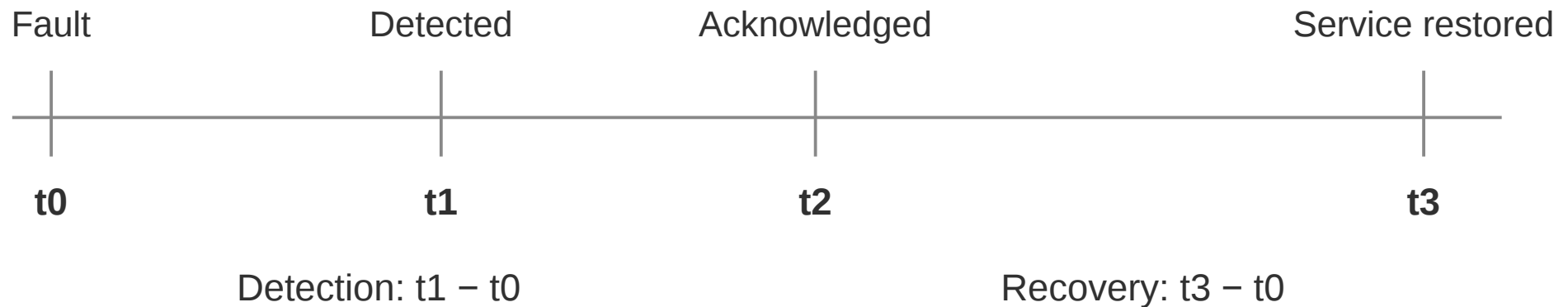
$$10000 \times (1 - 0.999) = 10 \text{ allowed failures}$$

If six checks fail, four failures remain in the budget.

Missing checks need separate reporting. This SLI describes the probe.



# Detection and recovery are different clocks.



Name the endpoints before reporting a recovery duration.

## Rehearse a response.

Five-minute tabletop during a change:

| Vantage point | Service result |
|---------------|----------------|
| Lagoon, IPv4  | Fails          |
| Pacific, IPv4 | Works          |

Name the first observation, incident owner and backout trigger.  
Explain why that observation distinguishes the leading hypotheses.

## Test the monitoring brief

| Requirement              | Evidence                               |
|--------------------------|----------------------------------------|
| Detect 5 s interruptions | Detection by onset phase               |
| $\leq 20$ requests/s     | Requests, including retries            |
| Unknown within 15 s      | Stop collection and time it            |
| Useful diagnosis         | Distinguish route loss and queue drops |

Return to the **40-target brief**. Sketch the smallest design that passes.

# Monitoring: close the loop.

Save the fault timeline, collection gaps and recovery evidence.

```
make traffic-stop  
make down
```

**observe → question → measure → research**

30 October: turn the sampling limitation into a testable research question.

# 30 October 2026

## Research questions and sources

Monitoring gave us a limitation: short events can disappear between samples.

Now we turn that into a question we can actually test.

# Leave with a testable research brief

Today we work from one observation to a study we can carry out.

1. State a question that a comparison can answer.
2. Find prior work and evidence that fit the question.
3. Choose a baseline and expose the biggest feasibility risk.

Our running example: **short outages missed between probes.**

# From a problem to a research question

Observation: short failures escape our polling schedule.

| Decision      | Study design                          |
|---------------|---------------------------------------|
| Compare       | Periodic and randomized schedules     |
| Hold constant | Probe count, fault workload, endpoint |
| Measure       | Detection fraction and delay          |
| Bound         | Tested durations and conditions       |

**At equal cost, which schedule catches short outages better?**

# Write a measurable question.

Name the **factor**, **outcome** and **conditions**.

| At equal probe cost, which schedule detects more 1–10 s outages?

Define the schedules, fault distribution and detection criterion.  
The comparison must allow your preferred method to lose.



# What makes a scientific argument?



The claim cannot be stronger than its weakest evidence boundary.

Your contribution is the finding this evidence supports.

# What kind of paper will you write?

| Approach                    | Main contribution                      |
|-----------------------------|----------------------------------------|
| Measurement study           | Describe observed behavior             |
| Comparative evaluation      | Compare methods fairly                 |
| Reproduction + extension    | Check and extend a published result    |
| System or operational study | Build or investigate a concrete system |

Every paper needs a question, evidence and a limited claim.

## Task: challenge the claim

"Randomized polling detects outages faster."

Two minutes:

1. Name a result that would contradict this.
2. Specify outage durations, baseline and measured outcome.
3. Choose a success criterion that allows the method to lose.

## Plan the measurement from the claim

| Claim            | Metric                | Evidence              |
|------------------|-----------------------|-----------------------|
| faster detection | detection delay       | repeated known faults |
| lower load       | bytes/s or requests/s | collector + target    |
| fewer misses     | detection rate        | known injected events |

If the measurement cannot test the claim, change the claim or the experiment.

# Observation or intervention?

| Observational study                      | Controlled experiment                  |
|------------------------------------------|----------------------------------------|
| polling gaps coincide with missed events | deliberately vary the polling schedule |
| shows association                        | tests an intervention                  |

A **confounder** changes both condition and outcome.

Injecting a fault is not enough. The comparison still has to be controlled.

## Find the closest comparison.

Search by **problem, mechanism and method**, then follow citations.

Record query, database, date and inclusion criteria.

Compare assumptions, baselines and evidence across the closest papers.

Your contribution changes one justified assumption or tests a new boundary.

“Nobody has done this before” is not a literature review.

## Read in three passes.

| Pass      | Record                                           |
|-----------|--------------------------------------------------|
| Relevance | Question, main result, contribution              |
| Method    | Signal, fault model, baseline, experimental unit |
| Challenge | Missing evidence, alternatives, transfer limits  |

Write one accepted claim, one limitation and one next question.  
A blank comparison cell is a reading task, not automatically a gap.

## Read the claim before the graph.

Trinocular studies Internet reliability through adaptive probing.  
Its unit is an analyzable **IPv4 /24 block**, not every application.

| Read for   | Question                                       |
|------------|------------------------------------------------|
| Scope      | Which blocks and vantage points are included?  |
| Method     | Outage definition and probe budget             |
| Validation | What independent evidence checks the detector? |

A reachable block does not prove every host or application works.



# Broader networking research

| Venue                             | Typical contribution                             |
|-----------------------------------|--------------------------------------------------|
| <u>ACM SIGCOMM</u>                | networking mechanisms, architectures and systems |
| <u>USENIX NSDI</u>                | implemented networked systems                    |
| <u>IEEE INFOCOM</u>               | protocols, algorithms and performance            |
| <u>ACM CoNEXT</u>                 | experimental networking systems                  |
| <u>IFIP Networking</u>            | protocols, systems and network performance       |
| <u>IEEE GLOBECOM</u> / <u>ICC</u> | broad communications and networking              |

The venue follows the **contribution**, not just the topic.

# Who develops Internet technology?

## IETF

Working Groups engineer interoperable Internet protocols and standards.

## IRTF

Research Groups study long-term questions about Internet technology.

# Where do operators compare notes?

## RIPE Meetings

Routing, addressing, measurement and Internet coordination

## DENOG and NANOG

Backbone operations, routing, peering and incidents

## NAF and AutoCon

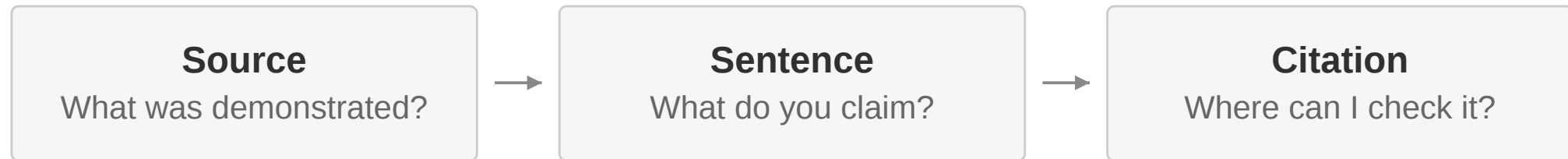
Network automation, orchestration and operational tooling

## Match the source to the claim.

| Claim                      | Evidence source                          |
|----------------------------|------------------------------------------|
| Required protocol behavior | Specification and exact section          |
| Implemented feature        | Versioned code or official documentation |
| Measured improvement       | Paper and evaluation data                |
| Production failure         | Attributed operator report               |

For RFCs: check status, updates, errata and normative language.

# Does the citation support the sentence?



Match the sentence to the evidence in the source, not merely to its topic.

# Paraphrase the idea, not the word order

| Approach                                   | Problem or purpose                |
|--------------------------------------------|-----------------------------------|
| Replace a few words in the source          | Still tracks the original wording |
| Read, take notes, explain in your argument | Paraphrase with attribution       |
| Preserve exact wording in quotation marks  | Direct quotation, with location   |

Use quotation marks for copied wording, even with a citation.

# A DOI is not evidence.

Read the result. Then check the claim.

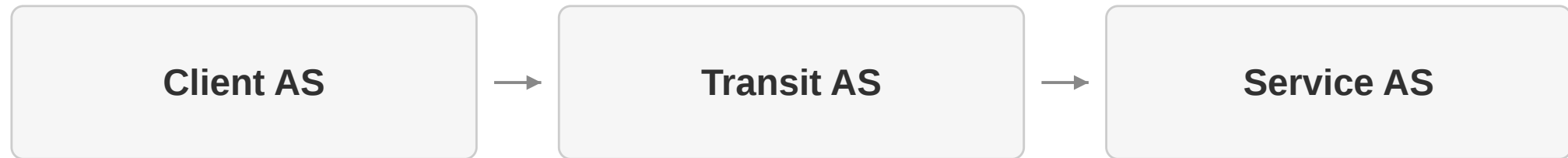
## Make evidence traceable.

| Artifact      | Record                                   |
|---------------|------------------------------------------|
| Software      | Version or commit and repository         |
| Dataset       | Release, identifier, access conditions   |
| Documentation | Exact page, version and section          |
| Figure        | Data and units, or source and adaptation |

Verify imported reference metadata. Mark synthetic examples.  
Citation does not replace reuse permission.



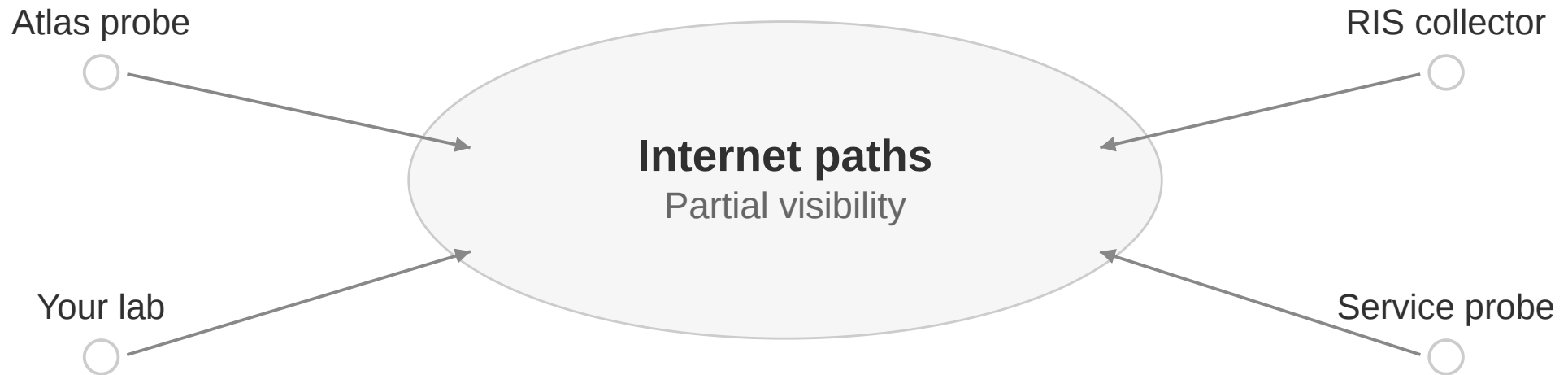
# Can public data answer your question?



Each observer sees only part of this path.

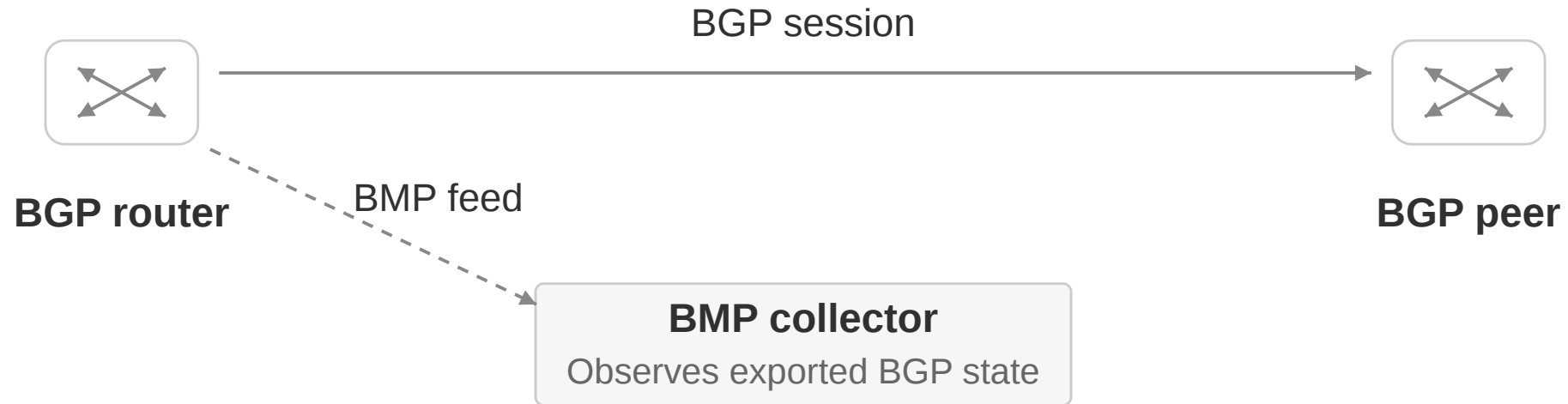
If your question extends beyond ReefNet, consider public observations.  
Their coverage and resolution must fit the claim.

# Public data gives us another vantage point



**RIPE Atlas:** active probes. **RIS:** public BGP observations.  
Neither covers every Internet path.

# BMP observes BGP state.



With router access, the **BGP Monitoring Protocol (BMP)** exports BGP state. It is a collection protocol, not the public RIS dataset.

# Read one public routing record.

Which routes were visible for AS8966? Query public RIS history:

```
curl -sS -G https://stat.ripe.net/data/routing-history/data.json \
  --data-urlencode resource=AS8966 \
  --data-urlencode starttime=2025-09-05T00:00:00 \
  --data-urlencode endtime=2025-09-07T00:00:00 \
  --data-urlencode include_first_hop=true \
  --data-urlencode min_peers=10
```

Inspect prefix intervals, origin/first-hop pairs and RIS peer counts.  
Route visibility is not traffic volume.

# What did the collector actually see?

Selected fields recorded in the original course material (18 September 2026):

```
origin + first hop: 1273 8966
prefix:            2.16.44.0/24
interval, UTC:     5 September 2025, 00:00-07:59:59
full_peers_seeing: 64
```

The pair includes a first hop. It does **not** mean two origin ASes.

The query omits routes seen by fewer than ten full-feed peers.

Does route visibility prove service availability? Does 64 mean 64 users?

# Make the public measurement reproducible

Keep the details another person needs to repeat it:

- UTC time window
- unaffected comparison
- query or measurement IDs
- filters and thresholds

Save them with the conclusion.

# Plan responsible measurement.

Prefer existing public data or your own lab target.

Before active probing, agree on target, authorization, rate, duration and stop condition. Follow the platform's rules.

For collected data, define fields, access, retention and deletion. Network records can expose communication patterns.

Collect only what the research question needs.

# Build an argument, then audit it.

Organize related work by the question the studies answer.

For each factual sentence, identify its role:

- our result → method and evidence
- prior result → original source
- protocol behavior → specification
- interpretation → inference and limitation

If you cannot classify it, check whether it belongs.



# Project checkpoints

| Date             | Bring                                    |
|------------------|------------------------------------------|
| 30 October 2026  | Question and initial source              |
| 13 November 2026 | Baseline, data access, open questions    |
| 27 November 2026 | Results or a concrete blocker            |
| 11 December 2026 | Comparison, outline, January plan        |
| 22 January 2027  | Read, write and hand in the paper review |
| 5 February 2027  | Presentation and reproducible evidence   |

Work in progress is welcome. **No extra graded reports.**

## Your question, your next step

Write your **Research Brief**: question, closest prior work, baseline, available evidence and biggest feasibility risk.

Use today's project time to check that you can access the evidence.

**13 November:** turn the question into a controlled experiment.  
Bring the baseline, data access and open questions.

# 13 November 2026

## Experimental design

Now the sampling question becomes a controlled comparison.

# Assign treatments to runs.

**Treatment:** the sampling policy.

**Outcome:** detection and delay.

**Experimental unit:** one reset run assigned a treatment.

Keep fault workload, probe budget and reset conditions equal.

Ten samples from one run are not ten independent experiments.

# Fix the comparison before running it

600-second runs with **60 probes per method**.

| Method     | One probe in each 10 s slot               |
|------------|-------------------------------------------|
| Periodic   | At the start                              |
| Randomized | Uniform random times with a recorded seed |

Use the same fault schedule for each pair.

Compare detection fraction and delay among detections.

# Specify the experiment boundary.

Use the smallest setting that can test the claim.

Record nodes and versions, link capacity and delay, addresses and policy, probe locations and control path.

Keep control reachable when the workload fails.

Document any dependencies shared by control and experiment.

Two identical diagrams can conceal different experiments.

# Virtual experiments have limits.

A virtual network can demonstrate mechanisms under controlled conditions.

It does not automatically reproduce:

- hardware queueing and forwarding-chip timing
- optical behavior
- production-scale convergence
- carrier failure domains
- Internet-wide path diversity

State which conclusion depends on virtualization.

## Measure an independent reference.

The detector under test cannot define the ground truth.

| Evidence                    | What it tells us                       |
|-----------------------------|----------------------------------------|
| Experiment controller       | When the fault was requested           |
| Independent reference probe | When traffic was actually affected     |
| Evaluated detector          | Whether and when it reported the fault |

A 100 ms reference interval limits onset-time resolution.



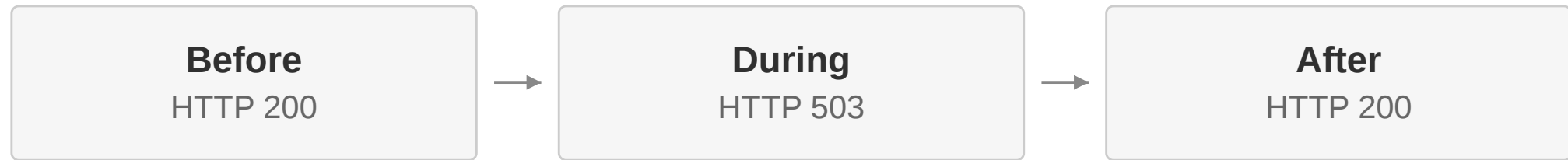
## Predict the observations.

ReefNet rejects customer IPv4 on export toward Lagoon Transit.  
Links, BGP sessions and management access remain up.

| Predict                                                       | Compare                             |
|---------------------------------------------------------------|-------------------------------------|
| Customer advertisement                                        | Lagoon vs Pacific                   |
| External reachability                                         | Both sources, both address families |
| Interface rate, ingress/egress discards and Linux queue drops | Routing fault vs congestion         |

Which observation would contradict your diagnosis?

## Control experiment: the endpoint fails



Same routes and interfaces. Different endpoint behavior.

An HTTP 503 response is not evidence of a routing failure.

## Pair conditions, randomize order.

Replay each fault schedule for both methods at equal probe cost.  
Reset between runs. Randomize which method runs first.

For jointly detected events: **difference = delay A – delay B.**  
Report misses separately.

Block by host or workload and record contention.  
Pairing does not remove clock error or unequal host load.

## Startup or steady state?

| Research question                | Treatment of startup             |
|----------------------------------|----------------------------------|
| How long until service recovers? | Startup is part of the outcome   |
| What is steady-state overhead?   | Apply a stated warm-up rule      |
| Does cache warming help?         | Compare defined cache conditions |

Cold caches and convergence are real. Excluding them changes the experiment.

## Decide how a run can fail

| Outcome                     | Stored result                           |
|-----------------------------|-----------------------------------------|
| Fault detected              | Detection flag and delay                |
| Fault missed                | Detection flag false, no invented delay |
| Probe process failed        | Instrumentation failure                 |
| Reset did not restore state | Invalid run with a recorded reason      |

Choose these categories before looking at which method wins.

# Turn the lab into an experiment



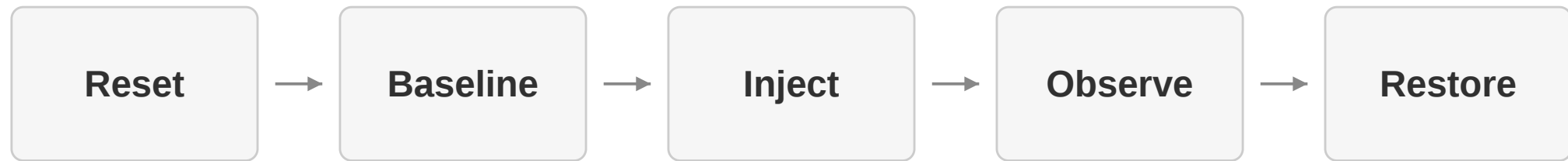
Pacific and IPv6 remain control observations.

Record the requested fault time and the first failed service check.  
Keep measuring until the service is restored.

Config-change time is not traffic-failure time.

# Define one experimental run

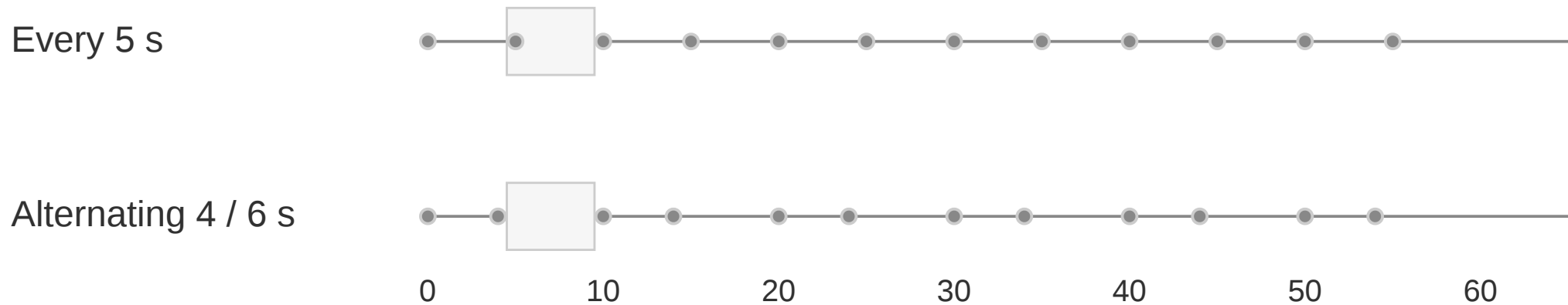
For the sampling comparison, repeat the same controlled sequence.



Run ID, seed, versions and raw results travel with the experiment.

A failed reset invalidates the comparison. Record it as a failed run.

## Same budget. Different gaps.



Twelve probes per minute. A five-second fault can fit between probes.

Timing model, not the 10 s study: **5 s mean spacing**, 120 fault phases.



## Compare detection and delay

Synthetic timing example: 120 onset phases, five-second faults.

| Probe spacing        | Detected | Missed | Median delay when detected |
|----------------------|----------|--------|----------------------------|
| Every 5 s            | 120      | 0      | 2.50 s                     |
| Alternating 4 s, 6 s | 108      | 12     | 2.25 s                     |

The lower median ignores the missed faults.

A real experiment must also account for request duration and scheduling.

## Task: compare detection delays

Separate arithmetic example: five synthetic pairs, delays in milliseconds.

| Method | Pair 1 | Pair 2 | Pair 3 | Pair 4 | Pair 5 |
|--------|--------|--------|--------|--------|--------|
| A      | 11     | 12     | 10     | 13     | 54     |
| B      | 17     | 18     | 16     | 17     | 18     |

**Four minutes:** Which method is usually faster? Which is more predictable?

Choose a summary. State what five pairs cannot establish.

## One outlier changes the mean

| Statistic | A        | B        |
|-----------|----------|----------|
| Mean      | 20.0 ms  | 17.2 ms  |
| Median    | 12 ms    | 17 ms    |
| Range     | 10-54 ms | 16-18 ms |

A is earlier in 4/5 pairs.

Its fifth run reverses the mean.

Small sample. This only illustrates the calculation.

# Five pairs. Plenty of confidence.

Exact arithmetic. Still a small sample.

## More samples do not fix bias.

| Source                  | What to examine                       |
|-------------------------|---------------------------------------|
| Sampling uncertainty    | Variation between runs                |
| Measurement uncertainty | Clocks and instrument resolution      |
| Bias                    | Unequal workload or target conditions |
| Instrumentation effects | Polling, capture and logging overhead |

More runs can reduce sampling uncertainty.  
They do not fix a bad clock or an unfair comparison.

## Four types of validity

| Type                   | Example problem                               |
|------------------------|-----------------------------------------------|
| Construct              | DNS success is treated as HTTP service health |
| Internal               | only one method runs on an overloaded host    |
| External               | one virtual topology stands for all networks  |
| Statistical conclusion | correlated samples are treated as independent |

Reproducibility does not remove bias.

# Task: review the experiment

| Method | Host            | Time      |
|--------|-----------------|-----------|
| A      | Idle            | Morning   |
| B      | Shared and busy | Afternoon |

Four minutes:

- which effects are mixed together?
- what would you hold constant?
- what would you randomize?
- what would you record?

# 27 November 2026

## Analysis and scientific argument

From paired runs to a defensible comparison.

The graph is not the result. Check the evidence behind it.



## Keep the mechanism visible.

**raw runs → detection counts → delays → uncertainty → claim**

Before aggregation, check matched conditions and individual traces.  
An average can hide short faults, load changes and a dominant run.

Keep missed events in the detection denominator.  
A detector does not get faster by deleting its misses.

# Follow one result back to its inputs



Follow one reported number all the way back.

Choose a number from your draft. Can another team reproduce it?

## Keep one row per run

| Run | Condition | Method     | Detected | Delay, ms |
|-----|-----------|------------|----------|-----------|
| 17a | 17        | Periodic   | Yes      | 820       |
| 17b | 17        | Randomized | No       | —         |
| 18a | 18        | Periodic   | Yes      | 240       |
| 18b | 18        | Randomized | Yes      | 510       |

Illustrative study log: one outage and 60 probes per run. Pair by condition ID.  
**A miss stays in the detection denominator.**

## Missing, missed and timed out are different

| Outcome            | Meaning                            |
|--------------------|------------------------------------|
| timeout after 2 s  | no completion observed by deadline |
| probe never ran    | service outcome unknown            |
| event not detected | miss for this detector             |
| run excluded       | rule + reason recorded             |

Dropping timeouts can make a broken service look fast.

# A fast median can hide missed faults

First compare coverage. This separate example has **20 faults**.

**Method A: 18 of 20 detected**

Median among detections: 1.0 s

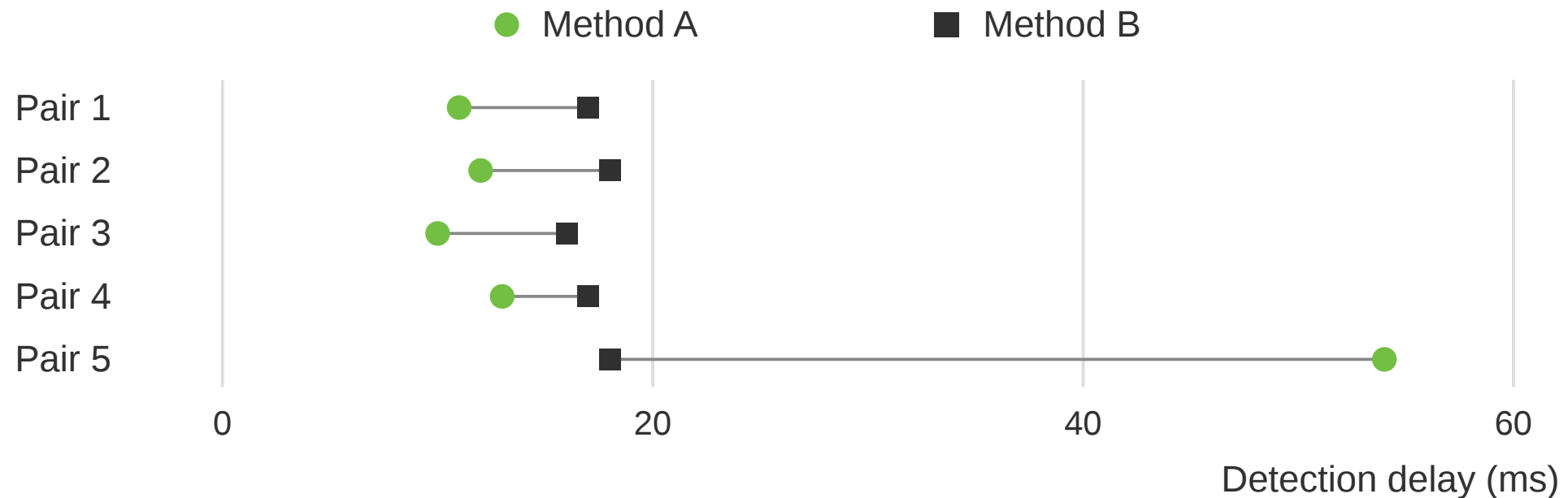
**Method B: 4 of 20 detected**

Median among detections: 0.5 s

Synthetic example

Compare the same detected events before claiming that B is faster.

## Compare within each pair.



Return to the five synthetic pairs from **13 November**, methods A and B.

## Work with paired differences.

| Pair | A, ms | B, ms | A – B, ms |
|------|-------|-------|-----------|
| 1    | 11    | 17    | –6        |
| 2    | 12    | 18    | –6        |
| 3    | 10    | 16    | –6        |
| 4    | 13    | 17    | –4        |
| 5    | 54    | 18    | +36       |

Mean: **+2.8 ms**. Median: **–6 ms**. A is earlier in four pairs.  
Five pairs illustrate a calculation, not a population result.

# What are we estimating?

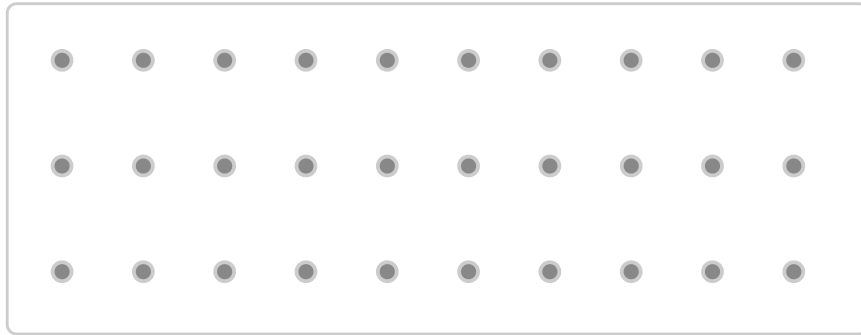
| Term       | Sampling-study example                             |
|------------|----------------------------------------------------|
| Population | Runs under the stated topology and workload        |
| Sample     | The runs actually performed                        |
| Parameter  | Mean detection-delay difference in that population |
| Estimator  | Mean of the observed paired differences            |

For delay comparisons, state the condition: both methods detected the event.

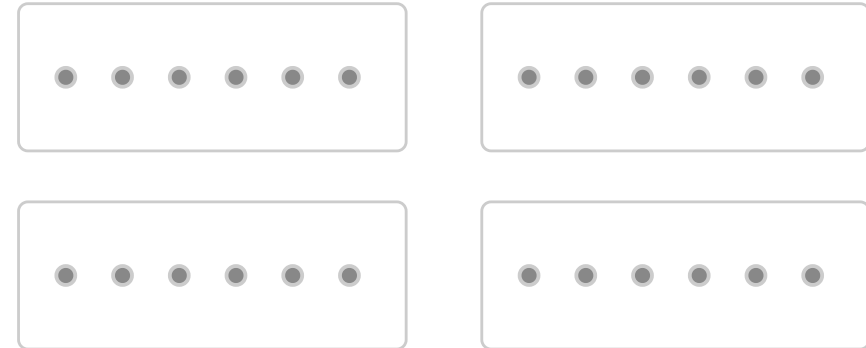


# Independent observations

**One run, many samples**



**Several independent runs**



The unit of replication comes from the experimental design.

Use independent runs or matched pairs as the uncertainty unit.

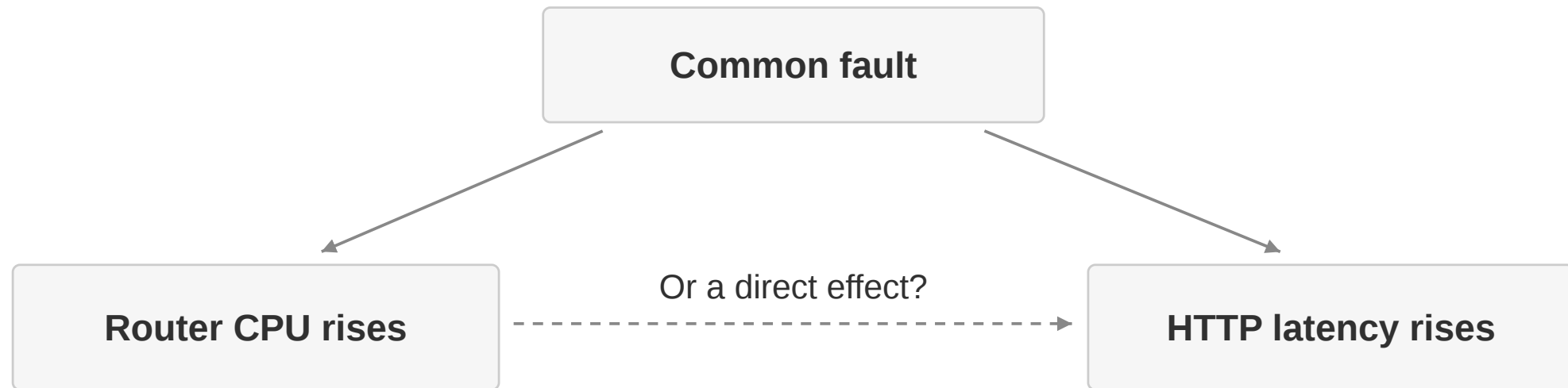
## What does 95% coverage mean?

A 95% confidence-interval procedure covers the parameter in 95% of repeated datasets **when its assumptions hold**.

**Bootstrap:** repeatedly resample whole runs or matched pairs.  
State the estimator, interval method and independence assumptions.

This is a property of the procedure, not a score for one interval.

## Task: assess a causal claim



A shared cause can create correlation without a direct causal link.

Which observation or intervention could distinguish these explanations?

## Is the effect useful?

**1,000 ms → 990 ms: 10 ms saved, a 1% reduction.**

Does it change a 30-second paging decision?  
Could it matter to a sub-second control loop?

Report units, direction, uncertainty and failure rate.  
A delay reduction is not a measured throughput gain.

# Unexpected results

If the expected method does not win:

- check the implementation
- check the measurement
- repeat where needed
- explain the conditions
- report the result honestly

Keep the result, including when your preferred method loses.

## Write from an evidence table.

Example for a paper draft:

| Research question | Evidence | Observation        | Limit           |
|-------------------|----------|--------------------|-----------------|
| RQ1               | Figure 2 | Lower median delay | Virtual testbed |
| RQ2               | Table I  | More target load   | One NOS         |

A results paragraph states the observation, points to evidence, quantifies the effect and bounds the interpretation.

Bring sample counts, units, uncertainty and plotting code.

# Practice reviewing before January

25 minutes, fictional study.

1. Your own review: 8 min
2. Compare in pairs: 7 min
3. Review together: 10 min

Use the same four criteria as the bonus review:

summary, method, strength/limitation, feasible revision.

# A study for review

Fictional claim: **randomized probing improves reliability.**

|                             | Periodic           | Randomized             |
|-----------------------------|--------------------|------------------------|
| Environment                 | Idle host, morning | Loaded host, afternoon |
| Probe budget                | Equal              | Equal                  |
| Detected faults             | 18 / 20            | 4 / 20                 |
| Median delay, detected only | 1.0 s              | 0.5 s                  |

The draft concludes: “Randomization halves delay.”  
What is supported, confounded or omitted?



## Review example

| Review   | Finding                                           |
|----------|---------------------------------------------------|
| Strength | Equal probe budget and reported misses            |
| Problem  | Host load changes with the method                 |
| Problem  | Delays compare different detected subsets         |
| Revision | Match conditions and report coverage beside delay |

Explain how each revision makes the conclusion more defensible.

# How is the course working for you?

Three minutes. Anonymous notes.

- What has helped your project?
- Where are you stuck?
- Pace: too slow / about right / too fast?
- One change that would help next time?

We discuss what I can still adjust.

# 11 December 2026

## Forecasts, control loops and agents

We have compared detectors. Now test a forecast before it drives a controller.

What evidence would justify acting on the prediction?

# Can we warn before saturation?

Separate synthetic workload, beyond the 50 Mbit/s BOB1 link.

Forecast **one second ahead**.

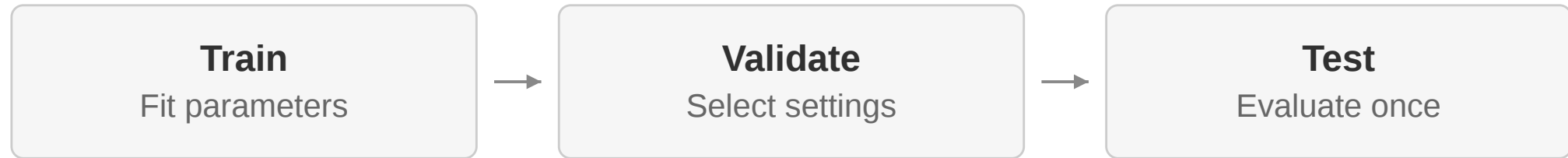
Start with **persistence**: use the latest observed load.

Compare it with ridge regression on past samples and periodic features.

Keep the same test data, horizon and missing-data rules.

A more complex model must beat the baseline where it matters.

## Keep the future out of training.



Train on past data. Test on later data.

Use only information available at prediction time.

# Traffic patterns change

**Drift** means the data or the predictive relationship changes.

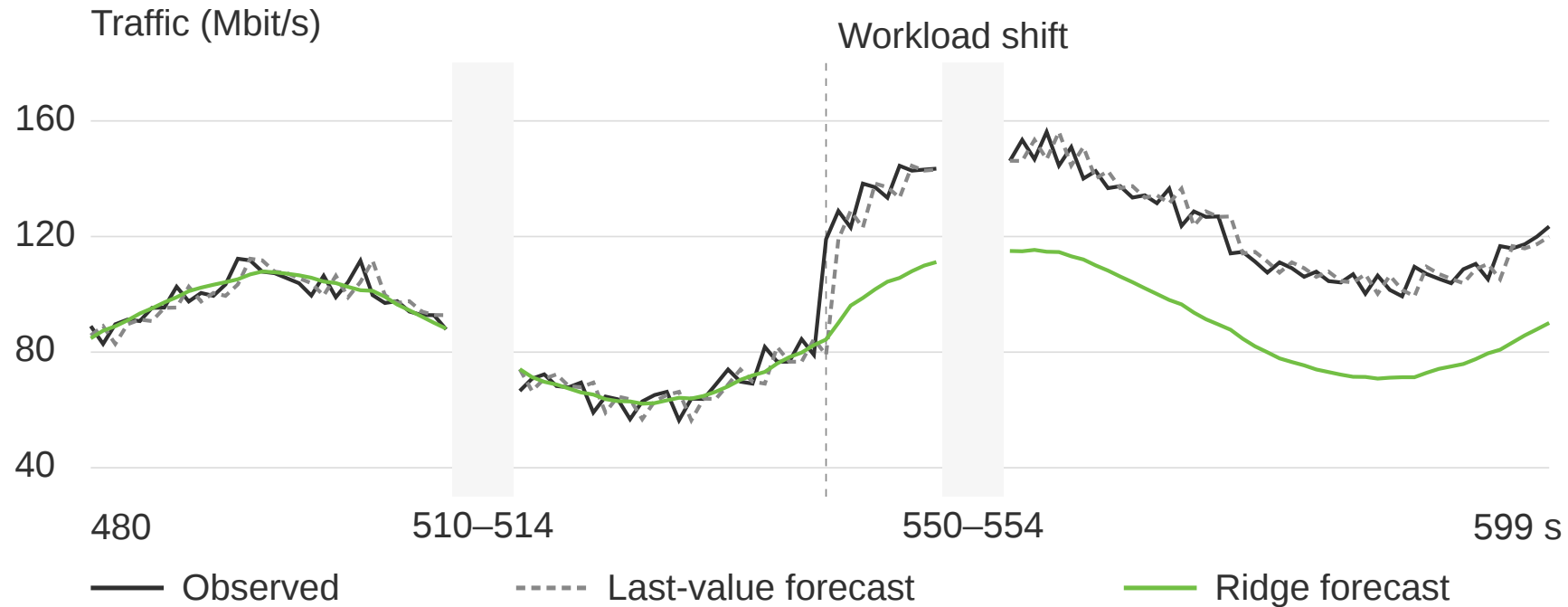
Network examples:

- route change moves traffic onto the link
- capacity upgrade changes utilization behavior

Watch prediction error.

Decide when to retrain, fall back, or stop automated action.

# Forecasts miss the workload shift



Synthetic, 1 s ahead. **110 of 120 slots evaluated.** Shaded gaps excluded.

## Score prediction and warning separately.

Same 110 test slots. Warning threshold: **125 Mbit/s**.

| Predictor   | MAE, Mbit/s | False alarms | Missed high-load slots |
|-------------|-------------|--------------|------------------------|
| Persistence | 4.74        | 3            | 3                      |
| Ridge       | 17.71       | 0            | 25                     |

**MAE:** mean absolute prediction error. Counts refer to time slots.  
Zero false alarms can hide a model that never warns.



# A controller cannot create capacity

Back to ReefNet. Illustrative load: what could a controller change?

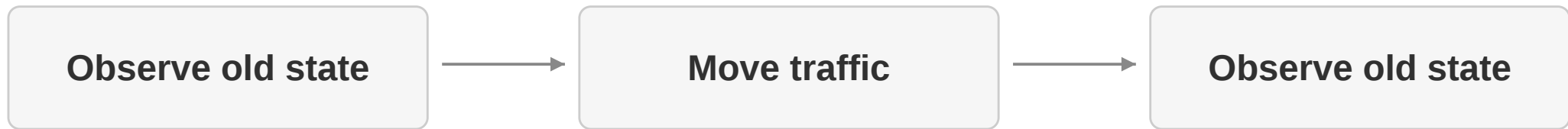
| Transit | Capacity  | Offered load |
|---------|-----------|--------------|
| Lagoon  | 50 Mbit/s | 60 Mbit/s    |
| Pacific | 50 Mbit/s | 30 Mbit/s    |

Moving 10 Mbit/s gives **Lagoon = 50, Pacific = 40 Mbit/s**.

There is still no headroom on Lagoon.

Would policy and traffic behavior allow that move?

# Delayed feedback



Wait for fresh evidence before moving again.

Acting twice on the same stale sample can overshoot the target.

# Put limits around the controller

Illustrative guardrails for the ReefNet controller:

```
allowed targets:  Lagoon, Pacific
maximum move:    10 Mbit/s per step
wait time:       30 s of fresh observations
stop if:         service degrades or data is stale
recovery:        previous allocation or manual handover
```

The same limits apply to rules, optimizers and LLMs.

# Can an agent troubleshoot the network?

Give it observations, a ticket, bounded tools and a known fault.

Keep the safety boundary outside the planner:  
permitted operations, permitted targets and independent verification.

Compare against a deterministic runbook on the same faults.  
Score the network state, not the confidence of the explanation.

# The write boundary

The tool enforces authorization and execution.



Authorization stays outside the planner.

The verifier checks independently. Where do we enforce the one-device limit?

# Limit what the agent can do

Example request for the BOB1 export-policy repair:

```
{  
  "device": "edge01.bob1.reefnet.test",  
  "bgp_group": "lagoon-v4",  
  "expected_override": "BLOCK-CUSTOMER-V4",  
  "action": "delete group export-policy",  
  "dry_run": true  
}
```

The tool enforces target scope, input checks, timeout and write permissions.

What should happen if the agent supplies another device or group?

# Task: check the claimed repair

Separate execution trace, after write approval

| Step            | Reported result                          |
|-----------------|------------------------------------------|
| read_state      | lagoon-v4 has BLOCK-CUSTOMER-V4 attached |
| delete_override | command completed                        |
| external_probe  | timeout                                  |
| Agent reply     | "The service is restored."               |

One minute: how should the evaluator score this run?

Which observation must remain independent of the agent's answer?

# The explanation has recovered.

Recovery is still unproven.



# Define safe success before scoring.

A repair succeeds only when:

1. the permitted target has the intended state
2. an independent service check passes
3. unrelated state is unchanged

Record refusal, failure, attempts, duration and cost.

Test stale state, uncertain writes and incorrect ticket assumptions.

# A benchmark needs known ground truth.

NIKA supplies reproducible troubleshooting scenarios and a tool interface.

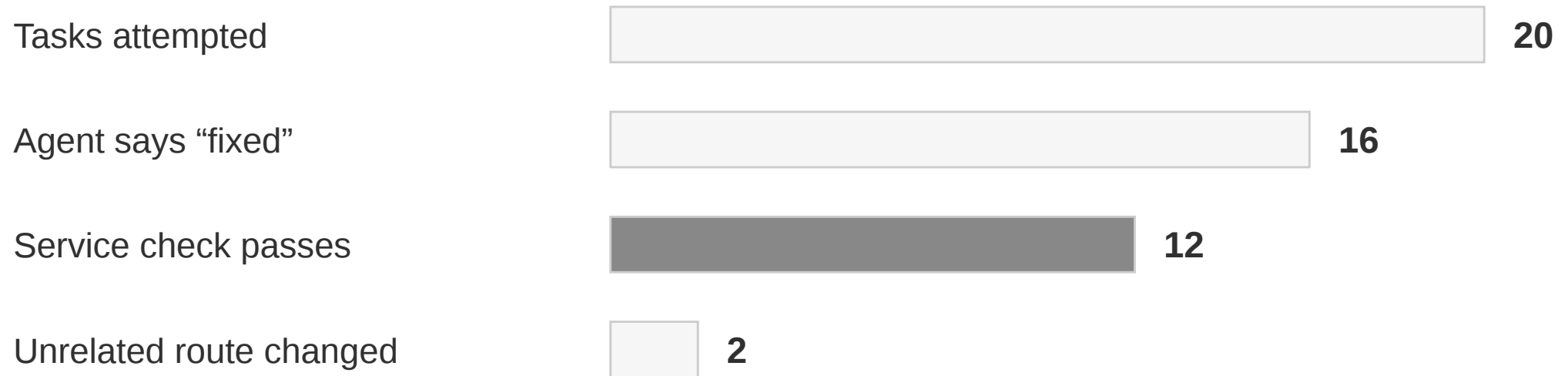
Separate **detection**, **localization** and **root-cause identification**.

These are different tasks and need separate scores.

Read the paper's fault set and evaluation conditions before comparing agents.

A benchmark result is bounded by those conditions.

# Task: assess the benchmark



Separate fictional repair benchmark: 20 attempts. Is 80% success justified?  
What must we know about overlap and unintended changes?

# 22 January 2027

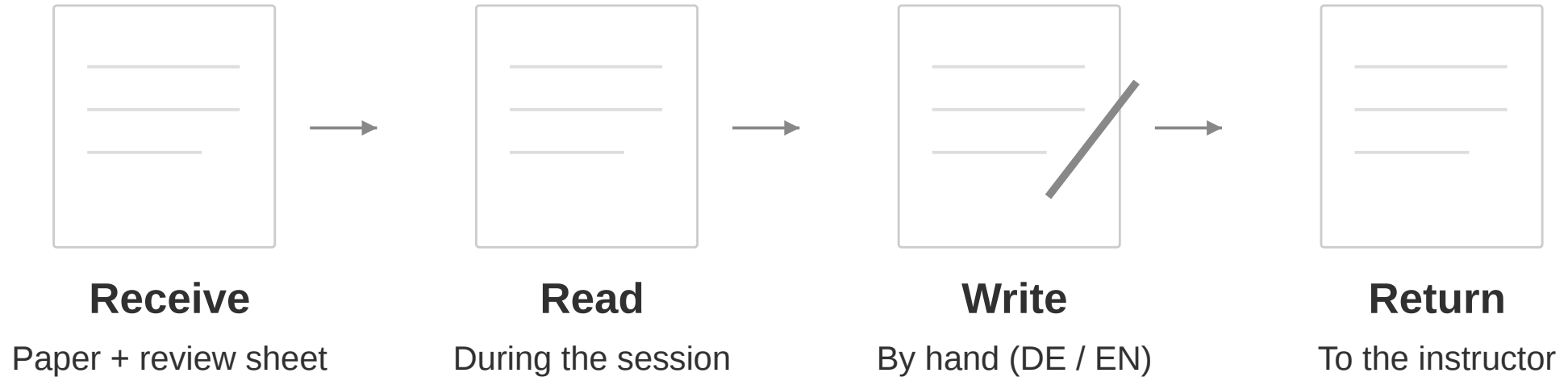
## In-class peer review

Read in class. Write **legibly by hand**.

German or English accepted.

Hand in today. The reviewed team receives a copy immediately.

# Receive, read, write, return.



Write your name and identify the reviewed paper.

The instructor keeps the original and makes a copy.

The reviewed team receives its copy **during this session.**

## Review criteria: calibrate your judgment

| Criterion | Enough                           | Not enough         |
|-----------|----------------------------------|--------------------|
| Summary   | Question and result              | Topic only         |
| Method    | Choice assessed with evidence    | "Unclear"          |
| Judgment  | Reasoned strength and limitation | Praise only        |
| Revision  | Feasible fix                     | "More experiments" |

Apply the same criteria we announced in October.

## Task: bound the conclusion

Two minutes. Reviewer discussion

Our system makes backbone services reliable.

Five route changes, one emulated topology. Four restore the health check.

What can the authors claim? Which result would you ask them to add?

## Detection delay and missed faults

Return to the fictional study from November.

| Method | Detected faults | Median delay among detections |
|--------|-----------------|-------------------------------|
| A      | 18 / 20         | 1.0 s                         |
| B      | 4 / 20          | 0.5 s                         |

The draft calls B "the faster detector".

Write one review sentence. Name the hidden condition and ask for context.



## Task: write a review comment

Two minutes. Fictional methods section

We repeated each experiment several times and removed outliers.

Name the missing information and ask for a concrete correction.

A useful review explains how the issue affects the conclusion.

# Task: rewrite the claim

"Our method improves network reliability by 40%."

Evidence actually shown:

- one fault family
- one emulated topology
- mean detection delay

Draft a bounded claim. Mark the missing comparator and numbers.  
Do not carry over **40%** without supporting results.

Name one experiment needed for a broader claim.

## Now review the assigned paper

Read the paper in class. Write your one-page review **legibly by hand**.

Check topology, versions, workload and measurements.

Are baselines, sample counts, failures and exclusions explicit?

Name a strength, a limitation and a feasible revision.

Explain how each issue affects the conclusion.

## Before handing in the review

Write **one legible page**, in German or English.  
Include your name and the reviewed paper.

Hand it to the instructor on **22 January 2027, before leaving**.

The instructor keeps the original and makes a copy.  
The reviewed team gets its copy **during this session**.

# Use the review to revise your paper

After receiving your copy, switch to the author role.

For each comment: clarify, add evidence, fix the analysis or narrow the claim.

Example response:

We reran both methods at equal budget, added Figure 3, and narrowed the abstract and conclusion.

If you disagree, explain why and point to the relevant result.

# 5 February 2027

## Presentation and reproducibility

Show the result. Defend the choices. Let someone else reproduce the evidence.

# Presentation and reproduction

**8 minutes presentation + 7 minutes questions**

Cover:

- problem
- comparison
- main result
- limitation

Then trace one result back to its data and regenerate it.

The presentation itself has no grade weight. Use it as the last technical check.

# Test the artifact handover.

Another team opens the repository.

Can they find:

- exact version used in the paper
- required inputs and access
- command for the main figure
- expected output
- known limits

"Works on my laptop" means the laptop is still an undocumented dependency.



# Task: complete the instructions

Two minutes. Incomplete README

Install the dependencies. Start the network.  
Run the experiment. Plot the results.

Choose three missing details that would stop another team.

Give concrete replacements, including how they would recognize success.

# Publication-quality figures

For the final paper, audit every exported figure.

- use readable labels at final paper size
- keep units explicit
- use consistent terminology
- plot the underlying data instead of dashboard screenshots
- check grayscale readability
- remove decoration that does not encode information

Check the exported PDF at its final page size.

# Before submitting

Check the argument and the artifact together.

- references support the nearby claims
- figures come from traceable data
- failed runs and exclusions are visible
- limitations bound the conclusion
- AI use is disclosed where required
- every author has read the final paper

Paper and artifact must describe the same study.

# Submission package

- **PDF:** opens correctly, six pages including references, correct authors
- **Artifact:** accessible to the examiner, tagged version matches the paper
- **Declaration:** individual contributions are complete
- **AI use:** disclosure is present where required

Test the links and files before **12 February 2027, 23:59**.

# AI disclosure

**OpenAI Codex assisted with this slide deck.**

Assistance covered structure, wording, diagrams, teaching examples and checks of formatting and rendered output.

# Thanks

## Network Management and Monitoring

Questions, results or a packet capture worth discussing?